

Ioana Cristina Plajer

Curs

Contents

1	Introducere	7
1.1	Generalități	7
1.2	Biblioteca STL	8
1.2.1	Containere simple	8
1.2.2	Containere secvențiale	13
1.3	Observații și concluzii	23
2	Structuri de date elementare	25
2.1	Liste înlanțuite	25
2.1.1	Liste simplu înlanțuite	25
2.1.2	Liste dublu înlanțuite	29
2.1.3	Utilizarea listelor înlanțuite	32
2.1.4	Liste simplu și dublu înlanțuite în STL	33
2.1.5	Liste înlanțuite în alte limbaje de programare	35
2.2	Stive și cozi	35
2.2.1	Stiva	35
2.2.2	Coadă - <i>queue</i>	38
2.2.3	deque	42
2.2.4	Probleme	43
2.2.5	Aplicații ce utilizează stive și cozi	44
3	Tabele de dispersie	49
3.1	Tabele cu adresare directă	49
3.2	Tabele de dispersie - <i>Hash Tables</i>	51
3.2.1	Problema coliziunilor	51
3.2.2	Operațiile uzuale	53
3.2.3	Analiza complexității	54
3.3	Metode de dispersie	55
3.3.1	Metoda diviziunii	55
3.3.2	Metoda multiplicării	56
3.3.3	Dispersie universală	56
3.3.4	Adresare deschisă	57
3.3.5	<i>Perfect hashing</i>	63

3.4	Tratarea cheilor de tip șir de caractere	63
3.4.1	Funcții elementare de transformare a unui <i>string</i> într-un număr natural	63
3.4.2	Funcții polinomiale de dispersie	64
3.4.3	Algoritmul FNV	64
3.5	Tabele de dispersie în STL	65
4	Arbori	67
4.1	Arbori oarecare	67
4.1.1	Reprezentarea arborilor	69
4.1.2	Parcurgerea arborilor	71
4.1.3	Parcurgerea arborilor binari	72
4.2	Probleme și exerciții	75
5	Heap. Coadă de priorități	77
5.1	Heap	77
5.2	Algoritmul de sortare <i>HeapSort</i>	80
5.3	Coadă de priorități	82
6	Arbori binari de căutare	87
6.1	Noțiuni de bază	87
6.2	Operații într-un arbore binar de căutare	88
6.2.1	Căutarea binară	88
6.2.2	Minimul dintr-un arbore de căutare	89
6.2.3	Succesorul binar	89
6.2.4	Inserarea unui nod	90
6.2.5	Ștergerea unui nod	92
6.3	Exerciții rezolvate	95
7	Arbori AVL	99
7.1	Noțiuni generale	99
7.2	Operații într-un arbore AVL	102
7.2.1	Rotația	102
7.2.2	Insertia	103
8	Arbori Roșu Negru	111
8.1	Noțiuni generale	111
8.1.1	Insertia într-un arbore roșu-negru	112
8.1.2	Ștergerea dintr-un arbore roșu-negru	118
9	Îmbogățirea structurilor de date	125
9.1	Noțiuni generale	125
9.2	Arbori pentru statistici de ordine	126
9.3	Îmbogățirea arborilor RN - teorema complexității	132

9.4	Arbori de intervale	132
10	B Arbori	135
10.1	Elemente de bază	135
10.2	Operații	139
10.2.1	Căutarea unei chei	139
10.2.2	Inserarea unei chei	140
10.2.3	Eliminarea unei chei	143
11	Arbori quad	151
11.1	Quadtrees pentru regiuni	151
11.1.1	Adiacență și vecinătate	152
11.1.2	Determinarea vecinilor adiacenți după o direcție verticală sau ori- zontală	155
11.1.3	Determinarea vecinilor diagonali	158
11.1.4	Linear Quadtrees	160
11.2	Point Region Quad Trees - PR-Quadtrees	162
11.2.1	Căutarea unui nod	163
11.2.2	Inserarea într-un PR-arbore	164
11.2.3	Construcția unui PR-arbore quad	166
11.2.4	Eliminarea unui punct P	166
11.2.5	Înălțimea unui PR-QuadTree	168

Chapter 1

Introducere

1.1 Generalități

Structurile de date reprezintă colecții de elemente (date) în care există anumite relații structurale. Fiecare element dintr-o astfel de structură are o anumită poziție în cadrul structurii și există un mod specific de acces al acestui element.

În informatică, structurile de date sunt utilizate pentru memorarea și manipularea eficientă cu calculatorul a unor mulțimi dinamice de date. Denumirea de *mulțimi dinamice* provine de la faptul că acestea pot suferi modificări prin operații de inserare și/sau extragere de elemente.

Reprezentarea elementelor unei mulțimi

În anumite situații, de exemplu în cazul listelor înlănțuite sau a arborilor, un element al unei mulțimi dinamice de date este reprezentat printr-o structură / un obiect, care dispune de mai multe câmpuri, dintre care unele conțin informații de interes, iar altele pointeri către alte elemente ale mulțimii de date (următor, precedent în cazul listelor înlănțuite sau părinte, fii în cazul arborilor).

Unele structuri presupun existența unui câmp *cheie*, care identifică în mod unic fiecare element.

În cazul cheilor provenind din mulțimi bine ordonate (ex: numere întregi, caractere, șiruri de caractere) pot fi efectuate operații de ordonare / sortare sau de determinare a elementului cu cheia minimă respectiv maximă din mulțimea dinamică păstrată în structura de date respectivă.

Operațiile efectuate asupra unei mulțimi dinamice de date pot fi de două tipuri:

1. **Cereri** - operații care extrag informații din mulțimea de date, cea mai importantă fiind *căutarea*
2. **Operații de modificare a mulțimii** - inserarea / ștergerea unui element.

Există un număr mare de diferite structuri de date, fiecare având proprietăți specifice și fiind adaptate rezolvării unui anumit tip de probleme. Alegerea structurii de date

potrivite în cadrul unui program ar trebui să țină cont de rezultatul dorit, de complexitatea de timp și memorie pe care le presupune utilizarea acelei structuri pentru problema dată.

În cursurile care urmează vor fi prezentate o parte dintre cele mai des utilizate structuri de date, împreună cu exemple de aplicații, algoritmi în pseudo-cod, indicații de implementare în C++ precum și o serie de probleme propuse spre rezolvare.

1.2 Biblioteca STL

Biblioteca STL din C++ conține o serie de clase șablon - *template* - care implementează structuri de date uzuale, precum stive, cozi, liste înlănțuite, cozi de prioritate și altele, dar și algoritmi, precum sortare și căutare, funcții și iteratori.

Pentru implementarea structurilor de date se utilizează containere, adaptorii de containere și iteratori.

Șabloanele permit utilizarea structurilor de date indiferent de tipul datelor, pe care dorim să îl memorăm.

Containerele sunt cele care stochează efectiv informația, de exemplu tablourile - *arrays* - sau listele înlănțuite, iar adaptorii de containere au la bază un container simplu, cu anumite reguli de acces specifice (adaptează un container la cerințele problemei). Astfel de adaptorii sunt de exemplu stivele.

Iteratorii permit o iterare uniformă prin elementele structurilor, fără ca utilizatorul să fie nevoit să cunoască modul de acces al elementelor. Iteratorii sunt reprezentați prin pointeri către elemente ale structurii. Cei mai importanți iteratori sunt *begin()* și *end()*, care permit accesul la începutul, respectiv după ultimul element al structurii. Iterarea se face apoi cu ajutorul operatorilor de incrementare ++ și decrementare --.

Algoritmii implementați în STL au ca parametri containere din STL și operează pe acestea. În figura 1.1 sunt reprezentate tipurile principale de containere implementate în STL.

Prezentăm în continuare containerele simple și containerele secvențiale. Apoi, la fiecare capitol care descrie o structură implementată și în STL, va fi descrisă pe scurt clasa corespunzătoare, împreună cu exemple de utilizare.

1.2.1 Containere simple

tuple

```
template <class T1, class T2, class T3, ..> struct tuple;
```

Această clasă *template* definește un container care grupează două sau mai multe valori, care pot fi de orice tip, într-o singură variabilă.

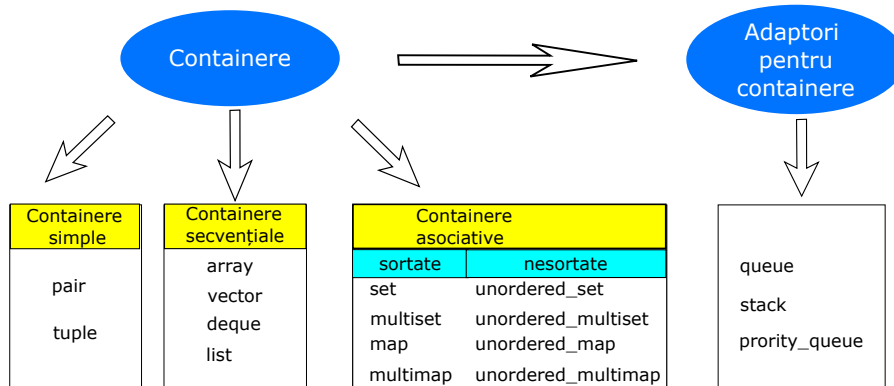


Figure 1.1: Containerele principale din STL

Clasa *tuple* dispune de o funcție **swap()**, care permite schimbarea valorilor câmpurilor cu alte valori de același tip, de funcția **get()** care returnează valorile tuplu-lui, precum și de un operator de atribuire.

Există de asemenea câteva funcții ne-membre care operează cu obiecte de tip *tuple*, dintre care cea mai importantă este:

```
tuple<T1,T2, T3...> make_tuple (T1 x, T2 y, T3 z, ...);
```

care creează un obiect de tip *tuple* pe baza a tuturor valorilor *x, y, z, ...*

Exemplu - declararea variabilelor de tip tuple

```
#include<iostream>
#include<tuple>

int main()
{
    // declararea obiectului pers1 de tipul tuple,
    // care retine un sir de caractere si doua
    // numere intregi = id si varsta
    std::tuple<std::string, int, int> pers1;

    // declararea obiectului pers2 si initializarea
    // acestuia
    std::tuple<std::string, int, int> pers2(
        "Ion", 12, 23);

    //copierea variabilei pers1 in variabila pers3
    std::tuple<std::string, int, int> pers3(pers1);

    //initializarea cu lista de initializare
    std::tuple<std::string, int, int> pers3 =
        {"Ana", 110, 32};

    return 0;
}
```

Exemplu - accesul la membrii unui tuplu

```
#include<iostream>
#include<tuple>

int main()
{
    int id, varsta;
    std::string nume;

    std::cout << "Nume=";
    std::getline(std::cin, nume);
    std::cout << "Id="; std::cin >> id;
    std::cout << "Varsta="; std::cin >> varsta;

    //crearea unui tuplu utilizand valorile citite
    pers1 = std::make_tuple(nume, id, varsta);
    //alternativa a functiei make_tuple
    pers1 = { nume, id, varsta };

    //accesarea campurilor variabilei cu get<indice>
    std::cout << "Nume:"<<std::get<0>(pers1)<< "\n";
    std::cout << "Id:" << std::get<1>(pers1)<< "\n";
    std::cout << "Varsta:" << std::get<2>(pers1);

    return 0;
}
```

```
template <class T1, class T2> struct pair;
```

Această clasă *template* definește un container care grupează două valori, care pot fi de orice tip, într-o singură variabilă. Clasa este un caz particular de tuplu și dispune de doi membri publici: *first* și *second*, care permit accesul la cele două valori, precum și funcții de manipulare asemenea tipului de date *tuple*.

Exemplu - declararea variabilelor de tip pair

```
#include <iostream>

int main()
{
    //declararea obiectului pers1 de tipul pair,
    //care retine un sir de caractere
    //si un numar intreg
    std::pair<std::string, int> pers1;

    //declararea obiectului pers2 si
    //initializarea acestuia cu valorile "Ion" si 124
    std::pair<std::string, int> pers2("Ion", 124);

    //copierea variabilei pers1 in variabila pers3
    std::pair<std::string, int> pers3(pers1);

    //initializarea cu lista de initializare
    std::pair<std::string, int> pers4 = {"Ana", 110};
    return 0;
}
```

Exemplu - accesul la membrii variabilelor de tip pair

```
#include<iostream>

int main()
{
    std::pair<std::string, int> pers1("Maria", 125);
    int id;
    std::string nume;

    std::cout << "Nume=";
    std::getline(std::cin, nume);
    std::cout << "Id="; std::cin >> id;

    //crearea unei perechi utilizand valorile citite
    pers1 = std::make_pair(nume, id);

    //alternativa a functiei make_pair,
    //utilizand lista de initializare
    pers1 = { nume, id };
    std::cout << "Prima persoana" << "\n";

    //accesarea campurilor variabilei
    std::cout << "Nume: " << pers1.first << "\n";
    std::cout << "id:" << pers1.second << "\n\n";

    //accesarea campurilor cu get
    std::cout << "A doua persoana" << "\n";
    std::cout << "Nume:" << std::get<0>(pers2) << "\n";
    std::cout << "Id:" << std::get<1>(pers2) << "\n";

    return 0;
}
```

1.2.2 Containere secvențiale**Vector**

```
template<class T, class Alloc = allocator<T>>class vector;
```

Clasa *vector* definește un container care stochează o secvență de elemente de tip *T*, tipul trebuie stabilit atunci când se declară un astfel de container. Elementele păstrate

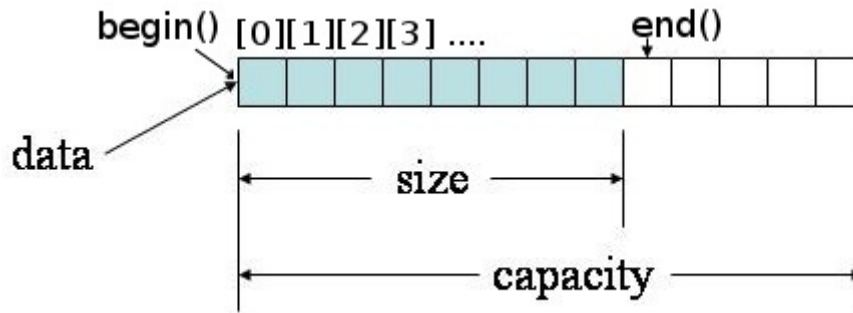


Figure 1.2: Un vector are stocate elementele în zone de memorie adiacente. Indicii încep de la 0. Size reprezintă numărul de elemente stocate în vector, iar capacity numărul de poziții alocate. Begin și end sunt iteratori.

în vector se stochează în zone de memorie succesive, astfel încât accesul se poate realiza prin poziție.

Un container de tip vector are dimensiune variabilă și utilizează intern alocare dinamică de memorie. atunci când se adaugă un element nou la vector, dacă memoria alocată nu este suficientă, se realocă o zonă de memorie nouă, se copiază elementele din zona veche și se dealocă zona veche de memorie. Aceste acțiuni se realizează automat și sunt gestionate de către *allocator*.

Deoarece operațiile de realocare și copiere sunt costisitoare din punct de vedere al timpului de execuție, acest lucru nu se realizează la fiecare nouă adăugare a unui element, ci allocatorul alocă de obicei ceva mai multă memorie decât este necesară pe moment. Numărul de poziții alocate la fiecare adăugare depinde de obicei de numărul de adăugări efectuate până la momentul respectiv și depinde de implementările diferitelor biblioteci.

Astfel numărul de elemente stocate efectiv în vector poate fi diferit de numărul de poziții alocate, obiectul dispunând de două funcții membre:

- **size()** - returnează numărul de elemente stocat în vector
- **capacity()** - returnează numărul de poziții alocate pentru vector

Alocarea memoriei necesare pentru stocarea elementelor din vector se poate realiza prin apelul consecutiv al funcției care inserează elemente în container sau prin utilizarea funcției **reserve(nr:int)** care alocă *nr* zone de memorie de tipul elementelor din vector. În figura 1.2 este reprezentat grafic un obiect de tip vector, iar în figura 1.3 este prezentat un exemplu de cod pentru redimensionarea unui vector împreună cu rezultatul execuției acestui cod sursă.

Un vectorul care nu conține niciun element se numește *vector vid*, iar această proprietate este verificată prin apelul funcției **empty()**, care întoarce un răspuns boolean.

Dintr-un vector care conține mai multe elemente putem obține un vector vid utilizând funcția **clear()**, care șterge toate elementele din vector, fără a elibera zonele de memorie

```
#include<iostream>
#include<vector>

int main()
{
    std::vector<int> numere;
    numere.resize(10);
    std::vector<int> numere2;
    numere2.reserve(10);
    std::cout << "numere:\n";
    std::cout << "size =" << numere.size();
    std::cout << std::endl;
    std::cout << "capacity =" << numere.capacity();
    std::cout << std::endl;

    std::cout << "\n\n" << "numere2:\n";
    std::cout << "size =" << numere2.size();
    std::cout << std::endl;
    std::cout << "capacity =" << numere2.capacity();
    std::cout << std::endl;
}
```

```
numere:
size =10
capacity =10

numere2:
size =0
capacity =10

D:\Facultate\SD\CursMate\Test
Press any key to close this
```

resize(nr) - alocă memorie și adaugă 0 de nr ori.

reserve(nr) - alocă memorie.

Figure 1.3: Cod sursă împreună cu rezultatul execuției.

ocupate de acestea.

Clasa *vector* dispune de o serie de constructori, care permit declararea și inițializarea obiectelor de tip vector. Câteva exemple sunt prezentate mai jos:

```
//declararea unui vector vid de nr întregi
std::vector<int>first;

//vector ce contine elementul 4 de 100 de ori
std::vector<int>second(4, 100);

//vector ce contine elementele {1, 2, 3}
std::vector<int>third = {1, 2, 3};

//vector ce contine elementele lui second
std::vector<int>fourth(second.begin(),second.end());

//vector copie a lui third
std::vector<int>fifth(third);
```

Observație: În exemplul de mai sus, vectorul *fourth* se obține prin *iterare* prin vec-

torul *second*, cu ajutorul iteratorilor *begin()* și *end()*. La finalul secțiunii este prezentată pe scurt noțiunea de iterator împreună cu un exemplu pentru vector.

Adăugarea unui element la sfârșitul unui vector se realizează cu ajutorul funcției *push_back*. Adăugarea unui element cu *push_back* determină creșterea dimensiunii vectorului cu o unitate. Dacă prin aceasta se depășește memoria alocată, adică capacitatea vectorului, atunci se produce realocare de memorie. Argumentul funcției trebuie să fie un obiect de tipul celor stocate în vector.

Ștergerea unui element la sfârșitul unui vector se realizează cu ajutorul funcției **pop_back**. Această funcție este apelată fără parametri și determină modificarea lungimii vectorului, dar nu și a capacității acestuia.

```
std::vector<int> vector = { 1,2,3 };
// stergerea elementului de pe ultima pozitie
vector.pop_back();

// inserarea valorii 4 la finalul sirului de valori
vector.push_back(4);
// vectorul contine valorile: { 1, 2, 4 }
```

Un exemplu de cod sursă pentru adăugarea unui element la sfârșitul unui vector împreună cu rezultatul execuției este prezentat în figura 1.4.

Accesul elementelor dintr-un container de tip vector este gestionat cu ajutorul operatorului `[]`, precum și a unor funcții membre:

- **operatorul `[]`** - returnează valoarea elementului de pe o poziție dată ca parametru. În momentul utilizării acestui operator este necesar să ne asigurăm de faptul că în vector există poziția solicitată prin parametru, deoarece operatorul nu verifică acest aspect, fapt ce poate cauza un comportament nedefinit.
- **funcția `at()`** - returnează valoarea elementului de pe o poziție dată ca parametru. Această funcție verifică și semnalează printr-o excepție de tipul *out_of_range* dacă poziția se află în afara intervalului de poziții valide
- **funcția `front()`** - returnează referința elementului de pe prima poziție
- **funcția `back()`** - returnează referința elementului de pe ultima poziție

The screenshot shows a C++ program in Visual Studio Code. The code defines a vector named 'numere' and pushes back elements from 1 to 10. It then prints the size and capacity of the vector before and after the push_back operation. The debug console shows the output: before push_back, size is 0 and capacity is 0; after push_back, size is 10 and capacity is 13. A tooltip explains that size() returns the number of elements and capacity() returns the allocated memory size.

```
#include<iostream>
#include<vector>

int main()
{
    std::vector<int> numere;
    std::cout << "before push_back:\n";
    std::cout << "size =" << numere.size();
    std::cout << std::endl;
    std::cout << "capacity =" << numere.capacity();
    std::cout << std::endl;

    for (int index = 1; index <= 10; index++)
        numere.push_back(index);

    std::cout << "\n\n after push_back:\n";
    std::cout << "size =" << numere.size();
    std::cout << std::endl;
    std::cout << "capacity =" << numere.capacity();
    std::cout << std::endl;
}
```

before push_back:
size =0
capacity =0

after push_back:
size =10
capacity =13

D:\Facultate\SD\CursMate\TestInf
Press any key to close this wind

size() - numărul de element din vector.
capacity() - memoria alocată - nr de poziții disponibile

Figure 1.4: Cod sursă pentru adăugarea de elemente la sfârșitul unui vector împreună cu rezultatul execuției.

Exemplu:

```
#include<iostream>
#include <vector>
int main()
{
    std::vector<int> vector = { 1, 2, 3, 4, 5 };

    std::cout << "Primul element: "
              << vector.front() << "\n";
    std::cout << "Ultimul element: "
              << vector.back() << "\n";
    std::cout << "Al doilea element: "
              << vector[1] << "\n";
    std::cout << "Al patrulea element: "
              << vector.at(3) << "\n";

    return 0;
}
```

Parcurgerea elementelor dintr-un vector se poate realiza cu ajutorul instrucțiunii

repetitive *for* într-unul din următoarele moduri:

- **metoda clasică** - prin intermediul *for*-ului variabila *poz* reține, la un moment dat, fiecare poziție din vector, iar elementele containerului pot fi accesate cu ajutorul operatorului `[]` sau al funcției `at()`.

```
for (int poz = 0; poz < vector.size(); poz++)
    std::cout << vector[poz] << " ";
```

- **range base for** - variabila *element* va reține, pe rând, toate elementele vectorului, această metodă nepermițând o parcurgere parțială a containerului. Din punct de vedere sintactic *for*-ul este format din: tipul elementelor din vector, urmat de numele unei variabile ce va reține pe rând fiecare element din container, apoi operatorul `:` și numele vectorului (containerului) parcurs.

```
for(int element : vector)
    std::cout << element << " ";
```

- **utilizând iteratori** - variabila *it* reține, la un moment dat, un pointer către fiecare element din vector. Sintactic observăm o manieră asemănătoare cu cea a *for-ului clasic*, doar că iteratorul reprezintă un pointer către elementul curent, pentru a accesa valoarea stocată la adresa reținută de pointerul *it* trebuie folosit operatorul `*`, cum se vede și în exemplu.

```
std::vector<int>::iterator it;
for(it = vector.begin(); it < vector.end(); it++)
    std::cout << *it << " ";
```

Array

```
template < class T, size_t N > class array;
```

Clasa *array* definește un container generic care stochează o secvență de elemente de tip *T*, asemenea unui *vector*, principala diferență a celor două containere fiind variabilitatea dimensiunii. Array-ul este un container caracterizat prin dimensiunea fixă de *N* poziții. Tipul *T* al elementelor din array, precum și capacitatea acestuia *N*, trebuie stabilite atunci când se declară un astfel de container.

Pentru manipularea și parcurgerea elementelor dintr-un array se pot utiliza funcții prezentate în cadrul secțiunii *vector*, cu următoarele observații:

-
- lungimea array-ului este egală cu capacitatea acestuia, iar această valoare este determinată folosind funcția **size()**
 - alocarea de memorie se realizează în momentul declarării variabilei, motiv pentru care nu există funcția **reserve()**
 - nu putem apela funcții de inserare sau de ștergere a unui element în/din container, dimensiunea array-ului fiind fixă.

```
#include<iostream>
#include <array>

int main()
{
    //declararea unui array de lungime 7,
    //ce contine valori default de tip float
    std::array<float , 7> first;

    //declararea unui array de lungime 10,
    //ce contine valorile intervalului [1,6]
    //pe primele 6 pozitii
    std::array<int , 10> second = {1, 2, 3, 4, 5, 6};

    //declararea unui array copie a lui second
    std::array<int , 10> third(second);

    std::cout << "Lungimea celui de-al doilea array:"
        << second.size() << "\n"; // lungime 10

    if (second.empty())
        std::cout << " vectorul este vid";
    else
        std::cout << " vectorul nu este vid";

    for (int poz = 0; poz < second.size(); poz++)
        std::cout << second[poz] << " ";
    //se va afisa: {1, 2, 3, 4, 5, 6, 0, 0, 0, 0}

    third.at(6) = 7;
    for(int element : third)
        std::cout << element << " ";
    //se va afisa: {1, 2, 3, 4, 5, 6, 7, 0, 0, 0}

    return 0;
}
```

Iteratori

Iteratorii sunt obiecte care *pointează* spre un element aflat într-o colecție iterabilă / container și care permit iterarea prin acea colecție utilizând operatori de incrementare (++) sau decrementare (-). Utilizarea iteratorilor contribuie la o eficientizare a timpului de execuție, întrucât se lucrează cu pointeri către elementele din container, astfel copierea

acestora este evitată.

Un alt avantaj oferit de către iteratori este acela că, utilizatorii nu sunt nevoiți să știe, cum trebuie iterat printr-un container, de exemplu listă înlanțuită sau arbore binar (map), acest lucru fiind gestionat de către iterator. O parcurgere, din punct de vedere al codului scris este absolut similară pentru oricare tip de container.

Containerele din STL conțin suport ce permite parcurgerea acestora utilizând iteratori cu ajutorul funcțiilor **begin()** și **end()**, care returnează un pointer către primul, respectiv către prima poziție de după ultimul element al colecției.

Anumite containere permit și iterarea de la ultimul element către primul, folosind *reverse iterators*, **rbegin()** și **rend()**, care returnează un pointer către ultimul, respectiv către poziția de dinaintea primului element al colecției.

Exemplu

```
#include<iostream>
#include <vector>

int main()
{
    std::vector<int> vector = { 1, 2, 3, 4, 5, 6 };

    std::vector<int>::iterator it;
    for (it = vector.begin(); it!=vector.end(); it++)
        std::cout << *it << " ";

    return 0;
}
```

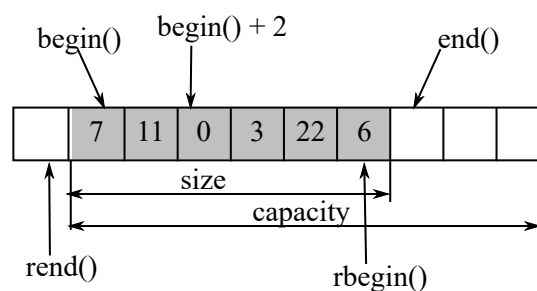


Figure 1.5: Iteratorii pentru un vector.

Observație! Deoarece iteratorul reține un pointer către un element din container este necesar să utilizăm **operatorul *** pentru a obține valoarea elementului.

Pentru a accesa elementul de pe o poziție specifică din cadrul containerului se pot utiliza funcțiile `std::next()` și `std::prev()` sau operatorii de adunare și scădere.

Iteratorii pe un obiect de tip vector sunt ilustrați grafic în figura 1.5 . În continuare este prezentat un exemplu de cod pentru utilizarea iteratorilor descriși.

Exemplu

```
#include<iostream>
#include <vector>

int main()
{
    std::vector<int> vector = { 1, 2, 3, 4, 5, 6 };

    // iterator catre primul element din vector
    std::vector<int>::iterator begin_it=vector.begin();

    //iterator catre urmatoarea pozitie de dupa
    //ultimul element al vectorului
    std::vector<int>::iterator end_it = vector.end();

    std::cout << *begin_it << " "; //se va afisa val.1
    //std::cout << *end_it << " "; //eroare

    auto next_it = std::next(begin_it, 2);
    std::cout << *next_it << " "; //se va afisa val.3

    //alternativa a functiei std::next()
    std::cout << *(begin_it + 2) << " ";

    auto prev_it = std::prev(end_it, 1);
    std::cout << *prev_it << " "; //se va afisa val.6

    //alternativa a functiei std::prev()
    std::cout << *(end_it - 1) << " ";
    return 0;
}
```

1.3 Observații și concluzii

În capitolul de față au fost discutate câteva aspecte generale legate de cursul de **Structuri de date**. În plus este introdusă biblioteca STL și sunt descrise câteva containere de bază din această bibliotecă, precum **tuple**, **array**, **vector**. Dat fiind faptul că, materia este predată în anul întâi, când studenții sunt familiarizați în special cu limbajul C și studiază limbajul C++, acest curs este orientat spre aceste limbaje de programare și se discută structurile din STL. Pe parcursul semestrului vor fi însă făcute referiri și la implementarea structurilor discutate și în alte limbaje de programare cunoscute.

Este recomandată utilizarea claselor STL la rezolvarea problemelor, acolo unde este posibil, deoarece acestea gestionează în mod corespunzător memoria și se evită probleme legate de alocarea și dealocarea potrivită a acesteia.

Documentație suplimentară poate fi găsită ușor online.

Chapter 2

Structuri de date elementare

Acest capitol prezintă câteva dintre structurile de date elementare cele mai utilizate în problemele de informatică, precum stive, cozi și liste înlănțuite. Vor fi descrise modul de funcționare, variantele de implementare și clasele corespunzătoare din biblioteca STL. De asemenea vor fi oferite exemple de probleme rezolvate cu ajutorul acestor structuri.

2.1 Liste înlănțuite

Vom începe prin abordarea listelor simplu și dublu înlănțuite. Stivele și cozile sunt de fapt niște *adaptoare de container*, deoarece au la bază un alt container (vector, listă, deque), dar cu un mod specific de acces la elemente.

O listă înlănțuită este o structură de date în care fiecare element este la rândul său o structură cu mai multe câmpuri, dintre care unul conține informația, iar celelalte reprezintă legături de tip pointer către elementele vecine din listă.

2.1.1 Liste simplu înlănțuite

În cazul listelor simplu înlănțuite, fiecare element este o structură dispunând de un câmp pentru informație și un câmp de legătură către următorul element din listă, ca în figura 2.1.

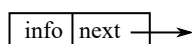


Figure 2.1: Structura unui nod: un câmp pentru informație și un câmp de legătură către următorul element.

Spre deosebire de un vector / array, elementele unei liste înlănțuite nu sunt neapărat memorate în zone de memorie adiacente și deci nu este necesară alocarea unui bloc de

memorie compact pentru elemente sale. În schimb, de fiecare dată când se adaugă un nou element la listă, este necesară alocarea de memorie pentru acest element, iar când se șterge un element din listă, se eliberează memoria corespunzătoare.

Accesul la elementele listei se realizează prin capul listei, reprezentând primul element. Astfel este necesară o variabilă pentru păstrarea adresei primului element al listei. Un exemplu de listă simplu înlănțuită este prezentat în figura 2.2.

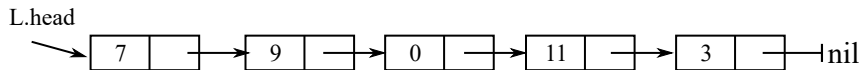


Figure 2.2: Listă simplu înlănțuită.

Operațiile principale pe o listă înlănțuită sunt: adăugarea / ștergerea unui element la începutul listei, ștergerea unui element cu o anumită valoare și parcurgerea listei. Pentru căutarea unui element, trebuie parcursă lista pornind de la primul element. Câțiva dintre algoritmi pentru implementarea operațiilor pe liste simplu înlănțuite sunt prezentați în continuare. Vom considera pentru aceasta o structură `F_LIST` (reprezentând `forward_list`), care are ca membru un pointer la primul element din listă, pe care îl vom denumi **head**. Fiecare element al listei este o structură de tip `NODE`, cu un câmp **key** pentru informație și un câmp **next**, care indică următorul element din listă. Pentru ultimul element câmpul **next** este `nil`.

Algoritm: `PUSH_FRONT`

Intrare: Lista simplu înlănțuită L în care adaug elementul *valoare*
 alocă memorie pentru *nod_nou*
 $nod_nou.key \leftarrow valoare$
 $nod_nou.next \leftarrow L.head$
 $L.head \leftarrow nod_nou$

Acest algoritm permite adăugarea unui element la începutul listei și este ilustrat în figura 2.3

Complexitate: Algoritmul de adăugare a unui element la începutul listei are complexitate constantă de timp și de memorie - $\Theta(1)$

Pentru a șterge o anumită valoare dintr-o listă simplu înlănțuită, lista trebuie parcursă pornind de la cap și fiecare *nod* care conține valoarea ce se dorește eliminată din listă se șterge, făcând legătura de la nodul care îl precede la nodul care îl succede. Algoritmul este prezentat în cele ce urmează.

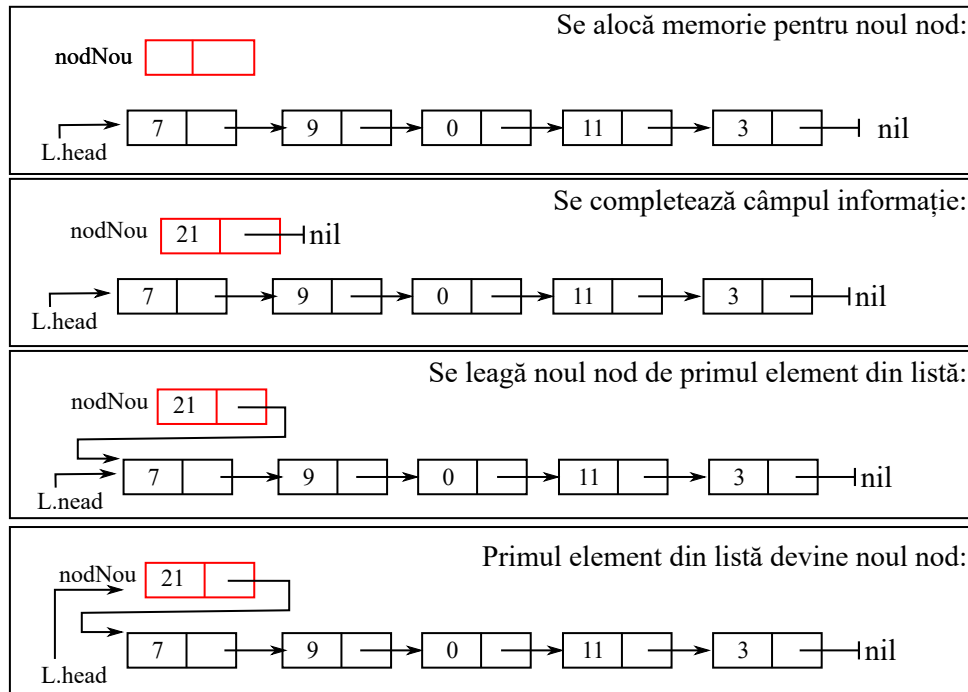


Figure 2.3: Adăugarea valorii 21 în lista simplu înlănțuită din imagine.

Algorithm: REMOVE

Intrare: Lista simplu înlănțuită L din care se șterg toate aparițiile cheii *valoare*

```

cat timp  $L.head \neq nil$  si  $L.head.key = valoare$  executa
  |  $ERASE\_FRONT(valoare)$ 
sfarsit_cat_timp
daca  $L.head = nil$  atunci
  | return
sfarsit_daca
 $current \leftarrow L.head$ 
cat_timp  $current.next \neq nil$  executa
  | daca  $current.next.key = value$  atunci
    |  $node = current.next$ 
    |  $current.next \leftarrow current.next.next$ 
    | elibereaza memoria pentru  $node$ 
  | sfarsit_daca
  | altfel
    |  $current \leftarrow current.next$ 
  | sfarsit_daca
sfarsit_cat_timp

```

În algoritmul de mai sus variabila *current* reprezintă nodul cu care se parcurge lista

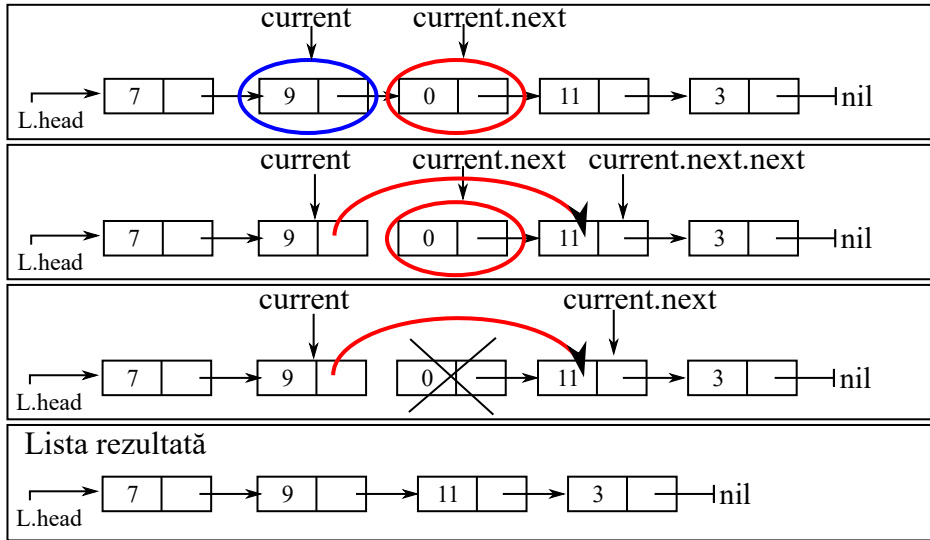


Figure 2.4: Ștergerea valorii 0 din lista simplu înlanțuită din imagine.

și, în final, nodul **dinaintea** nodului care va fi șters. Ștergerea din listă presupune de fapt legarea elementului *current*, de nodul care îi urmează lui *current.next*. În cazul în care valoarea pe care dorim să o ștergem se află chiar la începutul listei, se apelează funcția *ERASE_FRONT*, a cărei implementare rămâne ca exercițiu. Ștergerea unei valori dintr-o listă înlanțuită este ilustrată în figura 2.4.

Complexitate: Eliminarea unei anumite valori din listă presupune parcurgerea acesteia, pâna la capăt și atunci când se întâlnește valoarea, nodul respectiv se elimină. Complexitatea operației este liniară, $\Theta(n)$, unde n = lungimea listei.

Algoritm: FIND(valoare)

Intrare: Lista simplu înlanțuită L în care caut *valoare*

$curent \leftarrow L.head$

cat timp $curent \neq nil$ si $curent.key \neq valoare$ **executa**

$curent \leftarrow curent.next$

sfarsit_cat_timp

return $curent$

Algoritmul *FIND* returnează un pointer către zona de memorie în care este stocat elementul cu valoarea *valoare*. În cazul în care acest element nu există, algoritmul returnează *nil*.

Complexitate: Căutarea unui element cu o anumită valoare din listă presupune parcurgerea acesteia, pâna se întâlnește valoarea respectivă. În cel mai defavorabil caz, atunci când elementul nu se află în listă sau se află pe ultima poziție, trebuie parcursă întreaga listă. În medie complexitatea este liniară, $\Theta(n)$, unde n = lungimea listei.

Desigur, există o serie de alte operații, care pot fi efectuate pe o listă simplu înlanțuită,

majoritatea presupunând iterarea prin elementele acesteia. Spre exemplu, se pot insera sau șterge elemente pe o anumită poziție, se pot efectua sortări, etc. Implementarea câtorva dintre aceste operații este propusă în temele de laborator.

2.1.2 Liste dublu înlănțuite

Elementele unei liste dublu înlănțuite posedă, spre deosebire de elementele unei liste simplu înlănțuite, alături de o legătură spre următorul element și o legătură către elementul precedent. Accesul în listă se poate realiza prin capul listei, care indică primul element, dar și prin ultimul element. Vom considera fiecare element din lista dublu înlănțuită ca fiind o structură de tip **NODE**, care dispune de câmpurile *key* pentru informație, *prev* pentru legătura la elementul precedent și *next* pentru legătura la elementul următor din listă.

Algoritmii prezentați în [1] tratează liste, în care accesul se realizează doar la capul listei, prin variabila *head*. Adăugarea în listă se face în acest caz doar la începutul listei.

În acest curs însă vom considera o implementare similară cu cea din biblioteca STL, în care accesul poate avea loc atât de la primul element, cât și de la ultimul, adăugarea / ștergerea unui element la începutul listei se va face prin funcții de tip **PUSH_FRONT** / **POP_FRONT**, iar adăugarea / ștergerea unui element la sfârșitul listei se va realiza prin **PUSH_BACK** / **POP_BACK** în complexitate constantă. Structura LIST va dispune de două câmpuri: *head* și *tail*, care reprezintă pointeri către primul, respectiv ultimul element din listă.

Un exemplu de astfel de listă dublu înlănțuită este prezentat în figura 2.5.

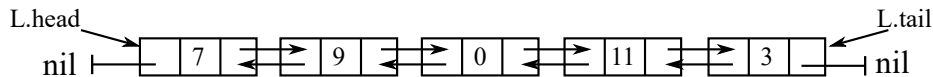


Figure 2.5: Listă dublu înlănțuită cu acces direct atât la primul, cât și la ultimul element.

Printre cele mai importante operații asupra unei liste dublu înlănțuite sunt: adăugarea / eliminarea unui element de la începutul / sfârșitul listei sau pe orice poziție în listă, ștergerea unei valori din listă, iterarea prin listă în ambele direcții. Căutarea unui element presupune parcurgerea listei de la primul element până la găsirea acestuia sau până la sfârșitul listei, în caz de eșec.

În continuare sunt prezentați algoritmii pentru adăugarea unui element la începutul listei, pentru ștergerea unui element de pe o anumită poziție din listă, indicată printr-un pointer, și pentru ștergerea unui element cu o anumită cheie din listă. De asemenea va fi prezentat un algoritm de căutare a unui element în listă.

Algorithm: PUSH_FRONT**Intrare:** Lista dublu înlanțuită L în care adaug elementul $valoare$

```

aloca memorie pentru  $nod\_nou$ 
 $nod\_nou.key \leftarrow valoare$ 
 $nod\_nou.next \leftarrow L.head$ 
 $nod\_nou.prev \leftarrow nil$ 
daca  $L.head \neq nil$  atunci
|    $L.head.prev \leftarrow nod\_nou$ 
sfarsit_daca
altfel
|   //Atat primul cat si ultimul element sunt reprezentate de
|   //  $nod\_nou$ , care e unicul element
|    $L.tail \leftarrow nod\_nou$ 
sfarsit_daca
 $L.head \leftarrow nod\_nou$ 

```

Acest algoritm permite adăugarea unui element la începutul listei și este ilustrat în figura 2.6.

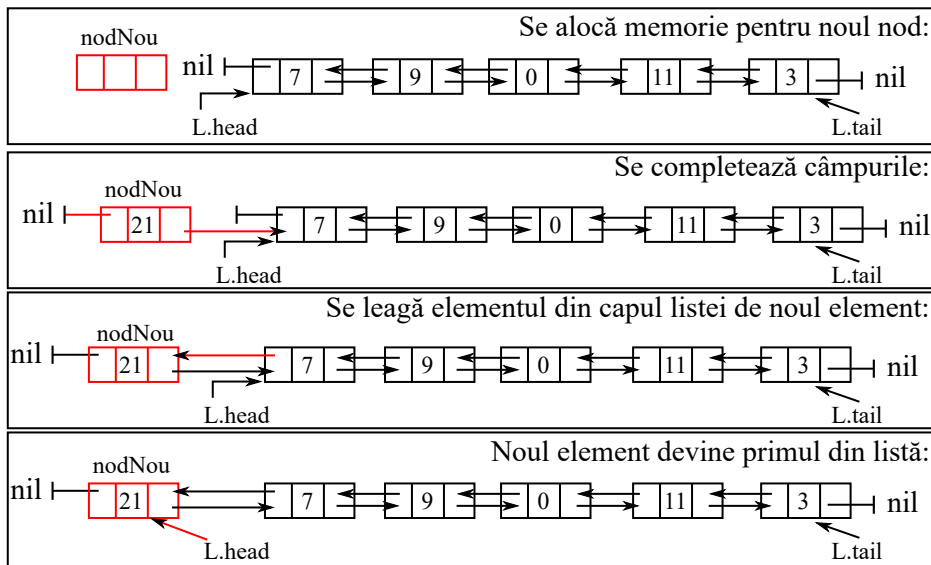


Figure 2.6: Adăugarea valorii 21 într-o listă dublu înlanțuită.

Complexitate: Algoritmul de adăugare a unui element la începutul listei are complexitate constantă de timp și de memorie - $\Theta(1)$

Funcția *ERASE* prezentată în continuare, realizează ștergerea unui element dintr-o listă înlanțuită reprezentat printr-un pointer la nodul respectiv. Acest lucru se poate

face, dacă în prealabil a fost identificat elementul pe baza unei căutări, respectiv iterări prin listă.

Algorithm: ERASE

Intrare: Lista dublu înlănțuită L din care se șterge elementul indicat de pointerul nod

```

daca  $nod.prev \neq nil$  atunci
  |  $nod.prev.next \leftarrow nod.next$ 
sfarsit_daca
altfel
  |  $L.head \leftarrow nod.next$ 
sfarsit_daca
daca  $nod.next \neq nil$  atunci
  |  $nod.next.prev \leftarrow nod.prev$ 
sfarsit_daca
altfel
  |  $L.tail \leftarrow nod.prev$ 
sfarsit_daca
dealoca memoria pentru  $nod$ 

```

Complexitate: Operația de ștergere efectivă are complexitate constantă. Desigur, identificarea elementului, care trebuie șters, presupune iterarea prin listă și deci complexitate liniară.

Funcția *REMOVE* are ca parametru o valoare ce se dorește eliminată din lista dublu înlănțuită. Acest lucru presupune parcurgerea listei și la întâlnirea valorii respective apelarea funcției *ERASE*, definită mai sus.

Algorithm: REMOVE

Intrare: Lista dublu înlănțuită L din care se elimină toate elementele cu cheia $valoare$

```

 $current \leftarrow L.head$ 
cat_timp  $current \neq nil$  executa
  |  $nod \leftarrow current.next$ 
  | daca  $current.key = valoare$  atunci
  | |  $ERASE(current)$ 
  | sfarsit_daca
  |  $current \leftarrow nod$ 
sfarsit_cat_timp

```

Complexitate:

- Ștergerea efectivă: $\Theta(1)$
- Parcurgerea listei: $\Theta(n)$, n - numărul de elemente din listă.

Putem defini și o funcție de căutare a unui element în listă. Algoritmul corespunzător

are în mod evident complexitate medie liniară.

Algorithm: FIND(valoare)

Intrare: Lista dublu înlănțuită L în care caut *valoare*

$curent \leftarrow L.head$

cat _ timp $curent \neq nil$ si $curent.key \neq valoare$ **executa**

 | $curent \leftarrow curent.next$

sfarsit _ cat _ timp

return $curent$

Celelalte operații pentru liste dublu înlănțuite sunt propuse pentru implementare la secțiunea de exerciții.

Observație: Modul de implementare al listelor simplu înlănțuite prezentat în acest capitol nu este singurul mod posibil. Denumirile funcțiilor și modul de abordare au fost gândite să se potrivească cu modul de implementare din biblioteca `stl`, în care, de exemplu, într-o listă simplu înlănțuită avem acces doar la primul element din listă, nu și la ultimul, așa cum avem în cazul celor dublu înlănțuite. Evident că din acest motiv ștergerea sau adăugarea la sfârșitul listei se realizează în complexitate liniară, nu constantă. Dacă dintr-un motiv sau altul se dorește implementarea unei liste simplu înlănțuite, la care aceste operații să se realizeze în complexitate constantă, se poate adăuga un membru suplimentar clasei, care să permită accesul la ultimul element, iar algoritmi se scriu în consecință. În [1] sunt prezentați algoritmi pentru operațiile pe liste înlănțuite similari, dar ușor diferiți de cei prezentați în acest curs.

2.1.3 Utilizarea listelor înlănțuite

Dacă ar fi să comparăm listele înlănțuite cu structuri de tip vector, se poate observa ușor că un dezavantaj semnificativ al listelor este acela, de a nu putea accesa elementele prin poziție. Accesul rapid prin poziție este specific vectorilor și este posibil, datorită memorării elementelor unui vector în locații de memorie succesive. În schimb, inserarea sau ștergerea unui element la începutul structurii, sau de pe o poziție arbitrară poate fi mai eficientă în cazul listelor înlănțuite. Deși și aici există controverse, depinzând de natura și de dimensiunea datelor.

Un alt dezavantaj al unei liste înlănțuite este acela că, elementele sale nu sunt stocate în zone de memorie succesive, ci pot fi oriunde în memoria atribuită programului. Astfel nu se cunoaște locația unui element, decât atunci când se accesează, prin traversarea listei. Acest lucru poate constitui un dezavantaj semnificativ de exemplu în paralelizarea execuției de instrucțiuni care accesează date diferite (*out of order execution*) sau transferul de pe CPU pe GPU. În calculatoarele moderne, o serie de operații se execută direct pe GPU, pentru a crește viteza de execuție. În acest caz folosirea listelor înlănțuite poate constitui un dezavantaj semnificativ, mai ales dacă este vorba de un număr redus de elemente.

Dacă aplicația presupune o mulțime de elemente, care se modifică relativ rar sau preponderent prin adăugarea de elemente la sfârșit, dar în care accesul prin poziție este

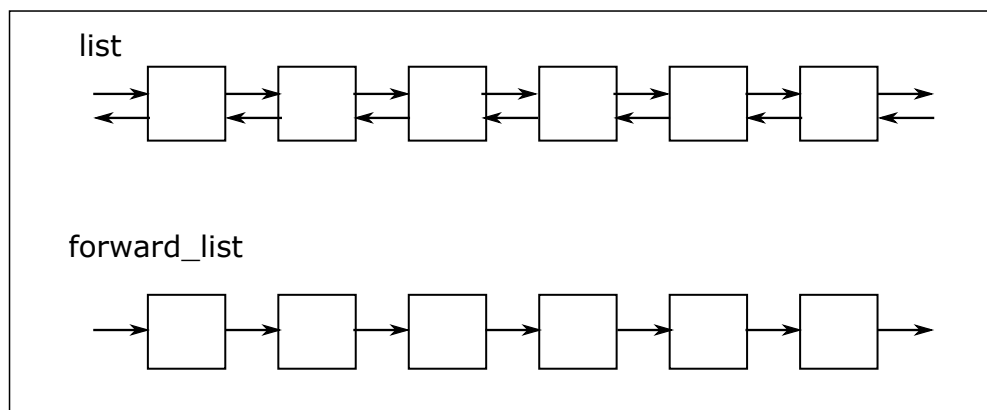
frecvent, evident o structură de tip vector este preferabilă. În schimb, acolo unde modificările prin adăugare / ștergere sunt frecvente, iar accesul prin poziție este de mică importanță, și numărul de elemente memorat este mare se preferă liste înlănțuite.

De asemenea, acolo unde sunt frecvente adăugări și extrageri la începutul structurii, vectorii au o performanță slabă. Este de preferat o listă înlănțuită sau un *deque* (urmează). O comparație interesantă a performanței celor două structuri, vector și listă, este prezentată în [10]

Avantajul unei liste simplu înlănțuite față de o listă dublu înlănțuită este acela că necesită semnificativ mai puțină memorie pentru stocarea legăturilor către elemente vecine. În schimb, spre deosebire de listele dublu înlănțuite, o listă simplu înlănțuită permite iterarea doar într-o singură direcție, de la capul listei spre sfârșit.

Listele înlănțuite pot fi utile de exemplu în implementarea tabelelor de repartizare - *hash tables*, care vor fi discutate în capitolul următor. De asemenea sunt utile, atunci când elementele cu care se lucrează sunt de dimensiuni relativ de mari, de exemplu pagini de text. O aplicație concretă este acela al navigării înainte și înapoi între imagini (Photoshop) sau al navigării înainte și înapoi între cântecele dintr-un *playlist*.

2.1.4 Liste simplu și dublu înlănțuite în STL



List:

- elementele nu sunt memorate în zone de memorie alăturate
- accesul elementelor în timp liniar, cu iteratori.
- inserție și ștergere de elemente în timp constant.

```

#include<iostream>
#include<vector>
#include<list>

int main()
{
    std::list<int> lista;

    std::list<int> lista2(4, 50); //contine 4 elem egale cu 50

    std::list<int> lista3(lista2); //copie a listei 2

    std::list<int> lista4 = lista3; //copie a listei 3

    std::list<int> lista5 = { 1, 2, 3, 4, 5 }; // initializare

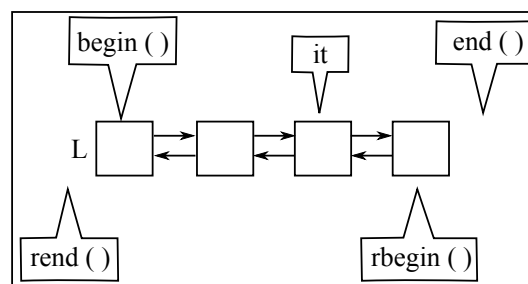
    std::vector<int> v = { 2, 4, 6, 8, 10 };
    std::list<int> lista6(v.begin(), v.end()); //initializare pe baza unui vector;

    return 0;
}

```

Observații

- Accesul la elemente NU se poate face prin indicele poziției, ca la vector!
- Pentru a parcurge o lista se folosesc ITERATORI



Exemplu de parcurgere

<pre> #include<iostream> #include<list> int main() { std::list<int> lista; int num = 5, elem; for (int index = 0; index < num; index++) { std::cin >> elem; lista.push_back(elem); lista.push_front(elem); } std::list<int>::iterator it = lista.begin(); for (; it != lista.end(); it++) { std::cout << *it << " "; } </pre>	<pre> std::cout << std::endl; //sau for (int element : lista) { std::cout << element << " "; } std::cout << std::endl; return 0; } </pre>
---	---

2.1.5 Liste înlanțuite în alte limbaje de programare

- În Java **List** este de fapt o interfață, implementată de
 - **ArrayList<T>** - vector dinamic de elemente de tip T.
 - **LinkedList<T>** - listă înlanțuită de elemente de tip T.
 - **Stack<T>** - stivă de elemente de tip T.
- În Python listele sunt colecții de date, accesibile prin poziție și pot fi de tipuri diferite. De fapt în spate sunt implementate printr-un *array* de pointeri. Liste înlanțuite nu sunt definite în biblioteca python standard.
- În C#:
 - **List<T>** reprezintă de fapt un *array* dinamic (poate fi redimensionat) de elemente de tip T.
 - **LinkedList<T>** reprezintă o listă dublu înlanțuită de elemente de tip T.

2.2 Stive și cozi

2.2.1 Stiva

Stiva - *stack* - este o structură cu o disciplină de intrare/ieșire de tip **LIFO** - *Last In First Out*, adică ultimul element introdus va fi primul care se extrage pentru prelucrare. Putem compara stiva cu o mulțime de obiecte puse unul deasupra celuilalt. Primul obiect introdus se află la bază, iar ultimul se află în vârf, deci primul obiect care poate fi luat din stivă este cel din vârf, iar adăugarea unui element nou se va realiza tot prin vârf. Accesul la un element aflat mai jos de vârful stivei nu se poate realiza, decât după ce au

fost extrase toate elementele aflate deasupra acestuia.

Stiva este un **adaptor de container**, adică în spate există o altă structură (container), în care se stochează efectiv elementele, iar modul de acces la elemente este cel descris mai sus. Pentru stocarea elementelor din stivă se poate utiliza de exemplu un tablou liniar / vector (reprezentare secvențială), o listă înlănțuită sau o structură de tip *deque* (va fi descrisă la secțiunea de STL).

Un obiect de tip stivă trebuie să dispună de funcții pentru adăugarea unui element în vârful stivei - *push*, pentru extragerea elementului din vârful stivei - *pop*, pentru obținerea elementului din vârful stivei - *top* și pentru verificarea dacă stiva este vidă. Vom considera cele două moduri de implementare ale stivei, secvențială și înlănțuită și vom prezenta pseudocodul pentru funcțiile *push* și *pop*.

Reprezentarea secvențială

Se va considera o structură *STACK*, care dispune de un câmp de tip vector / array pentru stocarea elementelor, precum și de un câmp *Top* care reprezintă poziția vârfului în vector. Dacă pentru stocarea elementelor se utilizează containerul *vector* din STL, atunci nu trebuie luată în considerare umplerea stivei, deoarece acest container realocă memorie atunci când este necesar. Dacă în schimb structura folosește un *array*, atunci memoria disponibilă este limitată și trebuie ținut cont de posibilitatea umplerii stivei. În acest caz vom considera un atribut suplimentar *Capacity*, reprezentând numărul maxim de elemente care poate fi stocat în stivă. Indicii într-un vector încep de la 0. Astfel:

- Dacă $Top = -1$ atunci stiva este vidă și nu pot extrage elemente.
- Dacă $Top = Capacity - 1$ stiva este plină și nu pot adăuga elemente noi.

Un exemplu de stivă care are la bază reprezentarea secvențială este ilustrat în figura 2.7. Stiva conține 7 elemente, iar variabila $Top = 6$.

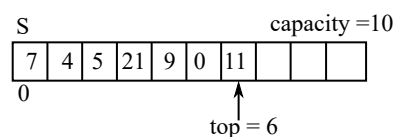


Figure 2.7: Stivă care memorează elementele într-un vector.

Adăugarea unui element în stivă se realizează prin intermediul unei funcții numită în general *push*. Aceasta crește indicele *Top* cu o unitate și plasează în vector pe această poziție noul element.

Algoritm: PUSH

Intrare: Stiva S în care adaug elementul val
daca $S.Top = capacity - 1$ **atunci**
 Eroare de depasire a capacitatii si stop
 sau realocare de memorie si copierea elementelor din stiva
sfarsit_daca
 $S.Top \leftarrow S.Top + 1$
 $S[S.Top] \leftarrow val$

Extragerea din stivă se realizează la vârf, prin intermediul unei funcții *pop*, care scade indicele *Top*. Dacă stiva este vidă, nu pot fi extrase elemente.

Algoritm: POP

Intrare: Stiva S din care se extrage elementul aflat în vârf
daca $S.Top = -1$ **atunci**
 Eroare de stiva vida
 return
sfarsit_daca
 $S.Top \leftarrow S.Top - 1$

Complexitate: ambele operații sunt practic instantanee - complexitatea este $\Theta(1)$. Doar dacă în cazul umplerii containerului pentru stivă, se realocă o zonă de memorie mai mare și se copiază elementele din zona veche în zona nouă, această operație este liniară relativ la numărul de elemente din stivă. Operația de realocare trebuie să aibă loc rar, astfel încât complexitatea să rămână în medie constantă.

Reprezentare înlănțuită

În cazul reprezentării înlănțuite se poate utiliza drept container o listă simplu înlănțuită, în care fiecare element este o structură de tip NOD cu 2 câmpuri: *key* - pentru informație și *next* - pointer către elementul următor. O astfel de stivă este reprezentată în figura 2.8. Desigur, ar putea fi folosită și o listă dublu înlănțuită, dar deoarece pe stivă nu au loc parcurgeri și este suficient accesul la primul element, nu se justifică consumul de memorie presupus de o listă dublu înlănțuită.

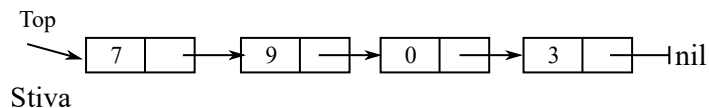


Figure 2.8: Stivă care memorează elementele într-o listă simplu înlănțuită.

Putem considera pentru stivă o structură / clasă care să aibă ca membru un element *Top* de tip pointer la NOD ce reprezintă vârful stivei.

Funcțiile de adăugare și eliminare a elementului din vârful stivei S sunt:

Algorithm: PUSH

Intrare: Stiva S în care adaug elementul val
 alocă memorie pentru y
 $y.info \leftarrow val$
 $y.next \leftarrow S.Top$
 $S.Top \leftarrow y$

Acest algoritm este ilustrat în figura 2.9

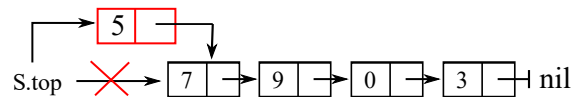


Figure 2.9: Adăugarea elementului 5 la stiva S , reprezentată printr-o listă înlanțuită.

Algorithm: POP

Intrare: Stiva S din care se elimină elementul din vârf
dacă $S.empty()$ **atunci**
 | returnează eroare de stivă vidă
sfarsit_dacă
 $y \leftarrow S.Top$
 $S.Top \leftarrow S.Top.next$
 delete y

Complexitate: ambele operații sunt practic instantanee - complexitatea este $\Theta(1)$.

2.2.2 Coadă -queue

Definiție: Coadă este o structură cu o disciplină de intrare/ieșire de tip **FIFO** - *First In First Out*, adică primul element introdus va fi primul care se extrage pentru prelucrare. Coadă modelează procese care presupun formarea de cozi, de exemplu deservirea clienților la un ghișeu. De asemenea se utilizează cozi pentru parcurgerea în lățime de grafuri, respectiv arbori.

Comparare cu stivă. Spre deosebire de stivă, unde se utilizează o variabilă pentru accesarea vârfului stivei și atât adăugarea cât și extragerea elementelor se realizează prin intermediul acesteia, în cazul unei cozi este necesară memorarea a două elemente: primul - aici se face extragerea și ultimul - aici se face adăugarea.

Ca și stiva, coada este un adaptor de container, deci are la bază un container în care se păstrează elementele, de exemplu un *array* (vector), o listă înlanțuită sau o structură de tip *deque*.

Un obiect de tip coadă trebuie să dispună de funcții pentru adăugarea unui element la sfârșitul cozii - *push*, pentru extragerea primului element al cozii - *pop*, pentru obținerea primului și ultimului element - *front*, respectiv *back*, precum și pentru verificarea dacă este vidă - *empty*.

Reprezentarea secvențială

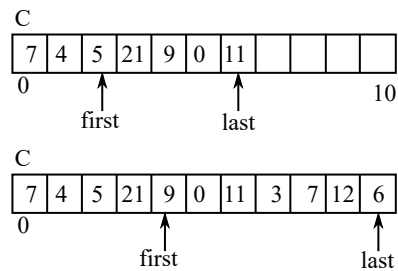


Figure 2.10: Din coada C s-au extras elementele 5 și 21 și s-au adăugat elementele 3, 7, 12, 6. În acest moment primele 4 poziții sunt considerate neocupate, dar funcția PUSH returnează mesajul de coadă plină.

Se va considera o structură *QUEUE* care dispune de un câmp de tip vector / array pentru stocarea elementelor și de două câmpuri de tip int FIRST și LAST care indică primul și ultimul element din coadă. Dacă dimensiunea containerului este fixă, atunci coada se poate umple și trebuie considerat și un atribut *capacity*, care reprezintă numărul maxim de elemente, ce poate fi stocat în coadă.

Observație: adăugarea unui element în coadă se realizează la sfârșitul cozii, ceea ce duce la incrementarea indicelui LAST, iar extragerea elementului de la începutul cozii se realizează prin creșterea indicelui FIRST. Astfel, elementul nu se șterge efectiv din memorie, doar că nu se mai consideră ca aparținând cozii. În figura 2.10 este ilustrat modul în care funcționează o astfel de coadă. Problema care apare este migrarea cozii spre dreapta, odată cu efectuarea de operații succesive de adăugare și ștergere de elemente.

Coadă este considerată plină, dacă indicele LAST ajunge să fie egal cu ultima poziție din vector, și este considerată vidă, dacă indicele FIRST devine mai mare decât LAST (prin avansare odată cu apelul funcției PUSH). Astfel se poate ajunge la situația în care în vectorul de date corespunzător cozii sunt multe poziții considerate neocupate, dar totuși la apelarea funcției PUSH se primește mesaj de coadă plină - fig. 2.10, deoarece pozițiile considerate libere, aflate înaintea poziției *prim* nu mai sunt accesibile. Această problemă se rezolvă în practică prin utilizarea unei cozi circulare.

Coadă circulară

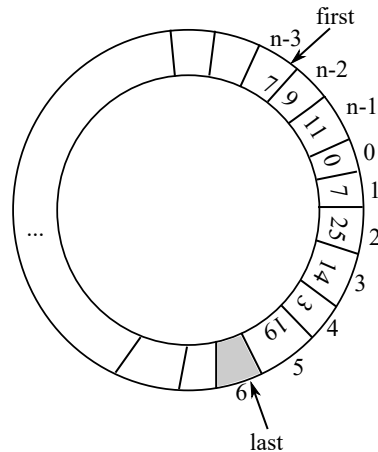


Figure 2.11: Coadă circulară cu n poziții în care la momentul curent variabila *first* indică poziția $n - 3$ și variabila *last* indică poziția neocupată 6.

În cazul unei cozi circulare, atunci când variabila $last = capacity - 1$ la următoarea inserție *last* va devine 0, deci se reia circular parcurgerea cozii de la început. Similar se procedează cu variabila *first* la extragere. Astfel se va primi mesaj de eroare de coadă plină doar atunci când sunt ocupate toate pozițiile alocate în vector pentru elementele cozii.

Condițiile de coadă vidă/plină

În cazul cozilor circulare condițiile de coada plină și coadă vidă descrise mai sus nu mai sunt valabile:

- Inegalitatea $first > last$ nu înseamnă decât faptul că s-a reluat parcurgerea cozii de la început - vezi fig. 2.11.
- Evident condiția $last = capacity - 1$ nu mai generează mesajul de coadă plină

Soluționarea problemei detecției cozii vide sau pline: Această problemă se rezolvă în modul următor. Poziție din vector indicată de elementul *LAST* nu va fi ocupată. Deci în vector va exista mereu o poziție neocupată. Ea este utilizată doar pentru marcarea sfârșitului cozii. Condițiile de coadă vidă / coadă plină sunt în acest caz:

- Dacă $first = last$ atunci coada este vidă
- Dacă $(last + 1) \bmod capacity = first$ atunci coada este plină

În figura 2.11 este reprezentat un exemplu de coadă circulară.

Funcțiile de adăugare și respectiv eliminare a unui element din coadă pot fi descrise prin:

Algoritm: PUSH

Intrare: Coada C în care adaug elementul *valoare*

```

daca  $(C.last + 1) \bmod C.capacity = C.first$  atunci
    | Eroare de depasire a capacitatii si stop
    | sau realocare de memorie si copierea elementelor cozii
sfarsit_daca
 $C[C.last] \leftarrow valoare$ 
 $C.last \leftarrow C.last + 1 \bmod capacity$ 

```

Algoritm: POP

Intrare: Coada C în care se extrage primul element

```

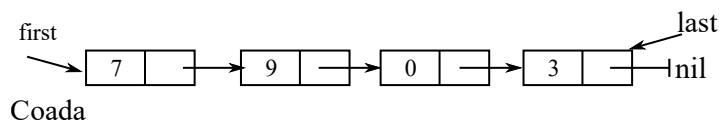
daca  $C.first = C.last$  atunci
    | Eroare de coada vida
    | return
sfarsit_daca
 $C.first \leftarrow C.first + 1$ 
daca  $C.first = C.capacity$  atunci
    |  $C.first \leftarrow 0$ 
sfarsit_daca

```

Complexitatea: operațiilor de adăugare și extragere din coadă este constantă, cu excepția realocării la *push*, dacă are loc.

Reprezentare înlănțuită

Pentru fiecare element poate fi utilizată o structură care conține 2 câmpuri: INFO - pentru informație și NEXT - pointer către elementul următor. Coada poate fi reprezentată grafic astfel:



Alocarea memoriei pentru coadă se face dinamic, nu static, așa ca la vector, adică la introducerea un element nou, trebuie mai întâi alocată memorie, iar la extragerea unui element, trebuie eliberată zona de memorie care era alocată acelui element.

Sunt necesare două variabile *first* și *last*, în care se rețin adresele primului respectiv ultimului element din coadă. Elemente noi se adaugă mereu după ultimul element iar la eliminare se consideră primul element din coadă. În continuare sunt prezentați cei doi

algoritmi.

Algoritm: PUSH

Intrare: Coadă C în care se adaugă elementul $valoare$

aloca memorie pentru nodul $nodNou$

$nodNou.info \leftarrow valoare$

$nodNou.next \leftarrow nil$

daca $C.first = nil$ **atunci**

$C.first \leftarrow nodNou$

$C.last \leftarrow nodNou$

 return

sfarsit_daca

$C.last.next \leftarrow nodNou$

$C.last \leftarrow nodNou$

Algoritm: POP

Intrare: Coadă C din care se extrage primul element

daca $C.first = nil$ **atunci**

 eroare de coada vida

 return

sfarsit_daca

$nod \leftarrow C.first$

$C.first \leftarrow C.first.next$

dealoca memoria alocata lui nod

Complexitatea: operațiilor de PUSH și POP este constantă.

2.2.3 deque

O structură interesantă este *deque* sau *double ended queue* și reprezintă o generalizare a ideii de coadă, în sensul că permite atât inserție cât și ștergere la ambele capete. În mod evident o astfel de structură poate fi implementată printr-o listă dublu înlănțuită. Acest lucru însă prezintă dezavantajele discutate la capitolul de liste și se recomandă a fi evitată în contextul calculatoarelor moderne. Implementări mai bune pot fi realizate folosind un *array*, parcurs circular, așa cum a fost descris la secțiunea de cozi sau o implementare care are la bază o secvență de vectori, așa cum este realizat în cazul celei din STL. În cel de-al doilea caz, structura este alcătuită din blocuri de dimensiune fixă N (de exemplu 8 sau 16). Blocul de început se completează de la ultimul element către primul (la adăugarea unui nou element), iar ultimul bloc de la primul către ultimul. Atunci când primul sau ultimul bloc se umple, se alocă un nou bloc de memorie. Adresele acestor blocuri sunt memorate la rândul lor într-un vector. Structura este reprezentată grafic în figura 2.12.

Avantajele unei structuri de tip deque implementate astfel sunt:

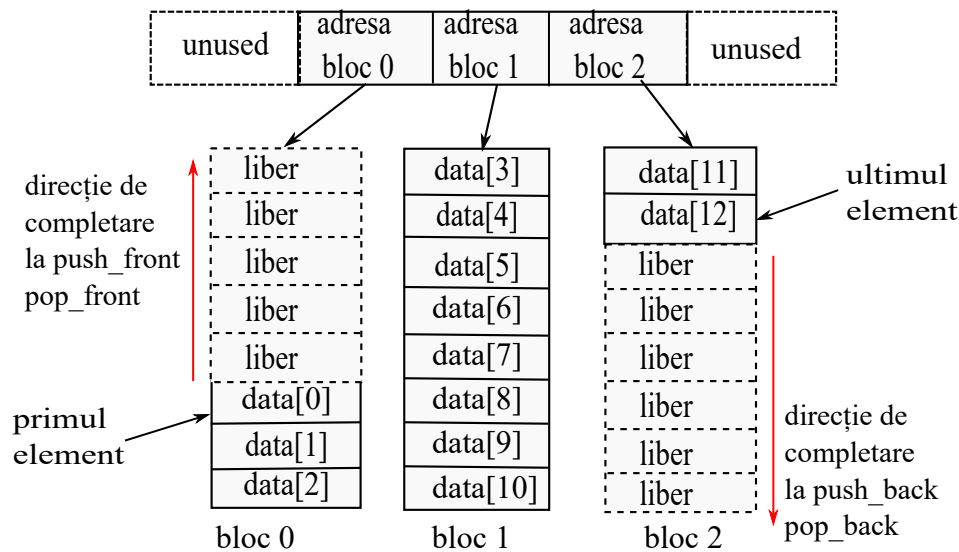


Figure 2.12: Reprezentare grafică a unei implementări pentru *deque* cu un vector de blocuri de dimensiune $N = 8$

- accesul rapid prin poziție la elemente (nu ar fi posibil cu o listă înlănțuită)
- adăugare / extragere de elemente la ambele capete în complexitate constantă
- la umplerea memoriei alocate nu este necesară realocarea întregului bloc de memorie (ca la *array*), ci doar a unui bloc nou.
- inserarea unui element nou la mijloc se face prin deplasarea a cel mult $n/2$ elemente, unde n este numărul de elemente stocate în structură, spre deosebire de *array*, unde se poate ajunge la deplasarea tuturor elementelor, dacă se dorește inserția la începutul structurii

Astfel un deque poate combina avantaje atât de la *array* cât și de la liste. Este o structură ideală pentru implementarea unei cozi.

În [11] este realizată o analiză a performanței acestui container comparativ cu containerul vector.

2.2.4 Probleme

1. Implementați o coadă folosind două stive și operații pe stivă.
2. Implementați o stivă folosind două cozi și operații pe cozi.
3. Căutați un element x într-o coadă folosind o altă coadă suplimentară.

4. Considerăm o stivă S_1 în care se introduc pe rând caracterele A, D, C, B și două stive suplimentare S_2 și S_3 inițial vide. Să se scrie operațiile necesare pentru a trece caracterele din S_1 în S_2 , astfel încât la extragerea pe rând din S_2 și afișarea pe ecran, literele să apară în ordine alfabetică.
5. Scrieți un algoritm pentru implementarea a două stive folosind un singur vector. Stivele se consideră pline, doar atunci când toate pozițiile din vector sunt ocupate.

2.2.5 Aplicații ce utilizează stive și cozi

Stivele se utilizează în diferite probleme, care presupun obținerea unor date în ordine inversă citirii lor. De asemenea recursivitatea are în spate gestionarea memoriei sub formă de stivă. Algoritmii de tip *backtracking* construiesc soluția în mod analog unei stive. Parcurgerile grafurilor și arborilor în adâncime se realizează cu ajutorul stivelor.

Cozile sunt utile în modelarea problemelor ce presupun tratarea datelor de intrare în ordinea citirii lor. De asemenea se utilizează în probleme de parcurgere în lățime a grafurilor și arborilor.

În continuare sunt prezentate câteva probleme rezolvate cu ajutorul stivelor. Aplicații pentru cozi vor fi prezentate la capitolul despre arbori, în cadrul parcurgerii arborilor oarecare și a arborilor binari.

1. Problema parantezării corecte: Se consideră un șir de paranteze rotunde închise și deschise. Să se verifice, dacă parantezarea este corectă. **Exemplu:** $((()))$ este corectă, $()()$ - nu este corectă. Această problemă poate fi extinsă pentru toate tipurile de paranteze: $[]$, $\{\}$, $()$, cu condițiile suplimentare că fiecare tip de paranteză închisă trebuie să închidă același tip de paranteză deschisă și nicio pereche de paranteze de ordin mai mare nu se poate afla în interiorul uneia de ordin mai mic.

Soluție: În continuare vom prezenta soluția la problema simplă, doar cu paranteze rotunde. Problema extinsă se rezolvă în mod similar.

Idee: Se folosește o stivă S – *Paranteze*. Se citește șirul *paranteze*. Se parcurge acest șir caracter cu caracter și pentru fiecare caracter, dacă este '(' se pune pe stivă. Altfel, dacă este ')', avem două posibilități:

- stiva este vidă - caz în care această paranteză închisă nu corespunde niciunei paranteze deschise $\Rightarrow$ eroare de parantezare
- stiva nu este vidă - se șterge vârful stivei, deoarece s-a închis această paranteză.

Dacă la sfârșitul parcurgerii șirului stiva este vidă, atunci parantezarea este corectă. Altfel există paranteze deschise, care nu au fost închise și deci parantezarea nu este corectă.

Funcția care verifică parantezarea corectă, scrisă în C++ folosind structuri din STL este prezentat mai jos.

```
bool parantezare(std::string paranteze)
{
    std::stack<char>S_paranteze;
    for (char c : paranteze)
    {
        if (c == '(')
            S_paranteze.push(c);
        else{
            if (c==')' && S_paranteze.empty())
                return false;
            S_paranteze.pop();
        }
    }
    if (S_paranteze.empty())
        return true;
    return false;
}
```

2. Evaluarea unei expresii aritmetice folosind forma poloneză postfixată.

Forma poloneză, denumită în engleză ***Reverse Polish Notation - RPN*** sau ***Polish postfix notation*** se numește astfel, deoarece a fost dezvoltată de un logician Polonez. Această formă presupune plasarea operatorilor după operanzi.

Exemple:

- $2 + 3$ se scrie în formă poloneză ca $2\ 3\ +$
- $2 + 3 * 5$ se scrie în formă poloneză ca $2\ 3\ 5\ *+$
- $(2 + 3) * 5$ se scrie în formă poloneză ca $2\ 3\ +\ 5*$

Observații: În forma poloneză postfixată ordinea operanzilor este aceeași ca în expresia obișnuită, DAR operatorii apar în ordinea efectuării lor. În plus, nu mai sunt necesare paranteze. Astfel evaluarea expresiei se va realiza foarte simplu în modul următor: se parcurge forma poloneză și la întâlnirea unui operator, acesta se aplică celor 2 operanzi care îl preced.

Exemplu: Se consideră expresia aritmetică: $2 + 3 * (5 - 1 - 1) - 3 * (5 - 2 * (2 - 1))$
Forma poloneză postfixată este: $2\ 3\ 5\ 1 - 1 - * + 3\ 5\ 2\ 2\ 1 - * - * -$

Evaluarea acestei forme poloneze se realizează astfel:

Primul operator întâlnit este $-$, care se aplică pentru 5 și 1:

2 3 **5 1 $-$** $1 - * + 3 5 2 2 1 - * - * -$.

Se obține: 2 3 4 1 $- * + 3 5 2 2 1 - * - * -$.

Acum, primul operator întâlnit este $-$ și se aplică pentru 4 și 1:

2 3 **4 1 $-$** $* + 3 5 2 2 1 - * - * -$

Se obține: 2 3 3 $* + 3 5 2 2 1 - * - * -$.

Acum, primul operator întâlnit este $*$ și se aplică pentru 3 și 3:

2 **3 3 $*$** $+ 3 5 2 2 1 - * - * -$

Se obține: 2 9 $+ 3 5 2 2 1 - * - * -$.

Acum, primul operator întâlnit este $+$ și se aplică pentru 2 și 9:

2 9 $+$ $3 5 2 2 1 - * - * -$

Se obține: 11 3 5 2 2 1 $- * - * -$.

Acum, primul operator întâlnit este $-$ și se aplică pentru 2 și 1:

11 3 5 2 **2 1 $-$** $* - * -$

Se obține: 11 3 5 2 1 $* - * -$.

Acum, primul operator întâlnit este $*$ și se aplică pentru 2 și 1:

11 3 5 **2 1 $*$** $- * -$

Se obține: 11 3 5 2 $- * -$.

Acum, primul operator întâlnit este $-$ și se aplică pentru 5 și 2:

11 3 **5 2 $-$** $* -$

Se obține: 11 3 3 $* -$.

Acum, primul operator întâlnit este $*$ și se aplică pentru 3 și 3:

11 **3 3 $*$** $-$

Se obține: 11 9 $-$.

Singurul operator rămas este $-$ și se aplică pentru 11 și 9:

11 9 $-$

Se obține: 2, care este soluția expresiei aritmetice.

Construirea formei poloneze postfixate. Pentru construirea formei poloneze se utilizează o stivă de caractere, în care vor fi plasați operatorii din expresie împreună cu parantezele deschise. Numim această stivă *op_stack*. De asemenea se utilizează un vector *polish* pentru stocarea formei poloneze. Acesta este inițial vid.

Construcția formei poloneze postfixate se realizează în modul următor: se parcurge iterativ șirul de caractere care reprezintă expresia aritmetică. Pentru caracterul curent *current* avem următoarele posibilități:

- *current* = cifră (presupunem deocamdată că operanzii au o singură cifră) $\Rightarrow$ se pune automat în vectorul *polish*
- *current* = '(' $\Rightarrow$ se pune automat în stivă.
- *current* = ')' $\Rightarrow$ se scot de pe stivă toți operatorii până la prima paranteză deschisă și se pun în vectorul *polish*. Apoi se scoate și paranteza deschisă. Dacă s-a golit

stiva și NU s-a întâlnit paranteză deschisă $\Rightarrow$ eroare de parantezare și nu se poate construi corect forma poloneză.

- *current* = operator $\Rightarrow$ trebuie pus pe stivă. DAR înainte se analizează stiva. Cât timp operatorul din vârful stivei e de prioritate cel puțin egală cu cea a lui *current*, se extrage din stivă și se plasează în *polish*. Abia apoi se pune *current* pe stivă.

După terminarea parcurgerii șirului de intrare, se extrag toți operatorii rămași în stivă și se plasează în forma poloneză. Dacă în stivă au mai rămas paranteze deschise, atunci este eroare de parantezare.

Algoritmul pentru construirea formei poloneze postfixate este prezentat la sfârșitul capitolului.

Observații:

- prioritatea operațiilor indică ordinea efectuării lor, într-o expresie fără paranteze. De exemplu prioritatea operatorului '+' e egală cu cea a lui '-' și mai mică decât a operatorului '*'. În plus, se recomandă ca paranteza deschisă să primească cea mai mică prioritate, pentru a nu fi scoasă din stivă de alt operator, decât de paranteza închisă.
- în algoritmul prezentat în continuare s-a presupus că expresia aritmetică este corectă. Nu s-au tratat erorile de parantezare, numărul și amplasarea corectă a operatorilor sau prezența de caractere nepermise. De asemenea nu s-au tratat spațiile albe. Aceste aspecte rămân ca exercițiu.
- pentru evaluare se poate utiliza o stivă de numere. Se parcurge forma poloneză caracter cu caracter. Dacă caracterul curent este o cifră, se introduce în stivă, dacă este un operator, se extrag din stivă două numere din vârf, se aplică operatorul, iar rezultatul se pune din nou în stivă. Numărul rămas la sfârșit în stivă este rezultatul expresiei aritmetice.

Implementarea algoritmului rămâne ca exercițiu de laborator, împreună cu tratarea erorilor și a situației, când se iau în considerare numere cu mai mult de o cifră.

Algoritm: PolishForm(expression)

Intrare: expresia aritmetica *expression**polish* $\leftarrow \emptyset$ *op_stack* $\leftarrow \emptyset$ **pentru** fiecare *current* din *expression* **executa** **daca** *current* este cifra **atunci** | *polish.push_back(current)* **sfarsit_daca** **altfel** **daca** *current* = '(' **atunci** | *op_stack.push(current)* **sfarsit_daca** **altfel** **daca** *current* = ')' **atunci** **cat_timp** *op_stack*! = $\emptyset$ **si** *op_stack.top()*! = '(' **executa** | *polish.push_back(op_stack.top())* | *op_stack.pop()* **sfarsit_cat_timp** **daca** *op_stack*! = $\emptyset$ **executa** *op_stack.pop()* //scoate paranteza **sfarsit_daca** **altfel** //daca *current* e operator **cat_timp** *op_stack*! = $\emptyset$ **si** *priorit(op_stack.top())* $\geq$ *priorit(current)* **executa** | *polish.push_back(op_stack.top())* | *op_stack.pop()* **sfarsit_cat_timp** *op_stack.push(current)* **sfarsit_daca** **sfarsit_daca****sfarsit_for****cat_timp** *op_stack*! = $\emptyset$ **executa** | *polish.push_back(op_stack.top())* | *op_stack.pop()***sfarsit_cat_timp**

Chapter 3

Tabele de dispersie

Așa cum s-a menționat în primul capitol al acestui curs, structurile de date au fost dezvoltate cu scopul memorării și manipulării *eficiente* a unei mulțimi dinamice de date. Această manipulare eficientă depinde desigur semnificativ de scopul urmărit și de tipurile de operații care se doresc a fi efectuate.

Una dintre principalele operații pe un set de date este căutarea sau *accesul prin cheie*. În cazul mulțimilor sortate, căutarea binară permite găsirea unei anumite chei în complexitate logaritmică. Vom vedea în capitolele ce tratează arborii binari de căutare, cum poate fi rezolvată în complexitate logaritmică problema căutării în cazul mulțimilor dinamice de date, deci a mulțimilor care se modifică prin inserții și ștergeri.

În capitolul de față sunt prezentate tabele de dispersie - *hash tables*, *hash maps* - structuri de date care permit accesul prin cheie în timp constant. O astfel de structură de date este eficientă, atunci când nu sunt necesare operații de sortare și păstrare ordonată a setului de date.

O tabelă de dispersie este de fapt o generalizare a noțiunii de vector - *array*. Operațiile de bază sunt inserția, căutarea și eventual ștergerea. Tabelele de dispersie pot fi utilizate cu succes pentru implementarea de dicționare, în care căutarea este cea mai frecventă operație. O altă utilizare a tabelelor de dispersie este în cadrul compilatoarelor pentru implementarea tabelii de identificatori sau pentru cuvintele cheie. Vom vedea pe parcurs că, într-o tabelă de dispersie operațiile au în medie complexitatea $O(1)$. O tabelă de dispersie se memorează într-un *array*.

3.1 Tabele cu adresare directă

Considerăm $U = \{0, 1, \dots, m-1\}$ universul cheilor posibile, m relativ mic și tabloul $T[0, \dots, m-1]$ în care se memorează elementele din tabelă. Elementul cu cheia k se plasează în T pe poziția $T[k]$. Dacă în tabelă nu există cheia k , atunci $T[k] = NIL$. O astfel de tabelă este reprezentată grafic în figura 3.1.

Algoritmii pentru operațiile de căutare, inserție și ștergere sunt extrem de simpli.

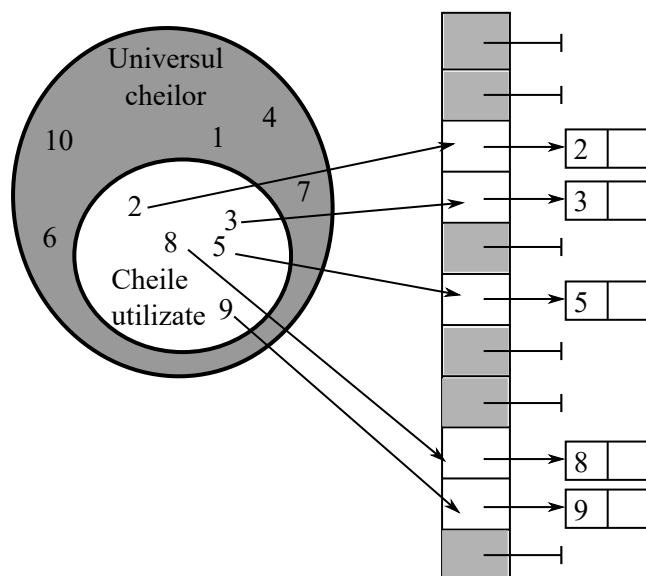


Figure 3.1: Tabelă cu adresare directă.

Algorithm: AD-CAUT

Intrare: tabela T și cheia k
 RETURN $T[k]$

Algorithm: AD-INSEREAZA

Intrare: tabela T și elementul x
 $T[x.cheie] \leftarrow x$

Algorithm: AD-STERGE

Intrare: tabela T și elementul x
 $T[x.cheie] \leftarrow NIL$

Complexitate: evident complexitatea este $O(1)$ pentru toate cele trei operații.

Observații:

- se presupune faptul că fiecare element x are o cheie unică, diferită de cheile tuturor celorlalte elemente din tabelă. Practic un element este identificat în mod unic prin cheie.
- o tabelă cu adresare directă este potrivită, atunci când universul U al cheilor este relativ redus și numărul de elemente memorate în tabelă este comparabil cu $|U| = m$.
- de obicei elementele dintr-o tabelă cu adresare directă sau dintr-o tabelă de dispersie au un câmp pentru cheie, după care se face căutarea, și un câmp pentru informația

de interes. Uneori cheia corespunde informației și atunci este suficient un singur câmp.

3.2 Tabele de dispersie - *Hash Tables*

Pentru situații în care numărul de elemente stocate într-o tabelă este mult mai mic decât universul cheilor, de exemplu atunci când orice număr natural poate fi cheie, tabelele cu adresare directă devin practic inutilizabile, datorită cantității de memorie necesare, care poate fi astfel nelimitată. În această situație se înlocuiește tabela cu adresare directă printr-o tabelă de dispersie.

Pentru memorarea unei tabele de dispersie se utilizează de asemenea un *array* T de dimensiune m : $T[0, \dots, m-1]$. Accesul la un element se face prin intermediul unei funcții de dispersie (repartizare) - *hash function* - $h : U \rightarrow \{0, 1, \dots, m-1\}$, $U \gg m$.

Elementul cu cheia k va fi plasat în tabelă pe poziția $T[h(k)]$. Spunem că, elementul cu cheia k este repartizat pe poziția $h(k)$ (*hashes to slot $h(k)$*). În figura 3.2 este reprezentată grafic o tabelă de dispersie.

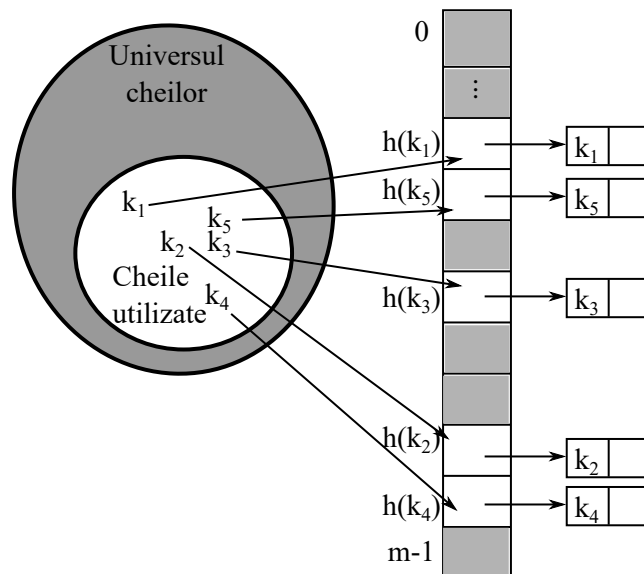


Figure 3.2: Tabelă de dispersie.

3.2.1 Problema coliziunilor

Exemplu de funcție de dispersie:

O funcție foarte simplă pentru repartizarea numerelor naturale într-o tabelă de dispersie este dată prin funcția *modulo* (mod), care reprezintă restul împărțirii la un număr

natural.

$$h : U \rightarrow \{0, 1, \dots, m-1\}, h(k) = k \bmod m$$

Considerând $m = 11$ și cheile $\{3, 12, 15, 17\}$ atunci: $h(3) = 3$, $h(12) = 1$, $h(15) = 4$, $h(17) = 6$ (fig. 3.3).

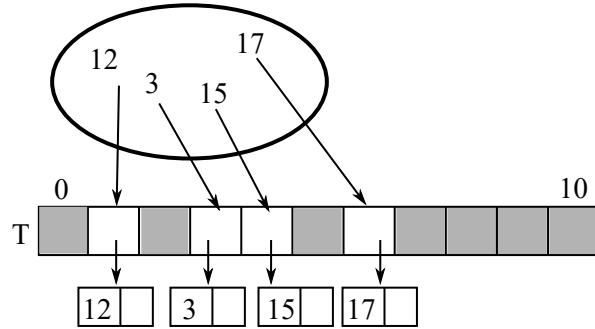


Figure 3.3: Tabela de dispersie cu funcția de dispersie $h(k) = k \bmod 11$ în care au fost plasate cheile 3, 12, 15 și 17.

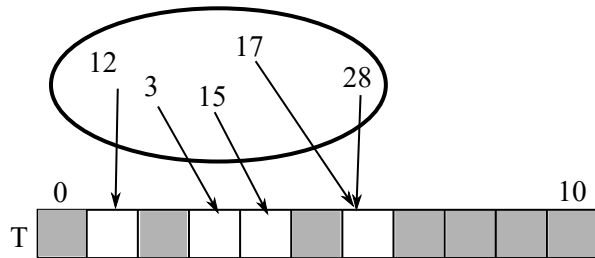


Figure 3.4: Coliziune între cheile 28 și 17.

Coliziunea

În cazul unei funcții de dispersie h este posibil ca pentru două chei diferite k_1 și k_2 să se obțină $h(k_1) = h(k_2)$, adică ambele elemente ar fi repartizate pe aceeași poziție. O astfel de situație se numește *coliziune*. În exemplul de mai sus $h(28) = h(17) = 6$ (fig. 3.4).

Se pune problema rezolvării coliziunilor. Ideal ar fi, evitarea completă a acestora. Din faptul că se presupune că $|U| > m$, prin tipul de repartizare descris mai sus, acest lucru nu este posibil. Totuși, prin construirea atentă a funcției de dispersie, poate fi redusă semnificativ probabilitatea unei coliziuni. Vor fi discutate pe parcursul acestui curs câteva metode de construcție a unei funcții de dispersie, care să îndeplinească acest lucru.

În practică rezolvarea coliziunilor pentru tabele de dispersie poate fi făcută de exemplu cu ajutorul listelor înlănțuite. Astfel, în loc de a stoca într-o poziție a tabelului de dispersie un singur element, se păstrează o listă de elemente (ideea de *bucket*). Lista de la poziția k va conține toate elementele a căror cheie determină repartizarea pe poziția respectivă.

Exemplu: Se consideră o tabelă de dispersie T cu dimensiunea $m = 11$ care utilizează liste înlănțuite și în care se introduc elementele cu cheile 33, 35, 55, 46, 12, 2, 7, 10, 11. Rezultatul este reprezentat în figura 3.5. Inserția în liste s-a realizat la începutul listelor.

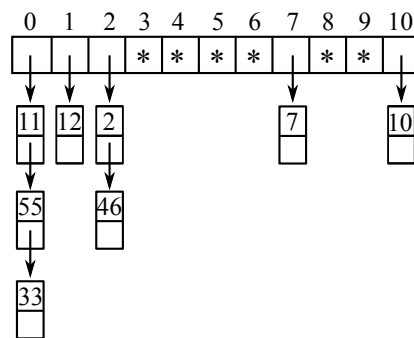


Figure 3.5: Tabela de dispersie de dimensiune $m = 11$ în care au fost plasate succesiv cheile 33, 35, 55, 46, 12, 2, 7, 10 și 11. Au fost marcate cu * pozițiile neocupate din tabelă.

O altă metodă de rezolvare a coliziunilor este *adresare deschisă*, metodă prezentată la sfârșitul capitolului.

3.2.2 Operațiile uzuale

În continuare sunt prezentați algoritmi pentru operațiile de căutare, inserare și ștergere a unui element dintr-o tabelă de dispersie ce utilizează liste înlănțuite pentru rezolvarea coliziunilor. Se consideră că fiecare element x din tabelă este o structură cu două câmpuri $x.cheie$ pentru cheie și $x.val$ pentru informația de interes. În algoritmi se consideră notația TDL pentru Tabelă de Dispersie cu Liste înlănțuite.

Algoritm: TDL-INSEREAZA

Intrare: tabela T de dimensiune m cu funcția de dispersie h și elementul x
 Insereaza x în capul listei $T[h(x.cheie)]$

Algorithm: TDL-CAUTA**Intrare:** tabela T de dimensiune m cu funcția de dispersie h și cheia k **Iesire:** elementul cu cheia x sau NIL în caz de eșecCauta elementul x cu cheia k în lista $T[h(k)]$ return x **Algorithm: TDL-STERGE****Intrare:** tabela T de dimensiune m cu funcția de dispersie h și elementul x Sterge x din lista $T[h(x.cheie)]$ **Complexitate:**

- funcția de inserție are evident complexitatea $O(1)$,
- la funcția de căutare complexitatea depinde de lungimea listelor înlănțuite
- funcția de ștergere, care are ca parametru un nod x deja identificat în lista înlănțuită, are complexitatea $O(1)$ dacă lista e dublu înlănțuită, altfel depinde de lungimea listei. Dacă întâi trebuie căutat elementul cu cheia k , apoi șters, se adaugă complexitatea căutării.

3.2.3 Analiza complexității

Considerăm tabela de dispersie T cu m poziții, în care rezolvarea coliziunilor se realizează prin înlănțuire. Presupunem că în T se stochează n elemente. Atunci numărul mediu de elemente stocate într-o listă este $\alpha = n/m$, unde α poate fi mai mic sau mai mare decât 1. α se numește *factor de încărcare* al tabelului.

Cea mai defavorabilă situație este aceea când toate cele n elemente se repartizează prin h pe aceeași poziție, adică sunt stocate în aceeași listă. În această situație complexitatea la căutare este $O(n)$. Pentru a evita astfel de situații este necesară alegerea unei funcții de dispersie potrivită. Modul de construcție a unor funcții de dispersie care să permită o repartizare cât mai uniformă va fi discutată ulterior.

Dispersia uniformă presupune ca probabilitatea ca un anumit element să fie de repartizat pe oricare dintre pozițiile din tabelă este aceeași.

Notăm cu n_j lungimea listei de pe poziția $T[j]$, $j = \overline{0, m-1}$. Observăm că $n_0 + n_1 + \dots + n_{m-1} = n$, iar valoarea medie sau valoarea **estimată** [1] pentru lungimea oricărei liste n_j este dată prin: $E[n_j] = n/m = \alpha$.

Considerând o funcție de dispersie care repartizează cheile uniform în tabela T și pentru care calculul lui $h(k)$ este $O(1)$, se demonstrează faptul că funcția de căutare a unei chei este $\Theta(1 + \alpha)$ cu ajutorul următoarelor două teoreme.

Teorema 3.2.1 *Într-o tabelă de dispersie care tratează coliziunile prin înlănțuire, o căutare fără succes are complexitatea medie de timp $\Theta(1 + \alpha)$ [1].*

Demonstrație: Presupunem că se caută în T cheia k , care nu există. Datorită faptului că s-a presupus o repartizare uniformă cu funcția h , probabilitatea repartizării cheii k pe oricare dintre poziții este aceeași. Timpul necesar pentru a constata că această cheie nu este în T este timpul necesar de parcurgere a listei $T[h(k)]$. În medie acest timp depinde de lungimea medie a listei $h(k)$, deci de $E[n_h(k)] = \alpha$. Dat fiind că se ține cont și timpul necesar pentru calculul valorii $h(k)$ se obține o complexitate de $\Theta(1 + \alpha)$.

Teorema 3.2.2 Într-o tabelă de dispersie care tratează coliziunile prin înlănțuire, o căutare cu succes are complexitatea medie de timp $\Theta(1 + \alpha)$ [1].

Demonstrație Situația căutării cu succes diferă de cea a căutării fără succes prin faptul că, timpul de căutare al elementului x cu cheia k din tabelă depinde de numărul de elemente aflate în lista $T[h(k)]$ înaintea lui x . Modul de calcul al acestui timp mediu presupune ceva cunoștințe de probabilități și depășește cadrul acestui curs. O descriere detaliată poate fi găsită în [1].

Dacă numărul de elemente stocate în tabelă, n , este proporțional cu dimensiunea m a tabelii, adică $n = am$, atunci complexitatea la căutare devine $\Theta(1 + am/m) = \Theta(1 + a) = \Theta(1)$. Cu alte cuvinte, dacă funcția de dispersie este aleasă în mod potrivit, astfel încât să ofere o repartizare uniformă și dacă dimensiunea tabelii este aleasă proporțional cu numărul estimat de elemente ce vor fi introduse în tabelă, timpul de căutare a unei chei devine în medie constant.

3.3 Metode de dispersie

Din discuția complexității rezultă că, o funcție de dispersie bună permite o repartizare aproape uniformă în tabelă. Pentru acest lucru poziția obținută la repartizare ar trebui să fie independentă de orice **pattern** care ar putea fi prezent în setul de date.

În plus, funcțiile de dispersie au ca mulțime a valorilor poziții în tabela de dispersie, deci elemente din mulțimea numerelor naturale. În cazul în care cheile sunt de exemplu șiruri de caractere - situație frecventă de altfel - este necesară stabilirea unei metode de interpretare a acestora ca numere naturale, trebuie deci determinată o funcție de la universul cheilor către mulțimea numerelor naturale. La finalul capitoului vor fi prezentate câteva funcții de dispersie pentru *string*-uri.

În continuare sunt prezentate câteva metode pentru construcția funcțiilor de dispersie pentru numere naturale.

3.3.1 Metoda diviziunii

Funcția de dispersie în acest caz este dată prin:

$$h : U \rightarrow \{0, 1, \dots, m-1\}, h(k) = k \bmod m$$

Alegerea dimensiunii tabelii. O alegere bună pentru dimensiunea tabelii m este un număr prim nu prea apropiat de o putere a lui 2.

Justificare. Dacă s-ar alege $m = 2^p$ atunci de fapt împărțirea la m ar reprezenta o *shift*-are a biților cheii, ceea ce ar crea o dependență de *pattern*-uri în structura cheilor, lucru ce trebuie evitat.

În plus acest număr trebuie ales depinzând de numărul de elemente care se estimează că vor fi introduse în tabelă, precum și de factorul de încărcare dorit.

Exemplu: dacă $n = 3000$ și numărul mediu de elemente per poziție este considerat 4 atunci, $\alpha = n/m \Rightarrow m = n/\alpha = 3000/4 = 750$. Se poate considera $m = 751$, care este un număr prim nu prea apropiat de o putere a lui 2.

3.3.2 Metoda multiplicării

În acest caz funcția de dispersie este de tipul

$$h(k) = \lfloor m(kA - \lfloor kA \rfloor) \rfloor, 0 < A < 1$$

unde $\lfloor x \rfloor$ reprezintă partea întreagă a lui x .

În cazul acestor funcții modul de alegere a lui m nu influențează modul de repartizare. Se demonstrează [2] faptul că, o alegere bună pentru A este

$$A = (\sqrt{5} - 1) / 2 \approx 0.618033$$

3.3.3 Dispersie universală

Dacă repartizarea în tabelă este realizată printr-o funcție dată, există posibilitatea ca, pentru anumite seturi de date, toate cheile să fie repartizate pe aceeași poziție, ajungându-se din nou la complexitatea $\Theta(n)$ la căutare.

Acest lucru poate fi evitat, dacă în loc să se folosească o singură funcție de dispersie, se utilizează o mulțime H de funcții de dispersie. La fiecare execuție se selectează în mod aleatoriu din H una dintre funcțiile de dispersie, care va fi apoi utilizată.

Utilizarea dispersiei universale are ca efect faptul că, pentru același set de date de intrare, execuții diferite ale programului produc în general tabele diferite, astfel putându-se evita în orice situație cel mai defavorabil caz, eventual prin reluare a repartizării - **rehashing**.

Definiție: O mulțime finită de funcții de dispersie H , care repartizează cheile dintr-un univers U într-o tabelă $T[0, \dots, m-1]$ se numește universală, dacă pentru orice două chei k și l , $k \neq l$, numărul de funcții din H pentru care $h(k) = h(l)$ este cel mult $|H|/m$ [1].

Acest lucru asigură faptul că, pentru o funcție aleasă în mod aleatoriu din H , probabilitatea unei coliziuni între k și l este cel mult $1/m$. Evident, probabilitatea alegerii oricărei funcții din H este $1/|H|$. Probabilitatea alegerii unei funcții care repartizează k și l pe aceeași poziție este cel mult $(|H|/m) * 1/|H| = 1/m$.

Construcție a unei clase universale de funcții de dispersie

În [1] este prezentată o metodă de construcție a unei astfel de clase universale de funcții de dispersie. Se alege un număr prim p suficient de mare, astfel încât să fie mai mare decât orice cheie posibilă. Evident $p \gg m$. Pentru fiecare $a \in \{1, \dots, p-1\}$ și $b \in \{0, 1, \dots, p-1\}$ se definește funcția de dispersie:

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

Mulțimea de astfel de funcții de dispersie este

$$H_{pm} = \{h_{ab} | a \in \{1, \dots, p-1\} \text{ și } b \in \{0, 1, \dots, p-1\}\}$$

cu $|H_{pm}| = p(p-1)$.

Se poate demonstra că mulțimea astfel definită este universală (vezi bibliografia recomandată).

3.3.4 Adresare deschisă

În cazul adresării deschise, fiecare poziție a tabelului de dispersie T conține un singur element. Pentru rezolvarea coliziunilor la inserție nu se utilizează liste înlanțuite, ci se testează diferite poziții, obținute pe baza funcției de dispersie, până când se determină o poziție liberă, pe care poate fi inserat elementul dorit.

La căutare de asemenea se testează diferite poziții până când se găsește elementul căutat sau se ajunge la o poziție neocupată.

Faptul că nu se utilizează liste înlanțuite, ci fiecare poziție conține cel mult un element, are ca urmare posibilitatea umplerii tabelului. În schimb se evită pointerii, iar memoria, care altfel ar fi fost necesară pentru listele înlanțuite, poate fi utilizată pentru o tabelă de dimensiune mai mare.

Un alt avantaj al acestui tip de repartizare apare în contextul utilizării memoriei cache în calculatoarele moderne. La capitolul despre liste a fost explicat dezavantajul acestora din punctul de vedere al vitezei de acces la memorie.

Modul de testare a pozițiilor în tabelă în cazul adresării deschise nu este liniar ci depinde de cheia inserată și de numărul de testări efectuate până la momentul curent.

Forma generală a unei funcții de dispersie în cazul adresării deschise este de forma:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

$h(k, t)$ = poziția de repartizare a cheii k după t teste.

În mod evident, pentru a permite testarea tuturor pozițiilor posibile și a nu reveni periodic la aceleași poziții, testate anterior, funcția $f(k, t)$ trebuie să producă pentru valorile lui $t \in \{0, \dots, m-1\}$ o permutare a mulțimii pozițiilor $\{0, \dots, m-1\}$. Căutarea unei chei într-o astfel de tabelă este ilustrată în figura 3.6.

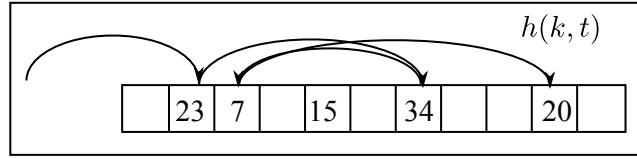


Figure 3.6: Căutarea elementului cu cheia 20. Acesta este găsit după 3 teste.

În continuare sunt prezentați algoritmi generali pentru inserarea și căutare într-o tabelă cu adresare deschisă.

Algoritm: TAD-INSEREAZA

Intrare: tabela T cum m elemente, elementul cu x cheia k

$t \leftarrow 0$ //numarul de teste deja efectuate

repetă

$j \leftarrow h(k, t)$ //determina pozitia pentru testare

dacă $T[j] = NIL$ **atunci**

$T[j] \leftarrow x$

return j

sfarsit_dacă

$t \leftarrow t + 1$ //incrementarea numarului de teste

cat_timp $t \neq m$;

return -1 //terminare cu esec

Algoritmul de căutare testează pentru o anumită cheie k aceeași secvență de poziții ca și algoritmul de inserție al cheii k .

Algoritm: TAD-CAUTA

Intrare: tabela T cum m elemente, elementul cu x cheia k

$t \leftarrow 0$

repetă

$j \leftarrow h(k, t)$

dacă $T[j].cheie = k$ **atunci**

return j

sfarsit_dacă

$t \leftarrow t + 1$

cat_timp $t \neq m$ și $T[j] \neq NIL$;

return -1

Operația de ștergere este problematică în cadrul adresării deschise. Dacă o cheie k este ștersă prin marcarea poziției cu NIL atunci o cheie p , inserată după k , dar care la inserție/ștergere presupune testarea poziției pe care s-a aflat k , nu va mai fi găsită prin algoritmul de mai sus.

O soluție a acestei probleme este marcarea pozițiilor șterse cu un marcaj special de poziție ștearsă, acest lucru presupune modificarea algoritmilor de căutare și inserare și în plus duce la o complexitate crescută a operației de căutare pentru un număr mare de

poziții șterse. Pe măsură ce crește numărul de poziții marcate ca șterse, crește timpul de căutare și scade performanța. Acest lucru se numește *contaminare*. Singura posibilitate de rezolvare a acestei probleme este prin *rehashing*.

În ceea ce privește alegerea unei funcții de dispersie pentru adresarea deschisă, aceasta poate fi realizată în diferite moduri. Vor fi descrise mai jos trei metode de construcție a unei astfel de funcții. Toate cele trei metode garantează faptul că secvența $\{h(k, 0), h(k, 1), \dots, h(k, m-1)\}$ este o permutare a mulțimii $\{0, 1, \dots, m-1\}$ pentru orice cheie k .

Funcții de dispersie cu testare liniară

Se consideră o funcție de dispersie auxiliară, de tipul celor descrise la începutul capitoului: $h_1 : U \rightarrow \{0, 1, \dots, m-1\}$. Se definește:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

$$h(k, t) = (h_1(k) + t) \bmod m$$

Exemplu: considerăm $m = 11$, $h_1(k) = k \bmod 11$ și

$$h(k, t) = ((k \bmod 11) + t) \bmod 11.$$

Atunci cheile 22, 33, 45, 59, 67, 13 vor fi repartizate după cum urmează:

$$h(22, 0) = (0 + 0) \bmod 11 = 0$$

$$h(33, 0) = (0 + 0) \bmod 11 = 0, \text{ este deja ocupat, deci testarea continuă}$$

$$h(33, 1) = (0 + 1) \bmod 11 = 1$$

$$h(45, 0) = (1 + 0) \bmod 11 = 1, \text{ este deja ocupat, deci testarea continuă}$$

$$h(45, 1) = (1 + 1) \bmod 11 = 2$$

$$h(59, 0) = (4 + 0) \bmod 11 = 4$$

$$h(67, 0) = (1 + 0) \bmod 11 = 1, \text{ este deja ocupat, deci testarea continuă}$$

$$h(67, 1) = (1 + 1) \bmod 11 = 2, \text{ este deja ocupat, deci testarea continuă}$$

$$h(67, 2) = (1 + 2) \bmod 11 = 3,$$

$$h(13, 0) = (2 + 0) \bmod 11 = 2, \text{ este deja ocupat, deci testarea continuă}$$

$$h(13, 1) = (2 + 1) \bmod 11 = 3, \text{ este deja ocupat, deci testarea continuă}$$

$$h(13, 2) = (2 + 2) \bmod 11 = 4 \text{ este deja ocupat, deci testarea continuă}$$

$$h(71, 0) = (2 + 3) \bmod 11 = 5$$

Inserearea cheilor din acest exemplu este ilustrată în figura 3.7.

Observație: Dezavantajul testării liniare este acela că, pe măsură ce se adaugă elemente, cresc secvențele de poziții succesive ocupate - **primary clustering**, ceea ce duce la creșterea complexității de inserție/căutare.

Un mod de îmbunătățire a acestui tip de testare este utilizarea funcțiilor de repartizare cu testare pătratică descrise în continuare.

Funcțiile de dispersie cu testare pătratică

Se consideră o funcție de dispersie auxiliară de tipul celor descrise la începutul capitoului: $h_1 : U \rightarrow \{0, 1, \dots, m-1\}$. Se definește:

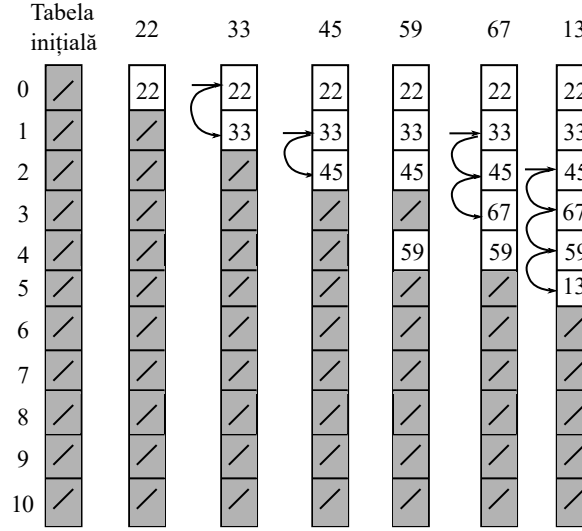


Figure 3.7: Inserarea cheilor 22, 33, 45, 59, 67, 13 într-o tabelă de dispersie de dimensiune $m = 11$ folosind testarea liniară. Pozițiile testate pentru fiecare element sunt marcate cu săgeți.

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

$$h(k, t) = (h_1(k) + c_1 t + c_2 t^2) \bmod m$$

c_1 și c_2 constante întregi pozitive auxiliare.

Prin acest mod de repartizare, se îmbunătățește ușor performanța. Problema este că, pentru două elemente k_1 și k_2 distincte, pentru care $h(k_1, 0) = h(k_2, 0)$, oricare ar fi t , $h(k_1, t) = h(k_2, t)$. Acest lucru are ca urmare faptul că, dacă se inserează numeroase elemente care au aceeași repartizare inițială, crește complexitatea inserției. Rezultă o formă oarecum atenuată de *clustering*, numită *secondary clustering* [1].

Una dintre cele mai bune metode de adresare deschisă se realizează cu ajutorul **dublei repartizări**.

Adresarea deschisă cu dublă repartizare

Se consideră două funcții de dispersie auxiliare distincte $h_1(k)$ și $h_2(k)$. Se construiește funcția de dispersie cu dublă repartizare:

$$h(k, t) = (h_1(k) + t h_2(k)) \bmod m$$

Observații:

- poziția inițială testată este $h(k, 0) = h_1(k)$, deci nu depinde de h_2 , oricare ar fi cheia k
- poziția $h(k, t)$ este la distanța $h_2(k)$, față de poziția $h(k, t-1)$, pentru orice $t > 0$

- pentru ca secvența de poziții $\{h(k, 0), h(k, 1), \dots, h(k, m-1)\}$ să fie o permutare a pozițiilor $\{0, 1, \dots, m-1\}$ trebuie ca funcția $h_2(k)$ să producă valori care sunt prime față de m . Dacă există o cheie k , pentru care $h_2(k)$ și m au un divizor comun $1 < d < m$, atunci cel mult după d teste se repetă din nou testarea poziției inițiale. Adică vor rămâne în tabelă poziții netestate!!

Satisfacerea condiției discutate mai sus, se poate obține dacă

- m putere a lui 2 și $h_2(k)$ produce întotdeauna un număr impar
- m prim și $h_2(k)$ produce numere naturale din $\{0, 1, \dots, m-1\}$, de exemplu considerăm funcțiile de repartiție

$$h_1(k) = k \bmod m$$

$$h_2(k) = 1 + (k \bmod m_1)$$

cu m prim și m_1 astfel încât m_1 ceva mai mic decât m , de exemplu $m_1 = m-1$.

Exemplu: Considerând funcțiile de dispersie $h_1(k) = k \bmod 11$ și $h_2(k) = 1 + (k \bmod 10)$, cheile 22, 33, 45, 59, 67, 13, 71 vor fi repartizate după cum urmează:

$$h(22, 0) = (22 \bmod 11) \bmod 11 = 0$$

$$h(33, 0) = (33 \bmod 11) \bmod 11 = 0, \text{ este deja ocupat, deci continuă testarea}$$

$$h(33, 1) = ((33 \bmod 11) + (1 + 33 \bmod 10)) \bmod 11 = (0 + 4) \bmod 11 = 4$$

$$h(45, 0) = (45 \bmod 11) \bmod 11 = 1$$

$$h(59, 0) = (59 \bmod 11) \bmod 11 = 4, \text{ este deja ocupat, deci continuă testarea}$$

$$h(59, 1) = (59 \bmod 11 + 1 + 59 \bmod 10) \bmod 11 = (4 + 10) \bmod 11 = 3$$

$$h(67, 0) = (67 \bmod 11) \bmod 11 = 1, \text{ este deja ocupat, deci continuă testarea}$$

$$h(67, 1) = ((67 \bmod 11) + (1 + 67 \bmod 10)) \bmod 11 = 9$$

$$h(13, 0) = (13 \bmod 11) \bmod 11 = 2$$

$$h(71, 0) = (71 \bmod 11) \bmod 11 = 6$$

Procesul de inserție a acestor chei este ilustrat în figura 3.8. Se observă că sunt necesare semnificativ mai puține încercări decât în cazul repartizării cu testare liniară.

Analiza complexității

Fie o tabelă de dispersie cu factorul de încărcare $\alpha = n/m < 1$, unde n = numărul de poziții ocupate și m = dimensiunea tabelului. De asemenea presupunem că funcția de dispersie asigură repartizarea uniformă. Acest lucru presupune că probabilitatea ca secvența de poziții testate pentru o anumită cheie k să fie oricare dintre cele $m!$ permutări ale mulțimii $\{0, 1, \dots, m-1\}$ este aceeași. Atunci complexitatea căutării unei chei este determinată pe baza următoarelor două teoreme.

Teorema 3.3.1 *Pentru o tabelă de dispersie cu adresare deschisă, cu factorul de încărcare $\alpha < 1$, numărul mediu de teste necesare pentru căutarea fără succes este cel mult*

Tabela inițială	22	33	45	59	67	13
0	22	22	22	22	22	22
1	/	/	45	45	45	45
2	/	/	/	/	/	13
3	/	/	/	59	59	59
4	/	33	33	33	33	33
5	/	/	/	/	/	/
6	/	/	/	/	/	/
7	/	/	/	/	/	/
8	/	/	/	/	/	/
9	/	/	/	/	67	67
10	/	/	/	/	/	/

Figure 3.8: Inserarea cheilor 22, 33, 45, 59, 67, 13 într-o tabelă de dispersie de dimensiune $m = 11$ folosind testarea dubla repartizare. Pozițiile testate pentru fiecare element sunt marcate cu săgeți.

$1/(1 - \alpha)$, presupunând o distribuție uniformă.

Teorema 3.3.2 Pentru o tabelă de dispersie cu adresare deschisă, cu factorul de încărcare $\alpha < 1$, numărul mediu de teste necesare pentru căutarea cu succes este cel mult $\frac{1}{\alpha} \log \frac{1}{1-\alpha}$, presupunând o repartizare uniformă.

Ambele teoreme sunt demonstrate cu ajutorul relațiilor probabilistice în [1].

Observații

- Testarea liniară și testarea pătratică nu garantează distribuție uniformă datorită fenomenului de *clustering*.
- Dubla repartizare se apropie cel mai mult de o distribuție uniformă.
- În practică, dacă factorul de încărcare devine foarte mare eficiența inserției, care presupune căutarea unei poziții neocupate, deci de fapt căutarea fără succes a cheii elementului ce se inserează, scade semnificativ. Astfel încât, atunci când α se apropie de 80% se recomandă redimensionarea tabelii și *rehashing*. De exemplu, în limbajul **Python**, care implementează dicționarele folosind adresarea deschisă, se pornește cu o tabelă de dimensiune 8 și atunci când s-a umplut 75% din tabelă, aceasta se redimensionează.

3.3.5 *Perfect hashing*

O funcție de distribuție ideală va produce pentru orice două chei inserate poziții de inserție diferite. Există situații, atunci când sunt cunoscute toate cheile posibile (de exemplu în cazul unui dicționar) să fie construite funcții de dispersie ideale în care să se evite orice coliziune. Acest lucru se numește *perfect hashing*. În [1] este propus un algoritm de dispersie perfect, care are la bază tot ideea de *bucket*-uri similar cu rezolvarea coliziunilor prin înlănțuire, doar că, în loc de liste înlănțuite se utilizează tabele de dispersie secundare.

În cazul general acest lucru este practic imposibil, astfel încât se recomandă construirea de funcții de dispersie care să producă o repartizare uniformă a cheilor în tabelă, rezultând o probabilitate mică de coliziune. Pentru chei numere întregi au fost prezentate câteva criterii de construcție a acestor funcții. În plus, o funcție de dispersie bună trebuie să utilizeze toate părțile unei chei pentru generarea valorii de dispersie. De exemplu, dacă se utilizează *string*-uri pentru chei, nu se vor considera doar primele 5 caractere pentru generarea valorii de dispersie sau dacă se folosesc chei numere întregi cu mai multe cifre, nu se vor utiliza doar anumite cifre ale acestora.

3.4 Tratarea cheilor de tip șir de caractere

În partea teoretică prezentată până acum s-a considerat doar distribuirea de chei numere naturale. În practică există chei de diferite tipuri, frecvent caractere sau șiruri de caractere. Acestor chei trebuie mai întâi să li se asocieze un număr natural.

În primul rând putem considera pentru fiecare caracter ASCII codul său numeric. Problema devine, cum combinăm codurile unei secvențe de caractere într-un număr natural, astfel încât să nu depășim valoarea maximă admisă și să evităm coliziunile care se produc, dacă pentru două șiruri de caractere se generează aceeași valoare numerică.

3.4.1 Funcții elementare de transformare a unui *string* într-un număr natural

Cele mai simple metode de a transforma o cheie de tip *string* într-o cheie numerică sunt *hashing*-ul aditiv și *XOR -hash*, descrise mai jos.

Hashing - aditiv - presupune adunarea codurilor numerice corespunzătoare tuturor caracterelor din șir. Acest tip de calcul al unei chei numerice este extrem de prost, deoarece generează coliziuni pentru toate permutările unui șir de caractere. În practică este deci inutilizabil.

XOR -hash - presupune aplicarea iterativă a operației de XOR asupra caracterelor din șir. Practic se pornește cu valoarea $h_val = 0$, care pentru fiecare caracter $sir[i]$ din șir se modifică prin instrucțiunea

$$h_val \leftarrow h_val \text{ XOR } sir[i]$$

Nici această metodă nu dă rezultate bune în ceea ce privește coliziunile, dar operația XOR poate fi utilizată în cazul unor funcții de repartizare mai sofisticate.

3.4.2 Funcții polinomiale de dispersie

Un tip funcție de dispersie frecvent utilizată pentru a asocia unui *string* un număr natural este un polinom de grad $L - 1$ unde L este lungimea șirului considerat:

$$h(sir) = sir[0] + sir[1] * p + sir[2] * p^2 + \dots + sir[L - 1] * p^{L-1} = \sum_{i=0}^{L-1} sir[i] * p^i$$

În cazul unor șiruri de caractere lungi există riscul de a obține valori extrem de mari, care să excedă maximumul admis pentru valorile numerice. Din acest motiv, adesea valoarea obținută se consideră modulo M , unde M este suficient de mare, ca să poată permite obținerea tuturor numerelor naturale din intervalul posibil $([0, \text{MAX_INT}])$. Formula devine astfel:

$$h(sir) = \left(\sum_{i=0}^{L-1} sir[i] * p^i \right) \bmod M$$

În [6] se folosesc metoda lui Horner și regulile pentru restul împărțirii la un număr natural pentru a calcula iterativ formula, astfel

$$h_L = 0, h_i = (h_{i+1} * p + sir[i]) \bmod M, i = L - 1, \dots, 0$$

Pentru a avea o probabilitate cât mai redusă de coliziuni, este esențială alegerea parametrilor p și M . În [6] este discutat modul de alegere al acestora. În diferite exemple practice se utilizează pentru p un număr prim, de exemplu 31 sau 33 (Bernstein hash), iar pentru M poate fi considerat un număr prim mai mic decât 2^{64} , o valoare întâlnită în literatura online fiind $10^9 + 7$.

3.4.3 Algoritmul FNV

Unul dintre cei mai cunoscuți algoritmi de asociere a unei valori naturale pentru un *string*, algoritmul FNV, utilizează o metodă similară cu cea a polinoamelor, doar că în loc de adunare între puteri se folosește operația de XOR. Algoritmul general este dat de:

Algoritm: FNV-Hash

Intrare: cheia k
 $hash \leftarrow FNV_Basis$
pentru fiecare byte b din șir **executa**
 $hash \leftarrow hash \times FNV_prime$
 $hash \leftarrow hash \text{ XOR } b$
sfarsit_for
return hash

Constantele FNV_Prime , și FNV_Basis depind de numărul de biți pe care este reprezentată valoare de dispersie rezultată. În [7] este explicat modul de alegere al acestor constante, iar pentru valori pe 32 de biți se consideră:

$$FNV_Prime = 16777619, FNV_Basis = 2166136261$$

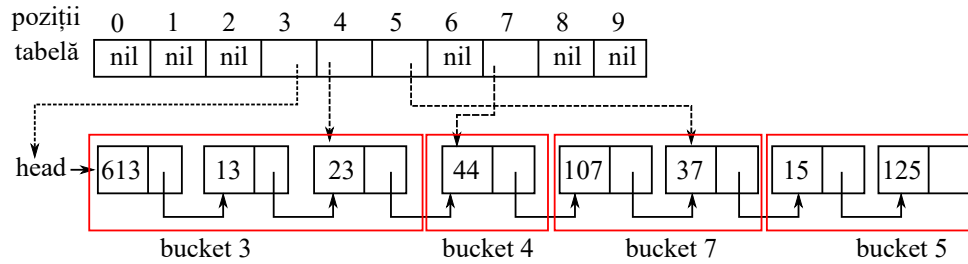


Figure 3.9: Reprezentarea unei tabele de dispersie de dimensiune 10, în care au fost inserate cheile 23, 44, 37, 107, 125, 15, 613. Dat fiind ca la bază se află un `std::forward_list`, inserția într-un **bucket** se face la începutul acestuia.

Observație Evident, în cazul tuturor acestor funcții care calculează o valoare naturală asociată unui **string**, pentru inserarea într-o tabelă de repartizare valoarea obținută trebuie considerată modulo dimensiunea tabelii.

3.5 Tabele de dispersie în STL

Modul de implementare

Modul de implementare al bibliotecilor `C++` depinde de compilator. În general din sursele online, rezultă că tabelele de dispersie, folosite de `std::unordered_set`, respectiv `std::unordered_map` au în implementare la bază liste înlănțuite, de fapt o singură listă, în care elementele din fiecare *bucket* sunt grupate în "subliste". Ordinea în care aceste "subliste" apar în tabelă nu este relevantă.

Tabela este reprezentată de un *array*, care conține pe fiecare poziție un iterator (pointer) către elementul dinaintea primului element (atunci când se folosește o listă simplu înlănțuită - de ex. GCC [13]) sau către primul element (în cazul unei liste dublu înlănțuite - implementarea Microsoft) din *bucket-ul* corespunzător cheilor repartizate pe poziția respectivă. Implementarea printr-o singură listă permite o iterare rapidă prin elementele tabelii de dispersie.

O reprezentare grafică a modului de implementare din GCC este prezentat în figura 3.9

Inițial, la declarare se pornește cu o tabelă ce dispune de 8 **bucket-uri**. Atunci când factorul de încărcare depășește valoarea maximă admisă, care în mod standard este $\alpha = 1.0$, are loc operația de *rehashing*. Aceasta presupune alocare nouă și copiere pentru toate elementele tabelii.

Ordinea în care sunt plasate în listă bucket-urile, precum și ordinea în care se află elementele unui bucket depinde de mai mulți factori, de exemplu de ordinea de inserare a elementelor sau de *rehashing*.

Chapter 4

Arbori

4.1 Arbori oarecare

În informatică un arbore este o structură ierarhică de noduri, conectate între ele. Într-un arbore:

- Unul dintre noduri se numește **rădăcină**.
- Nodurile sunt conectate între ele într-o relație ierarhică părinte - fiu. În această structură ierarhică se consideră ordonate nodurile pe niveluri. Pe primul nivel (nivelul 0) se află rădăcina. Pe fiecare nivel $k + 1$ se află fiii nodurilor de pe nivelul k , unde fiii unui nod x de pe nivelul k sunt acele noduri de pe nivelul $k + 1$ care sunt conectate de x printr-o muchie. Pentru aceste noduri x este **părintele / tatăl**. De obicei orientarea muchiei este de la părinte spre fii (*out-tree*), dar există și arbori în care orientarea este invers (*in-tree*). Orice nod are un părinte cu excepția rădăcinii.
- Orice nod poate avea 0 sau mai mulți fii. Fiii aceluiași nod se numesc **frați**. În plus niciun nod nu împarte vreun fiu cu alt nod.
- Orice nod care nu are niciun fiu se numește **frunză**. Un nod care nu este frunză se numește **nod intern**.
- Se numește **strămoș / ascendent** al unui nod x un nod y la care pot ajunge de la x urcând din părinte în părinte. Se numește **urmaș / descendent** al lui x un nod y la care pot ajunge printr-o succesiune de fii.

Un exemplu de arbore este prezentat în fig. 4.1. În acest arbore rădăcina este nodul cu cheia 21. Frunzele sunt nodurile cu cheile 3, 37, 7, 4, 19 și 0. Nodul cu cheia 9 este un strămoș al nodului cu cheia 37, iar nodul cu cheia 3 este un urmaș al nodului cu cheia 9.

Din punct de vedere al grafurilor, un arbore este un graf conex fără cicluri.

Un arbore se reprezintă ierarhic pe niveluri. De obicei se consideră că rădăcina se află pe nivelul 0.

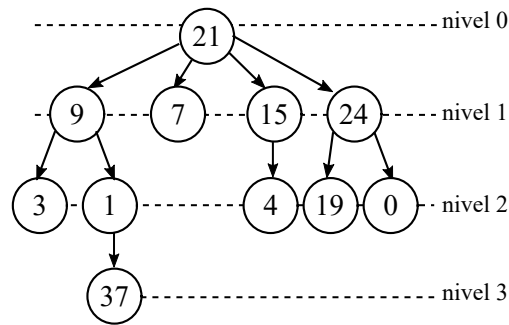


Figure 4.1: Arbore în reprezentarea ierarhică pe niveluri.

Definiție - adâncimea / înălțimea unui arbore: Lungimea drumului de la rădăcina r la un nod x se numește adâncimea nodului x . Adâncimea rădăcinii este 0.

Lungimea celui mai lung drum de la x la o frunză se numește *înălțime* și se notează prin $h(x)$.

Exemplu: Considerând arborele din figura 4.1 adâncimea nodului 1 este 2 iar înălțimea este 1. Înălțimea unei frunze este 0. **Atenție:** există documentații în care înălțimea unei frunze este considerată egală cu 1.

Definiție - subarbore: Se numește *subarbore de rădăcină x* al unui arbore, arborele format din nodul x împreună cu toți descendenții săi (fig. 4.2).

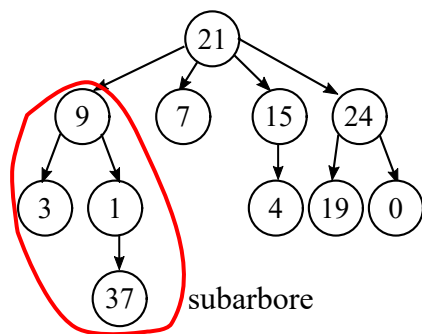


Figure 4.2: În arborele din figură nodurile cuprinse în marcajul roșu formează un subarbore, care are drept rădăcină nodul cu cheia 9

Arbori n -ari - definiții

- Se numește **arbore n -ar** un arbore pentru care fiecare nod are cel mult n fii.

- Un arbore n -ar se numește *arbore plin*, dacă fiecare nod intern are exact n fii.
- Un arbore n -ar se numește *arbore perfect*, dacă dacă fiecare nod intern are exact n fii și toate frunzele au aceeași adâncime.
- Un arbore n -ar se numește *complet*, dacă toate nodurile interne, cu excepția eventual a celor de pe penultimul nivel, au exact n descendenți, iar nodurile de pe ultimul nivel sunt așezate cel mai la stânga posibil pe nivelul respectiv.

În figura 4.3 este reprezentat un arbore complet.

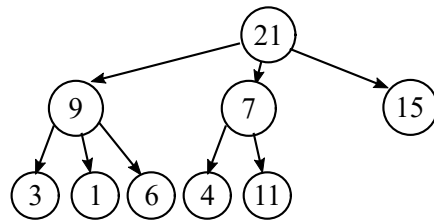


Figure 4.3: Arbore complet.

Definiție - Arbore binar: Un arbore binar este un arbore în care fiecare nod are cel mult doi descendenți direcți. Atunci când fiecare nod are 0 sau 2 fii, arborele se numește *arbore binar strict*. În cazul unui arbore binar un nod are un *fiu stâng* și un *fiu drept*.

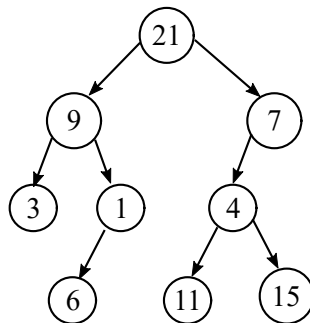


Figure 4.4: Arbore binar.

4.1.1 Reprezentarea arborilor

Reprezentare secvențială - se pretează în cazul arborilor compleți.

Un arbore n -ar complet de rădăcină r poate fi memorat într-un tablou unidimensional - *array* - în care pe poziția 0 se află rădăcina, pe următoarele n poziții se află fiii rădăcinii și în general pentru un nod oarecare aflat pe poziția i fiul al k -lea, $1 \leq k \leq n$ se află pe poziția $n * i + k$.

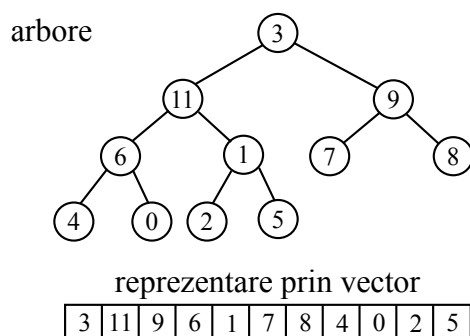


Figure 4.5: Arbore binar complet memorat într-un vector.

Reprezentare mixtă (pentru arbori binari)

Reprezentarea mixtă utilizează 3 vectori. Primul - INFO - conține informațiile din noduri, al doilea - *left* - are pe poziția i indicele fiului stâng al nodului i - în vectorul INFO -, iar vectorul al treilea - *right* - are pe poziția i indicele fiului drept al nodului i .

Pentru exemplul din fig. 4.6 a) reprezentarea mixtă este dată prin vectorii din tabelul din figura 4.6 b).

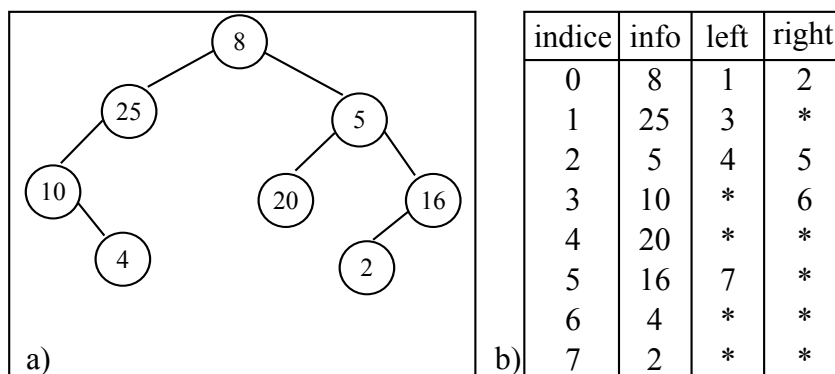
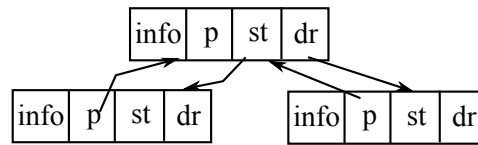


Figure 4.6: a) Arbore binar. b) Tabela de alocare mixtă.

Reprezentare cu ajutorul unei structuri de noduri

- **Pentru un arbore oarecare** (nu binar): pentru fiecare nod se utilizează o structură în care se reține informația, legătura către o listă / vector de descendenți direcți și (eventual) legătura către nodul părinte.
- **Pentru un arbore binar**: fiecare nod este reprezentat printr-o structură în care se reține informația, legătura către fiul stâng, legătura către fiul drept și legătura către părinte.



- Pornind de la reprezentarea unui arbore binar, se poate utiliza pentru arbori oarecare o structură asemănătoare, în care se păstrează informația, o legătură - fiu - către cel mai din stânga fiu și o legătură - frate - către primul frate din dreapta al nodului, precum și o legătură către părinte. Acest lucru este reprezentat grafic în figura 4.7.

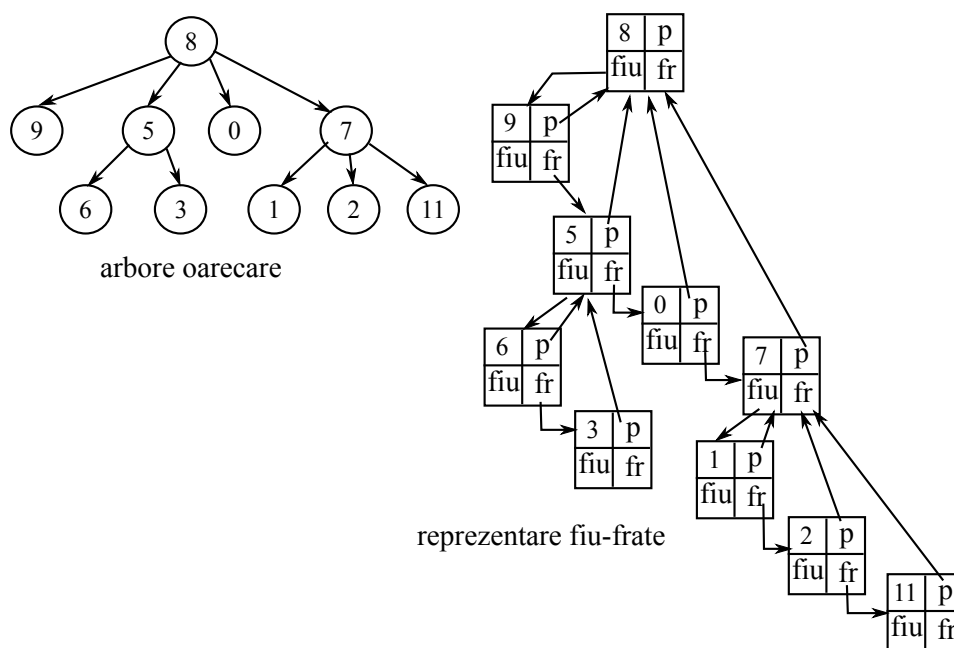


Figure 4.7: Arbore binar reprezentat printr-o structură de tip fiu-frate.

4.1.2 Parcurgerea arborilor

Există două moduri generale de parcurgere a arborilor oarecare:

- Parcurgerea în lățime:
 - presupune parcurgerea pe rând a fiecărui nivel de la stânga spre dreapta.
 - se poate realiza practic utilizând o coadă C :
- Parcurgerea în adâncime (preordine):

- În cazul parcurgerii în adâncime, fiii unui nod sunt vizitați tot de la stânga spre dreapta, dar trecerea de la nodul curent la fratele din dreapta se realizează numai după vizitarea tuturor descendenților nodului curent, deci a întregului subarboare al nodului respectiv.
- Se poate realiza utilizând o stivă S :

În cazul arborelui din figura 4.7, parcurgerea în lățime cu algoritmul de mai sus va avea ca rezultat afișarea cheilor în ordinea: 8, 9, 5, 0, 7, 6, 3, 1, 2, 11. Parcurgerea în preordine cu algoritmul de mai sus va avea ca rezultat afișarea cheilor în ordinea: 8, 9, 5, 6, 3, 0, 7, 1, 2, 11.

4.1.3 Parcurgerea arborilor binari

Parcurgerea în lățime a unui arbore binar este ilustrată grafic în figura 4.8. Algoritmul

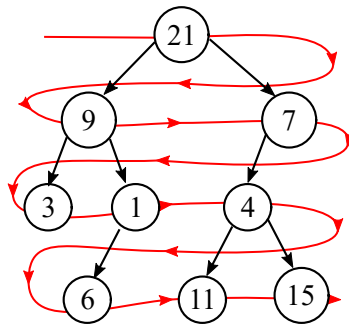


Figure 4.8: Ordinea de vizitare a nodurilor în cazul parcurgerii în lățime.

de parcurgere presupune utilizarea unei cozi, în care se stochează nodurile, pentru care încă nu au fost prelucrați fiii.

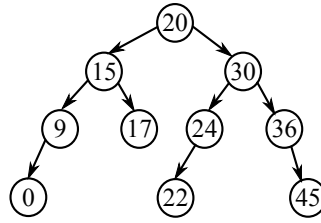


Figure 4.9: Arbore binar.

Algorithm: Latime**Intrare:** Arborele T **Iesire:** parcurgerea în lățimeDeclaram coada C $C.PUSH(T.rad)$ **cat timp** $!C.empty()$ **executa** $nod_curent \leftarrow C.TOP()$ $C.POP()$ scrie $nod_curent.info$ **daca** $nod_curent.st \neq nil$ **atunci** $C.PUSH(nod_curent.st)$ **sfarsit_daca** **daca** $nod_curent.dr \neq nil$ **atunci** $C.PUSH(nod_curent.dr)$ **sfarsit_daca****sfarsit_cat_timp**

Parcurgerea unui arbore binar în adâncime presupune parcurgerea fiecărui nod împreună cu subarborele stâng și subarborele drept. Subarborele stâng se ia în considerare înaintea subarborelui drept.

În funcție de ordinea în care parcurg rădăcina și subarborii, se disting trei tipuri de parcurgere:

- **Preordine (RSD)**: rădăcină, subarbore stâng, subarbore drept
- **Inordine (SRD)**: subarbore stâng, rădăcină, subarbore drept
- **Postordine (SDR)**: subarbore stâng, subarbore drept, rădăcină

Exemplu: Se consideră arborele binar din figura 4.9. Atunci:

RSD: 20, 15, 9, 0, 17, 30, 24, 22, 36, 45.

SRD: 0, 9, 15, 17, 20, 22, 24, 30, 36, 45.

SDR: 0, 9, 17, 15, 22, 24, 45, 36, 30, 20.

În practică parcurgerea poate fi făcută în mod secvențial utilizând o stivă, sau recursiv. Algoritmii recursivi sunt mai concisi și mai simpli de implementat dar presupun un consum mai mare de memorie.

Exemplu de algoritm secvențial - parcurgerea în preordine

Se utilizează o stivă S , în care se depun nodurile pentru care mai sunt de parcurs subarborii dreپți:

Algorithm: Preordine

Intrare: Arborele T

Iesire: parcurgerea în preordine

Declaram stiva $S \leftarrow \emptyset$

$nod \leftarrow T.rad$

repetă

cat timp $nod \neq nil$ **execută**

 scrie($nod.info$)

$S.push(nod)$

$nod \leftarrow nod.st$

sfarsit cat timp

dacă $S \neq \emptyset$ **atunci**

$nod \leftarrow S.top()$

$nod \leftarrow nod.dr$

sfarsit dacă

cat timp $!S.empty()$;

Algoritmi recursivi

Algorithm: Preordine(nod)

dacă $nod \neq nil$ **atunci**

 scrie($nod.info$)

 Preordine($nod.st$)

 Preordine($n=nod.dr$)

sfarsit dacă

Algorithm: Inordine(nod)

dacă $nod \neq nil$ **atunci**

 Inordine($nod.st$)

 scrie($nod.info$)

 Inordine($n=nod.dr$)

sfarsit dacă

Algorithm: Postordine(*nod*)

```
daca nod  $\neq$  nil atunci
    Postordine(nod.st)
    Postordine(nod.dr)
    scrie(nod.info)
sfarsit_daca
```

Exemple de aplicații ale arborilor: Arborii sunt structuri folosite în numeroși algoritmi din diverse domenii ale informaticii. În cele ce urmează sunt enumerate câteva dintre domeniile de utilizare a acestora.

- **Sortare:** Heap-sort - este un algoritm de sortare în complexitate $\theta(n \log n)$ și este prezentat ca aplicație la capitolul despre heap-uri și cozi de prioritate.
- **Codificare și compresie:** arborele de codificare Huffmann, care asociază caracterelor coduri de lungime diferită în funcție de frecvența cu care sunt utilizate caracterele într-un text / respectiv într-o limbă.
- **Compilatoare:** arbori sintactici pentru analiza codului sursă și construirea codului obiect
- **Procesare de imagine / grafică:** arbori Quad, arbori PR - pentru împărțirea și analizarea spațiului 2D.
- **Clasificare:** arbori de decizie în domeniul inteligenței artificiale.
- **Dicționare:** căutare eficientă cu ajutorul arborilor de căutare, subiect ce va fi discutat în capitolele ce urmează.

4.2 Probleme și exerciții

Chapter 5

Heap. Coadă de priorități

5.1 Heap

Definiție: Un heap binar este un arbore binar complet - fiecare nod intern are exact doi descendenți, cu excepția eventual a ultimului nod intern de pe penultimul nivel, iar frunzele se află doar pe ultimele două niveluri, frunzele de pe ultimul nivel sunt ordonate de la stânga spre dreapta - memorat cu ajutorul unui tablou unidimensional (vector) - *array*. În plus există o ordonare a cheilor într-un heap binar, determinată de tipul heap-ului.

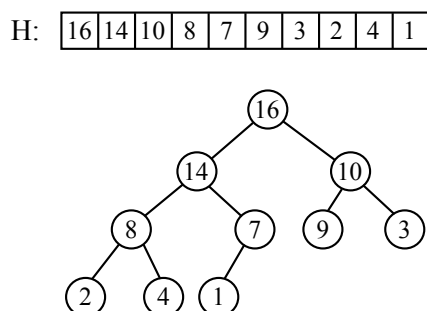


Figure 5.1: Exemplu de heap-max

1. **Max-heap:** informația din fiecare nod este mai mare decât informația din oricare descendent al său. Maximul din heap se află în rădăcină (fig. 5.1).
2. **Min-heap:** informația din fiecare nod este mai mică decât informația din oricare descendent al său. Minimul din heap se află în rădăcină.

Pentru implementare se poate utiliza structura heap H în care *size* - numărul de chei din Heap. Rădăcina heap-ului se află pe prima poziție a heapului $H[0]$. Pentru fiecare nod aflat pe poziția i :

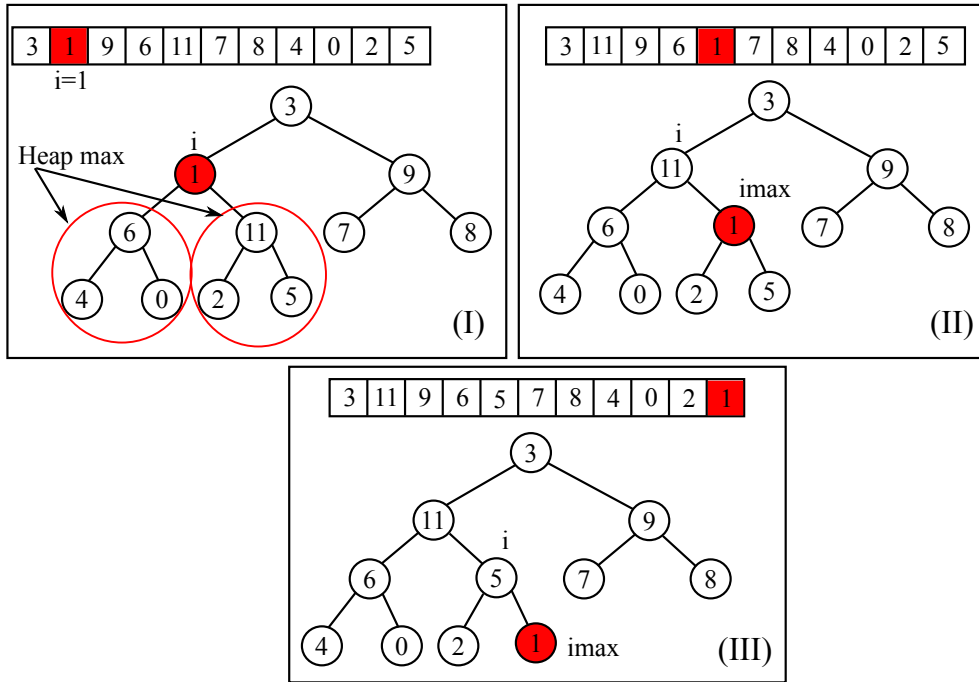


Figure 5.2: Funcția MAX_HEAP pornește în acest exemplu de la nodul i marcat cu roșu în fig. (I). Cei doi subarbori ai săi sunt heap-max. Funcția are două apeluri recursive, ilustrate în etapele (II) și (III).

- Părintele se află pe poziția $(i - 1)/2$.
- Fiul stâng se află pe poziția $2 * i + 1$.
- Fiul drept se află pe poziția $2 * i + 2$.

Observații:

- În cazul în care se începe în vectorul care stochează elementele heap-ului cu indicii de la 1, relațiile de mai sus se schimbă.
- Înălțimea arborelui care reprezintă heap-ul este $\log_2 n$ unde $n = H.size$ este numărul de noduri din heap.
- Complexitatea operațiilor de bază este proporțională cu înălțimea arborelui, adică $O(\log_2 n)$.

Construcția unui heap - max

Considerând un vector de elemente, pentru transformarea acestuia într-un heap-max sunt necesare două etape, reprezentate prin funcțiile:

- **MAX_HEAP**(H, i): în care H heap cu $H.size$ elemente și i indicele din vector a nodului de la se începe. Premiza este aceea că subarborii nodului i sunt heap-max și doar informația din nodul i strică eventual această proprietate. Funcția **MAX_HEAP** reface proprietatea de heap-max. Această funcție se întâlnește în literatură și sub denumirea *Sift-Down*.
- **CONSTR_HEAP**(H): pe baza funcției **MAX_HEAP** se construiește heap-ul.

Algorithm: MAX-HEAP

Intrare: Un heap H cu numărul de elemente $size$, poziția i

$st \leftarrow 2 * i + 1$

$dr \leftarrow 2 * i + 2$

$imax = i$

daca $st < H.size$ și $H[st] > H[imax]$ **atunci**

$imax = st$

sfarsit_daca

daca $dr < H.size$ și $H[dr] > H[imax]$ **atunci**

$imax = dr$

sfarsit_daca

daca $imax \neq i$ **atunci**

$H[i] \leftrightarrow H[imax]$

MAX-HEAP($H, imax$)

sfarsit_daca

În figurile 5.3 și 5.4 este ilustrată funcționarea acestui algoritm.

Complexitate: Algoritmul pornește de la nodul i și ajunge în cel mai defavorabil caz la o frunză, deci complexitatea depinde de înălțimea arborelui care este $h = \log_2 n$. Deci $T(n) = O(\log_2 n)$.

Se observă faptul că frunzele sunt heap-max. Atunci e suficient să pornim de la primul nod intern - pornind de la dreapta heap-ului H , adică primul care are cel puțin un descendent. Nodul respectiv se află pe poziția $H.size/2 - 1$, iar copiii săi sunt frunze, deci heap-max. Se aplică funcția de **MAX_HEAP** pentru nodul $H.size/2 - 1$, după care se trece la nodul precedent din vector și așa mai departe, până la rădăcină.

Algorithm: CONSTR_HEAP

Intrare: Un heap H cu $size$ elemente

pentru $i = size/2 - 1, 0, -1$ **executa**

MAX-HEAP(H, i)

sfarsit_for

În figura 5.4 este ilustrat procedeul de construcție a unui heap max.

Complexitate: fiecare apel al funcției **MAX_HEAP** are complexitatea $O(\log_2 n)$. **CONSTR_HEAP** efectuează $O(n)$ atfel de apeluri. Deci $T(n)$, unde prin T am notat

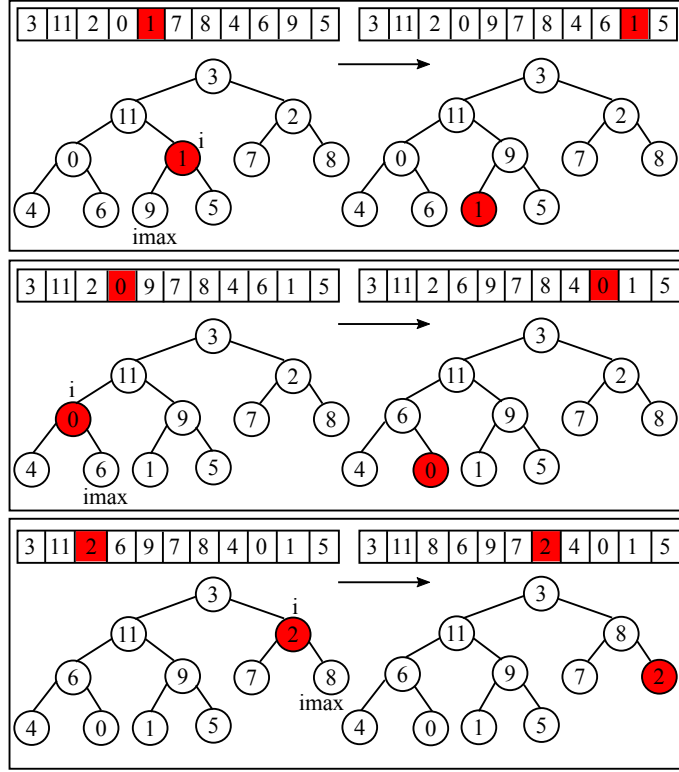


Figure 5.3: Funcția `CONSTR_HEAP` pornește în acest exemplu de la nodul de pe poziția a 5-a în heap-ul H , primul nod din dreapta vectorului ce are descendenți. Apoi se continuă cu nodurile de pozițiile 4 și 3.

complexitatea, pentru funcția `CONSTR_HEAP` este mărginită superior de $n \log_2 n$. Se demonstrează în literatură faptul că funcția are complexitate liniară, adică $T(n) = O(n)$, $n = H.size$.

5.2 Algoritmul de sortare *HeapSort*

Se observă faptul că într-un heap-max elementul maxim este plasat în rădăcina arborelui corespunzător, deci pe prima poziție din vectorul date. În vectorul sortat crescător, elementul maxim trebuie să se afle pe ultima poziție. Astfel vectorul poate fi sortat prin următorul algoritm: se interschimbă primul element din vectorul heap-ului cu ultimul. Se reduce dimensiunea heap-ului, apoi se reface proprietatea de heap-max aplicând funcția `MAX_HEAP` începând din vârful heap-ului. Procedura se reia pentru acest heap redus.

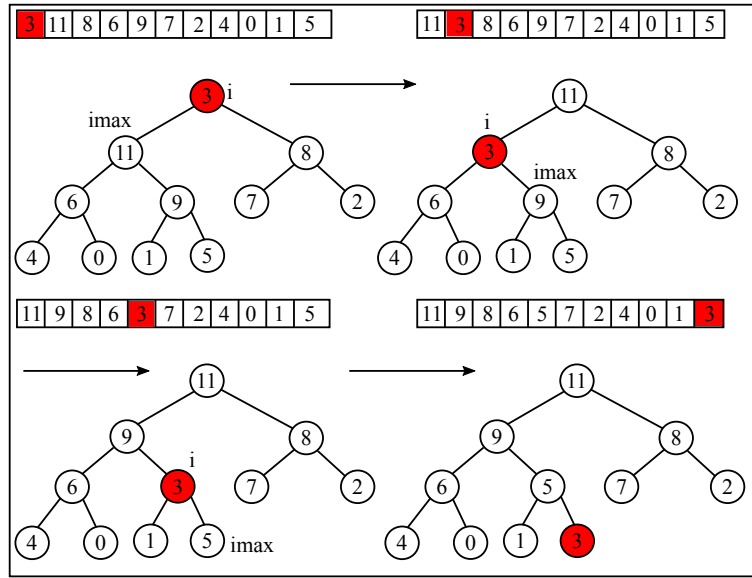


Figure 5.4: Funcția `MAX_HEAP` continuă de la poziția 2, la care însă nu sunt necesare modificări. Nodul cu cheia 11 satisface proprietatea de max-heap relativ la descendenții săi, astfel ca se trece la nodul de pe poziția 1. În final se obține un heap-max.

Algorithm: Heapsort

Intrare: Un vector v cu n

Construiește heap-ul H cu funcția `BUILD-MAX-HEAP`

$size \leftarrow n$

pentru $i = n - 1, 1, -1$ **executa**

$H[i] \leftrightarrow H[0]$

$H.size \leftarrow H.size - 1$

`Max-Heapfy`($H, 0$)

sfarsit_for

În figurile 5.5 și 5.6 este ilustrată funcționarea algoritmului Heapsort.

Complexitate:

- apelul funcției `CONSTR_HEAP` este $O(n)$
- apelul pentru `MAX_HEAP` este $O(\log_2 n)$
- funcția `MAX_HEAP` se apelează de $n - 1$ ori

Rezultă $T(n) = O(n \log_2 n)$.

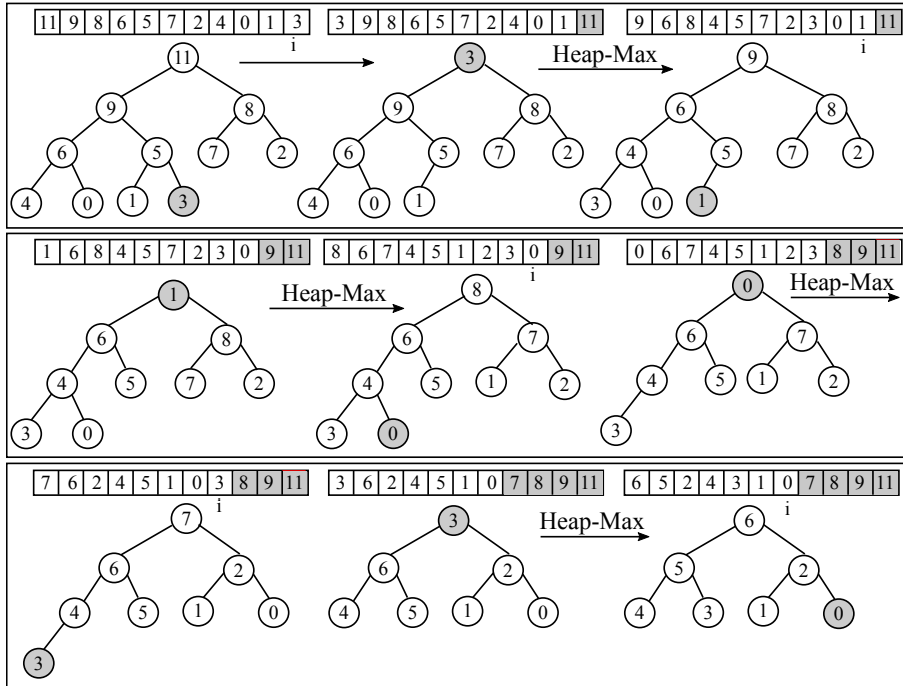


Figure 5.5: Algoritmul Heapsort.

5.3 Coadă de prioritate

Definiție: O coadă de prioritate este o structură de date utilizată pentru păstrarea unei mulțimi dinamice de date S în care fiecărui element i se asociază o valoare numită *prioritate*. Pentru gestionarea eficientă a cozilor de prioritate se utilizează heap-uri. Există cozi de max-prioritate - heap-max - și cozi de min-prioritate - heap-min.

În continuare vom considera cozi cu max-prioritate.

Operații în cozi de prioritate (max-heap):

- Inserția unui element nou
- Determinarea elementului de prioritate maximă
- Extragerea elementului de prioritate maximă
- Creșterea priorității unui element

Utilizare: Cozile de prioritate pot fi utilizate pentru gestionarea proceselor pe un calculator. La fiecare moment se execută procesul de prioritate maximă. Procese noi se pot insera în coadă.

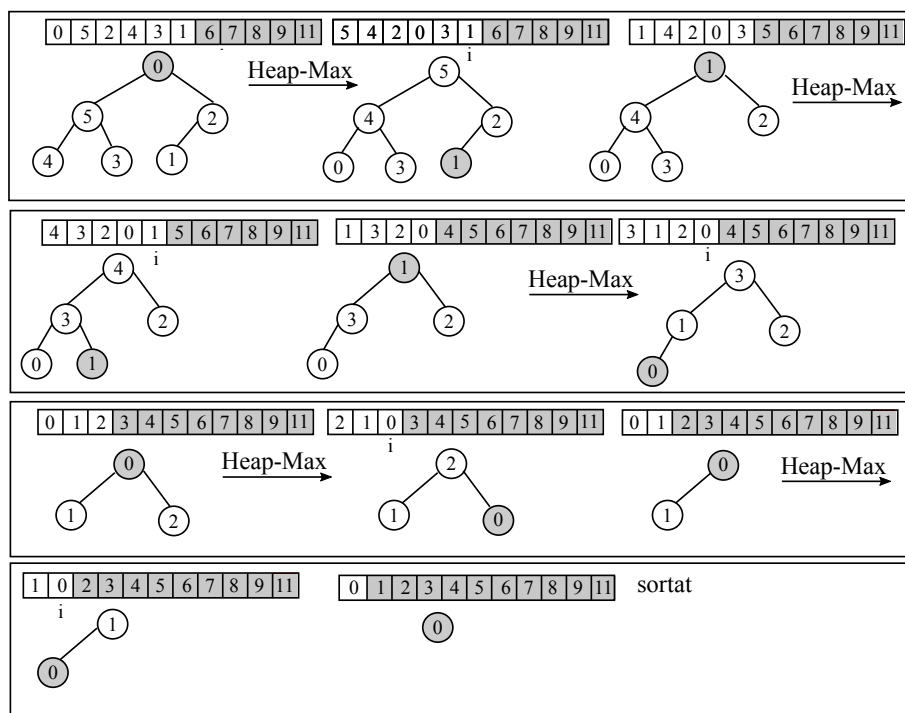


Figure 5.6: Algoritmul Heapsort.

Alt exemplu de aplicație este în implementarea unor algoritmi de căutare informată (Dijkstra, A^*).

1. Determinarea elementului cu prioritate maximă: acesta se află în nodul rădăcină al heap-max:

Algorithm: PRIORITY-MAX

RETURN H[0]

Complexitate: $O(1)$

2. Extragerea maximului: Se plasează ultimul element din heap pe prima poziție, se scade dimensiunea heap-ului cu o unitate iar apoi se reface heap-ul prin apelul MAX_HEAP.

Algorithm: PRIORITY-EXTRACT-MAX

Intrare: un heap cu câmpurile H și $size$

$$H[0] \leftarrow H[H.size - 1]$$
$$H.size \leftarrow H.size - 1$$
$$\text{Max-Heapfy}(H, 0)$$

Complexitate: $O(\log_2 n)$ - se aplică o singură dată MAX_HEAP.

În figura 5.7 este prezentat un exemplu pentru extragerea maximului.

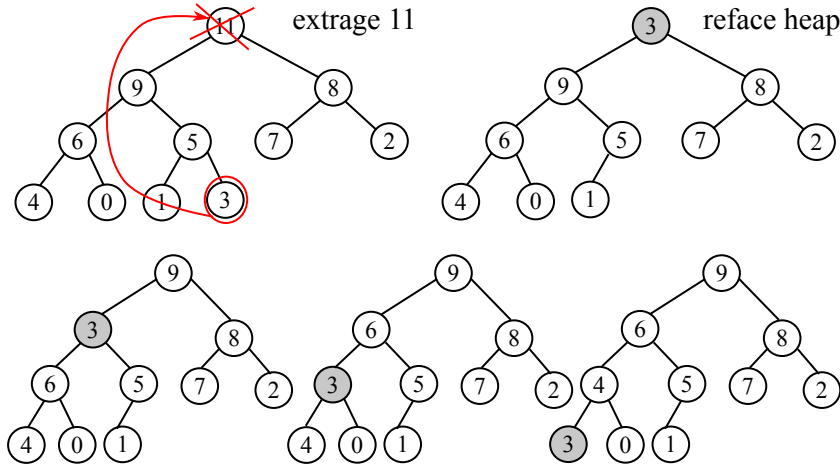


Figure 5.7: Extragerea maximului.

3. Creșterea priorității elementului de pe poziția i . Prin această modificare este posibil să se strice proprietatea de heap-max, deoarece s-ar putea ca noua valoare a elementului de pe poziția i să fie mai mare decât a părintelui său. Pentru aceasta se merge din părinte în părinte către rădăcină, până se găsește o poziție potrivită pentru noua valoare. Algoritmul următor este cunoscut în literatură și sub denumirea de *Sift-Up*.

Algoritm: INCREASE-PRIORITY

Intrare: poziția i , valoarea val cu care se modifică elementul $H[i]$

daca $val > H[i]$ **atunci**

$H[i] \leftarrow val$

$p \leftarrow (i - 1)/2$

cat timp $i > 0$ **si** $H[p] < val$ **executa**

$H[i] \leftarrow H[p]$

$i \leftarrow p$

$p \leftarrow (i - 1)/2$

sfarsit_cat_timp

$H[i] \leftarrow val$

sfarsit_daca

Complexitate: $O(\log_2 n)$ - se pornește de la o frunză către rădăcină și deci complexitatea depinde de înălțimea arborelui.

În figura 5.8 este prezentat un exemplu pentru creșterea priorității.

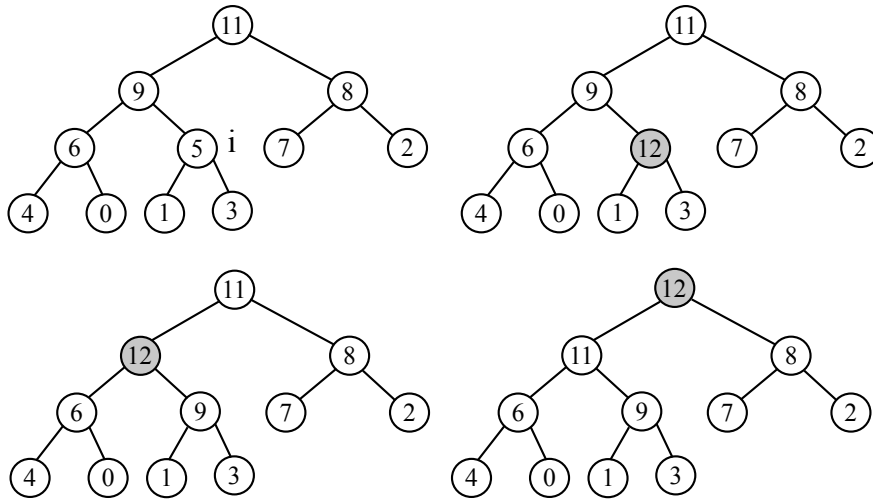


Figure 5.8: Creșterea priorității nodului cu cheia 5 la valoarea 12.

4. Inserția unui element nou într-un max-heap: se mărește dimensiunea heap-ului, se plasează noul element pe ultima poziție cu prioritatea considerată 0 și apoi se aplică funcția `CP_CRESTE_PRIORITATE` pentru acest nou element cu valoarea priorității asociate.

Algoritm: PRIORITY-INSERT

Intrare: elementul *val*, care se inserează în coadă
 $H[H.size] \leftarrow 0$
 $H.size \leftarrow H.size + 1$
 Increase-Priority(*H*, *H.size* - 1, *val*)

Complexitate: $O(\log_2 n)$ - se pornește de la o frunză către rădăcină și deci complexitatea depinde de înălțimea arborelui.

Observații:

- Pe un max-heap se pot implementa cu complexitatea $O(\log_2 n)$ operații cu cozi de prioritate.
- Construcția unui heap-max, care s-a făcut prin apelarea funcției `MAX_HEAP`, se poate realiza și prin inserții succesive ale nodurilor în heap.

Chapter 6

Arbori binari de căutare

6.1 Noțiuni de bază

Definiție: un arbore binar de căutare este un arbore binar în care fiecare nod are o valoare numită cheie, iar pentru fiecare nod toate nodurile din subarborele stâng au cheile mai mici decât cheia părintelui și toate nodurile din subarborele drept au cheile mai mari decât cheia părintelui.

În cazul în care relația de ordine nu este strictă, dacă nodurile cu chei egale se inserează pe aceeași parte a arborelui, inserția multor noduri cu chei egale are ca urmare obținerea unui arbore relativ dezechilibrat, având ca urmare o creștere a complexității operațiilor. În continuare se consideră arbori binari de căutare cu chei distincte. Cazul cheilor egale se va discuta separat la sfârșitul cursului.

În figura 6.1 este prezentat un exemplu de arbore binar de căutare.

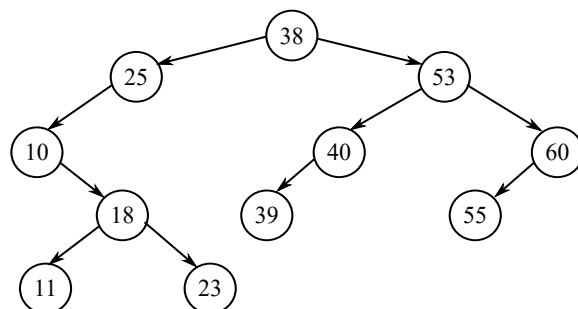


Figure 6.1: Exemplu de arbore binar de căutare.

Fiecare nod x al arborelui poate fi considerat o structură cu câmpurile: $x.info$ - cheia nodului, $x.st$ și $x.dr$ - pointeri către fiul stâng și respectiv fiul drept, $x.p$ - pointer către părintele nodului x .

6.2 Operații într-un arbore binar de căutare

Operațiile de bază într-un arbore binar de căutare sunt:

- Căutarea binară a unei chei.
- Determinarea nodului cu cheia minimă/maximă din arbore.
- Căutarea binară a succesorului / predecesorului unui nod
- Inserție/ștergere a unui nod cu o anumită cheie

Complexitatea operațiilor într-un arbore binar de căutare depinde de înălțimea h a acestuia. în funcție de numărul n de noduri din arbore putem avea următoarele situații limită.

1. Fiecare nod x are cel mult un fiu. Un astfel de exemplu este ilustrat în figura ...
a. În acest caz înălțimea arborelui este $h = n - 1$, care este și înălțimea maximă posibilă.
2. Fiecare nod care nu este frunză are exact doi fii și toate nivelurile cu excepția eventual a ultimului sunt pline. Această situație este ilustrată în figura ... b. În acest caz se obține înălțimea minimă a arborelui, care este $\log_2 n$.

Din aceste observații reiese faptul că operațiile de bază într-un arbore binar de căutare cu n noduri este delimitată prin $O(\log_2 n) \leq T(n) \leq O(n)$. Astfel, în cazul unui arbore binar de căutare oarecare nu poate fi garantată complexitatea căutării binare, adică $O(\log_2 n)$.

Parcursarea arborelui în inordine, pentru obținerea cheilor în ordine sortată.

Există arbori binari de căutare care se autobalansează, de exemplu arborii AVL și arborii roșu-negru. Pentru aceștia se demonstrează faptul că au complexitatea operațiilor de ordinul $O(\log_2 n)$.

6.2.1 Căutarea binară

Problemă: Considerând un arbore binar de căutare T cu rădăcina $T.rad$, să se determine nodul cu o cheie dată k .

Soluție: La fiecare moment dat compar cheia nodului curent x , $x.info$, cu k . Dacă $x.info = k$ atunci se returnează nodul x . Dacă $x.info < k$ atunci se continuă căutarea în subarborele drept al lui x , altfel se continuă căutarea în subarborele stâng al lui x .

Algoritm: Căutare binară

Intrare: Arborele binar de căutare T și elementul k care se caută**Iesire:** nodul x cu cheia k sau nil $x \leftarrow T.rad$ **cat_timp** $x \leq nil$ și $x.info \neq k$ **executa** **daca** $k < x.info$ **atunci** $x \leftarrow x.st$ **sfarsit_daca** **altfel** $x \leftarrow x.dr$ **sfarsit_daca****sfarsit_cat_timp**return x

6.2.2 Minimul dintr-un arbore de căutare

Nodul cu informația minimă din subarborele de rădăcină x a unui arbore binar de căutare poate fi găsit pornind de la nodul x și coborând în descendenții stângi până la cea mai din stânga frunză. Funcția AB_MIN returnează nodul cu informația minimă.

Algoritm: AB_MIN

Intrare: Arborele binar de căutare T și nodul x **Iesire:** nodul cu cheia minimă din subarborele de rădăcină x $y \leftarrow x$ **cat_timp** $y.st \neq nil$ **executa** $y \leftarrow y.st$ **sfarsit_cat_timp**return y

Observație: maximul se determină în mod similar și anume parcurgând descendenții dreپți până la cea mai din dreapta frunză.

6.2.3 Succesorul binar

Succesorul unui nod x într-un arbore binar de căutare este acel nod y din arbore, a cărui cheie are valoarea imediat următoare cheii lui x în șirul sortat al valorilor din arbore.

- Poate fi determinat prin comparații
- Dacă există, este:
 - Cel mai mic element din $x.dr$, dacă $x.dr \neq Nil$
 - Un nod părinte y pentru care x se află în subarborele stâng al lui y , dacă x nu are descendent drept.
- Dacă x este nodul cu cea mai mare cheie, atunci x nu are succesor.

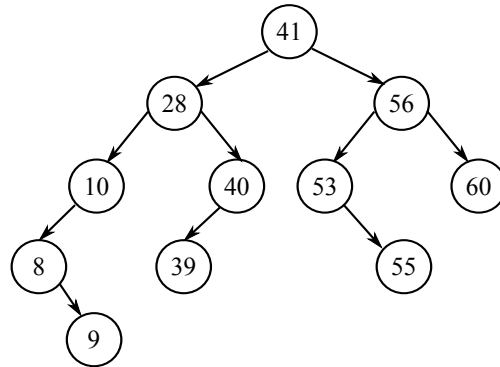


Figure 6.2: Arbore binar de căutare.

Exemplu: În arborele din figura 6.2 obținem:

$AB_SUCCESOR(41) = 53$
 $AB_SUCCESOR(39) = 40$
 $AB_SUCCESOR(9) = 10$
 $AB_SUCCESOR(60)$ - nu există

Algoritm: Succesor

Intrare: Arborele binar de căutare T și nodul x

Iesire: nodul y , sucesor binar al lui x

daca $x.dr \neq nil$ **atunci**

$y = AB_MIN(x.dr)$

sfarsit_daca

altfel

$y \leftarrow x.p$

cat timp $y \neq nil$ și $x = y.dr$ **executa**

$\bar{x} \leftarrow y$

$y \leftarrow y.p$

sfarsit_cat_timp

sfarsit_daca

return y

6.2.4 Inserarea unui nod

Se consieră arborele binar de căutare T cu rădăcina $T.rad$ și nodul z , care are câmpurile

$$z.info = k, z.st = Nil, z.dr = Nil.$$

Se dorește inserarea acestui nod în arborele binar T .

Ideea principală este următoare: pornind de la rădăcină se coboară în arbore, până la

un nod, care are cel mult un fiu și care poate fi părintele nodului z . Pentru a respecta proprietatea de arbore binar de căutare, dacă informația nodului curent x este mai mare decât $z.info$, atunci z se va insera în subarborele stâng al lui x , altfel se va insera în subarborele drept. În algoritmul următor x reprezintă nodul curent, care inițial este $T.rad$ = rădăcina lui T , y reprezintă părintele lui x , inițial Nil .

Algorithm: Insert

Intrare: Arborele binar de căutare T și cheia k

Iesire: Arborele în care s-a inserat nodul z cu cheia k

aloca memorie pentru nodul z

$z.info \leftarrow k$

$z.st \leftarrow nil$

$z.dr \leftarrow nil$

$x \leftarrow T.rad$

$y \leftarrow nil$

cat timp $x \neq nil$ **executa**

$\bar{y} \leftarrow x$

daca $z.info < x.info$ **atunci**

$x \leftarrow x.st$

sfarsit_daca

altfel

$x \leftarrow x.dr$

sfarsit_daca

sfarsit_cat_timp

$z.p \leftarrow y$

daca $y = nil$ **atunci**

$T.rad \leftarrow Z$

sfarsit_daca

altfel

daca $z.info < y.info$ **atunci**

$y.st \leftarrow z$

sfarsit_daca

altfel

$y.dr \leftarrow z$

sfarsit_daca

sfarsit_daca

În figura 6.3 este ilustrat algoritmul de inserție a unui nod cu cheia 38 într-un arbore binar de căutare.

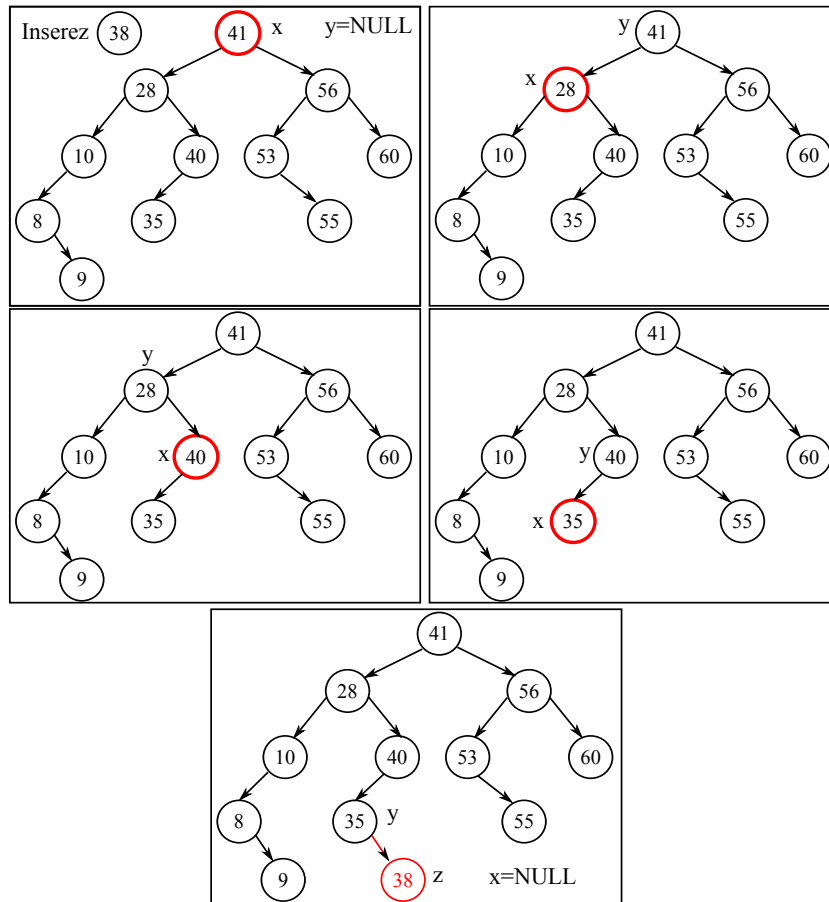


Figure 6.3: Inserarea nodului cu cheia 28 într-un arbore binar. Nodul curent cu care se compară nodul ce se inserează este marcat cu roșu.

6.2.5 Ștergerea unui nod

Ștergerea unui nod z dintr-un arbore binar T cu rădăcina $T.rad$ este ceva mai elaborată decât inserarea. Sunt luate în considerare următoarele cazuri:

1. z nu are fii și atunci este pur și simplu înlocuit cu Nil
2. z are un singur fiu nenul. Atunci se înlocuiește z cu acel fiu
3. z are doi fii nenuli. Atunci se determină succesorul y al lui z care se află în subarborul drept al lui z și evident nu are descendent stâng. Apoi se înlocuiește nodul z cu nodul y , iar y se înlocuiește cu fiul său drept.

Observație: În cazul în care z are doi descendenți nenuli, el poate fi înlocuit și cu predecesorul său.

În funcția AB_STERGE se consideră T arborele binar, z nodul care trebuie șters și y nodul cu care se înlocuiește z . Cele 3 cazuri descrise mai sus vor fi cuprinse în funcție în următoarele cazuri:

1. z nu are fiu stâng $\Rightarrow$ se înlocuiește z cu fiul drept - eventual Nil. Acest caz include și cazul în care z nu are nici un fiu.
2. z nu are fiu drept $\Rightarrow$ se înlocuiește z cu fiul stâng
3. z are ambii fii nenuli $\Rightarrow y =$ succesorul lui z care se află în subarborele drept al lui z și are fiul stâng Nil
 - a. y este descendentul drept direct al lui $z \Rightarrow$ se înlocuiește z cu y (fiul drept al lui y rămâne neschimbat iar fiul stâng al lui z devine fiul stâng al lui y)
 - b. y nu este descendentul drept direct al lui $z \Rightarrow$ se înlocuiește y cu fiul său drept iar apoi se înlocuiește nodul z cu nodul y .

În bibliografie (T.H. Cormen - *Introduction to Algorithms*) - este propusă utilizarea unei funcții ajutătoare TRANSPLANT(T, u, v) care înlocuiește în arborele T nodul u ca subarbore cu nodul v - de fapt această funcție realizează doar managementul legăturilor între părintele lui u și nodul v , legăturile cu fiii se realizează separat în funcția de ștergere propriu-zisă.

Cazurile luate în considerare de către funcția AB_DELETE sunt ilustrate în figura 6.4.

Algorithm: Transplant

Intrare: Arborele binar de căutare T , nodurile u și v

Iesire: Arborele în care s-a înlocuit u cu v ca fiu al lui $u.p$

daca $u.p = nil$ **atunci**

 | $T.rad \leftarrow v$

sfarsit_daca

altfel

 | **daca** $u = u.p.st$ **atunci**

 | $u.p.st \leftarrow v$

 | **sfarsit_daca**

 | **altfel**

 | $u.p.dr \leftarrow v$

 | **sfarsit_daca**

sfarsit_daca

daca $v \neq nil$ **atunci**

 | $v.p \leftarrow u.p$

sfarsit_daca

Modul de funcționare al funcției AB_TRANSPLANT este ilustrat în figura 6.5.

În continuare este prezentat algoritmul de ștergere a unui nod z dintr-un arbore binar de căutare T .

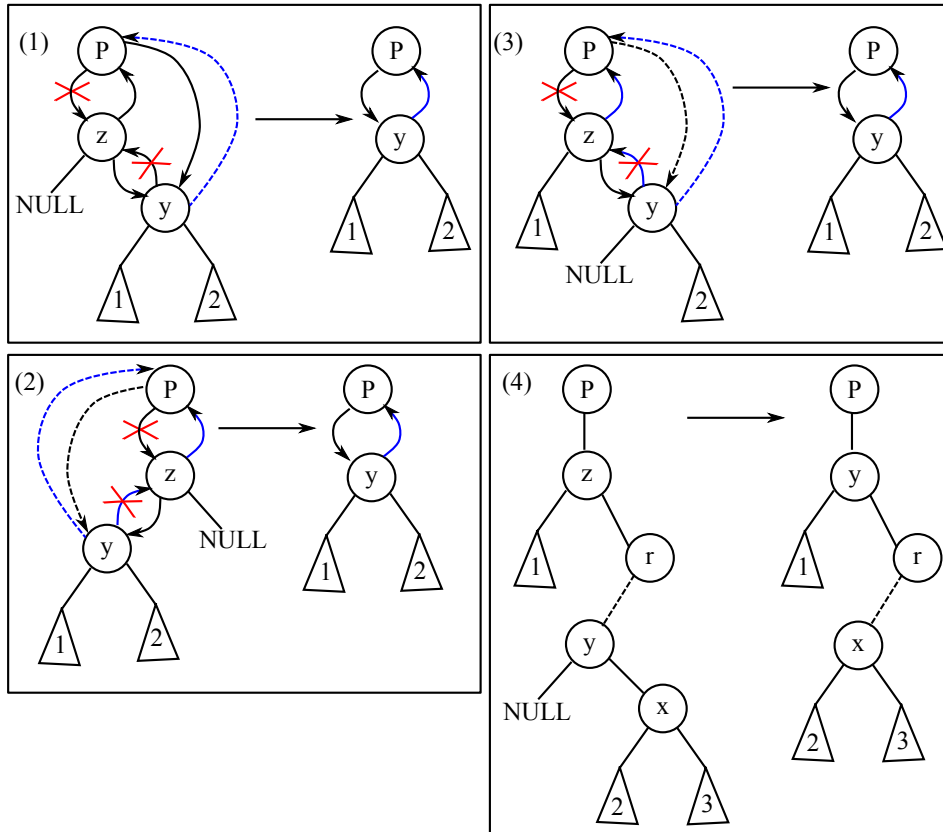


Figure 6.4: Ștergerea unui nod z care: (1) are fiul stâng nul; (2) are fiul drept nul; (3) are ambii fii nenuli, dar succesorul y este descendent direct al lui z ; (4) are ambii fii nenuli, dar succesorul y nu este descendent direct al lui z .

Algoritm: AB_DELETE

Intrare: Arborele binar de căutare T și nodul z , găsit cu funcția *Search*

Iesire: Arborele din care s-a șters z

daca $z.st = nil$ **atunci**

 Tansplant(z , $z.dr$)

sfarsit_daca

altfel

daca $z.dr = nil$ **atunci**

 Tansplant(z , $z.st$)

sfarsit_daca

altfel

$y \leftarrow \text{succesor}(z)$

daca $y \neq z.dr$ **atunci**

 Transplant(y , $y.dr$)

$y.dr \leftarrow z.dr$

$z.dr.p \leftarrow y$

sfarsit_daca

 Transplant(z , y)

$y.st \leftarrow z.st$

$z.st.p \leftarrow y$

sfarsit_daca

sfarsit_daca

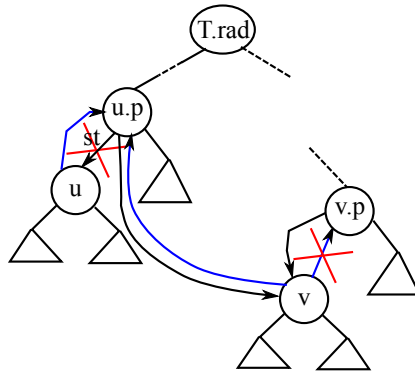


Figure 6.5: Managementul legăturilor între părintele nodului u și nodul v .

În figura 6.6 este prezentat un exemplu de ștergere dintr-un arbore binar de căutare..

Observație: în multe documentații este prezentată o metodă mai simplă de ștergere a unui nod z cu doi fii nenuli dintr-un arbore binar de căutare. În loc să se înlocuiască nodul z cu succesorul său y și apoi să se șteargă z , se înlocuiește informația lui z cu informația lui y și apoi se șterge y , care are un singur fiu. Problema cu acest mod de ștergere este că, de fapt nu se va șterge pointerul care a fost reimis către funcție, ci un alt pointer. Dacă alte componente al programului păstrează pointeri către y , acest lucru va avea consecințe nefaste, acești pointeri fiind invalidați. În schimb metoda de mai sus, prezentată și în Cormen, garantează faptul că pointerul șters este chiar cel trimis ca parametru către funcția de ștergere

Complexitatea: tuturor operațiilor descrise mai sus, începând de la căutarea binară, au complexitatea $O(h)$, unde h = înălțimea arborelui.

6.3 Exerciții rezolvate

1. Care este numărul minim de chei într-un arbore binar (de căutare sau nu) cu înălțimea h ?

Soluție: Numărul minim de chei se obține atunci când pe fiecare nivel avem număr minim de noduri, adică unul singur. În acest caz numărul de noduri este $h + 1$.

2. Care este numărul maxim de chei într-un arbore binar (de căutare sau nu) cu înălțimea h ?

Soluție: Numărul maxim de chei se obține atunci când pe fiecare nivel k avem număr maxim de noduri, adică unul 2^k noduri. În acest caz numărul de noduri este $2^{h+1} - 1$. Pentru detalii se poate vedea exercitiul 1 de la problemele rezolvate la capitolul de heap-uri, pentru că în acest caz este același calcul.

3. Care este înălțimea minimă / maximă a unui arbore cu n noduri?

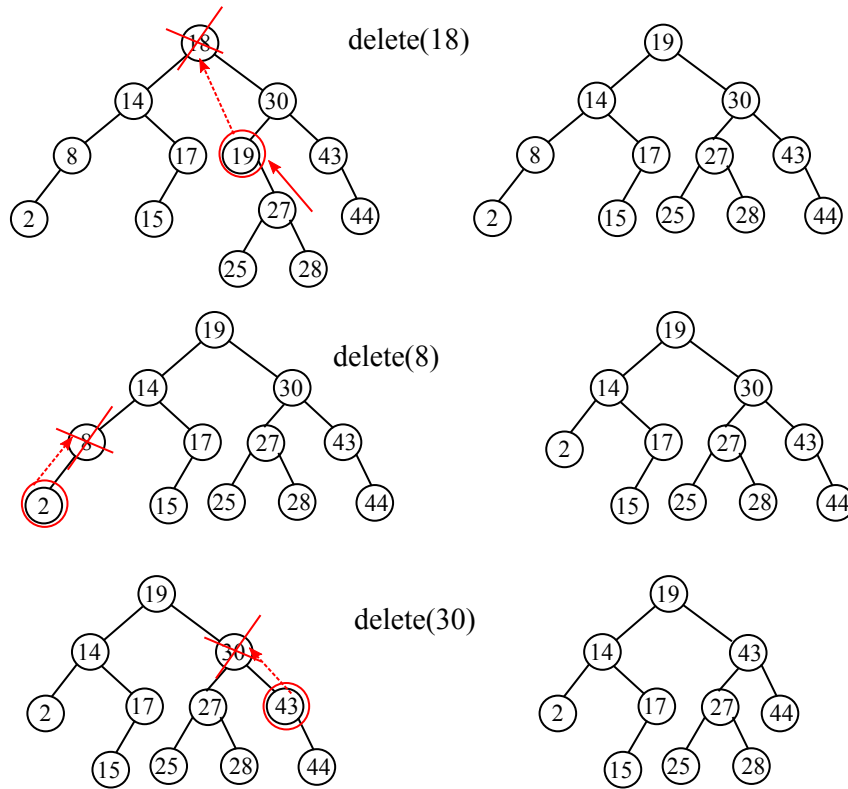


Figure 6.6: Se șterg pe rând cheile 18, 8, 30

Soluție: înălțimea maximă se obține pentru număr minim de noduri per nivel și este $n - 1$ și înălțimea minimă se obține pentru număr maxim de noduri per nivel și este $\log_2 n$.

4. Cunoscându-se parcurgerea în preordine a unui arbore binar de căutare, să se refacă arborele. Exemplu: RSD: 23, 17, 10, 15, 19, 35, 26, 24, 30, 37.

Soluție:

- var. 1 Observăm faptul că, prin parcurgerea în inordine a unui arbore binar de căutare se obține șirul cheilor sortat crescător. Deci, cunoscând care sunt cheile arborelui, din parcurgerea în preordine (RSD), putem imediat obține parcurgerea în inordine și apoi putem aplica algoritmul discutat la tema3. (vezi documentație de pe elearning).
- var. 2 Dacă luăm pur și simplu cheile în ordinea în care apar în parcurgerea RSD și le inserăm într-un arbore binar de căutare inițial vid, obținem arborele cerut.

Pentru cazul din exemplu, arborele este prezentat în fig. 6.7:

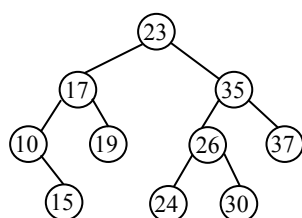


Figure 6.7: Arborele binar de căutare a cărui parcurgere în preordine este: 23, 17, 10, 15, 19, 35, 26, 24, 30, 37.

Chapter 7

Arbori AVL

7.1 Noțiuni generale

Arborii AVL sunt arbori binari de căutare aproape balansați. Denumirea provine de la autorii acestor arbori, doi matematicieni ruși G.M. Adelson-Velsky și E.M. Landis. Considerăm pentru fiecare nod x un *factor de balansare* dat prin

$$fb(x) = h(x.dr) - h(x.st)$$

adică, factorul de balansare al unui nod x este reprezentat de către diferența dintre înălțimea subarborelui său drept și înălțimea subarborelui său stâng. În unele documentații diferența se realizează între înălțimea subarborelui stâng și cea a subarborelui drept.

Înălțimea în arborii AVL

Definiția înălțimii unui nod este lungimea celui mai lung drum de la nod la o frunză, din acest lucru rezultă că înălțimea unei frunze este 0. Pentru calculul corect al factorilor de balansare într-u AVL există două variante:

- se consideră că frunzele, în loc să aibă pe stânga și pe dreapta Nil, sunt legate de fapt de niște noduri nule. Astfel înălțimea pentru fiecare nod crește cu 1 și deci înălțimea unei frunze va fi 1, iar a unui "nod nul" va fi 0. (fig. 7.1 a.)
- se consideră în continuare definiția clasică a înălțimii, iar pentru "Nil" se consideră "înălțimea" -1 (pentru a calcula corect factorul de balansare al unui nod, care are un singur copil - fig. 7.1 b.).

În ambele situații factorul de balansare obținut pentru un anumit nod este același. Precizările de mai sus sunt importante mai ales în contextul implementării unui arbore AVL. În continuarea acestui curs vom considera înălțimea unei frunze ca fiind 0, iar factorul de balansare pentru un nod, care are unul dintre copii nul se va calcula considerând copilul nul cu înălțimea -1, deci ca în figura 7.1 b.

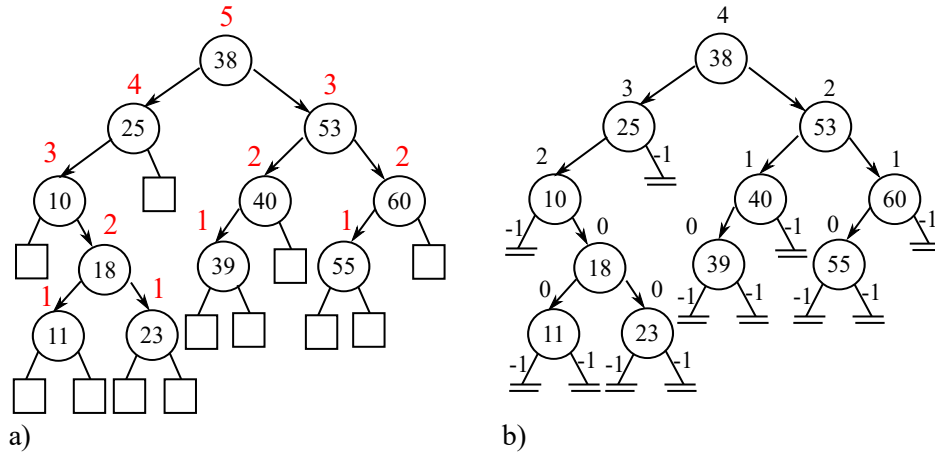


Figure 7.1: a) Înălțimile nodurilor, considerând frunzele legate de "noduri Nil". b) Înălțimile nodurilor conform definiției uzuale, fără "noduri Nil".

Definiție - nod balansat: Un nod x se numește balansat, dacă $fb(x) \in \{-1, 0, 1\}$.

În figura 7.2 este prezentat un exemplu de arbore binar de căutare împreună cu factorii de balansare corespunzători fiecărui nod. Se observă faptul că nodurile cu cheile 10 și 25 nu sunt balansate.

Definiție - arbore AVL: Un arbore binar de căutare se numește *arbore AVL*, dacă fiecare nod al său este balansat.

Un exemplu de arbore AVL este prezentat în figura 7.3.

Înălțimea maximă a unui arbore AVL

Pentru a determina înălțimea maximă a unui arbore AVL care conține n noduri se notează cu h înălțimea acestuia și cu $N(h)$ numărul minim de noduri ale unui arbore AVL de înălțime h . Evident $n \geq N(h)$.

Factorul de balansare al rădăcinii în cazul unui AVL cu număr minim de noduri, de înălțime h este sigur diferit de 0 (altfel s-ar mai putea șterge noduri din subarboarele stâng de ex. fără a modifica înălțimea, ceea ce ar contrazice numărul minim de noduri!). Atunci sigur că factorul de balansare al rădăcinii este -1 sau 1, rezultă că pentru un număr minim de noduri, unul dintre subarbori are înălțimea $h-1$ iar celălalt are înălțimea $h-2$.

$$N(h) = N(h-1) + N(h-2) + 1.$$

Cazurile de bază:

$$N(0) = 1$$

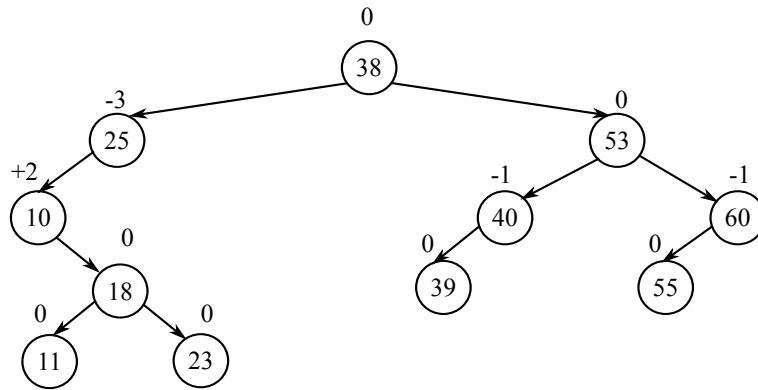


Figure 7.2: Arbore binar de căutare împreună cu factorii de balansare asociați nodurilor. Atenție: acesta NU este un arbore AVL.

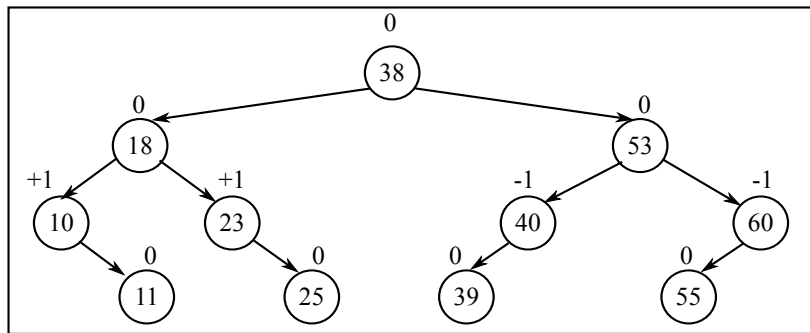


Figure 7.3: Arbore AVL.

$$N(1) = 2$$

Evident: $N(h-1) > N(h-2)$, de unde rezultă $N(h) > 2N(h-2) > 4N(h-4) > \dots > 2^k N(h-2k)$.

Ajungem la un caz elementar pentru $h-2k=0$ pentru h par - sau $h-2k=1$ pentru h impar. De aici rezultă:

$N(h) > 2^{h/2}$ pentru h par sau $N(h) > 2 * 2^{(h-1)/2} = 2^{(h+1)/2}$ pentru h impar.

Cum $N(h)$ = nr. minim de noduri pentru un AVL de înălțime $h \Rightarrow$ numărul n de noduri dintr-un AVL de înălțime h respectă: $n \geq N(h) > 2^{h/2}$

Deci $h < 2\log_2(n)$.

Deoarece complexitatea operațiilor de căutare, inserție și ștergere dintr-un arbore binar de căutare este $O(h)$ rezultă că într-un arbore AVL complexitatea acestor operații este $O(\log_2 n)$, dacă nu se ține cont de operațiile de reechilibrare ale arborelui.

Prin operații de inserție/ștergere se poate produce o debalansare a anumitor noduri. Pentru refacerea proprietății de arbore AVL, se utilizează operații de **rotație**.

7.2 Operații într-un arbore AVL

7.2.1 Rotația

Este o operație locală care schimbă structura de pointeri într-un arbore binar, dar păstrează proprietățile acestuia.

Tipuri de rotație:

- Rd = rotație spre dreapta în jurul nodului x
- Rs = rotație spre stânga în jurul nodului y

Operația de rotație este ilustrată în figura 7.4.

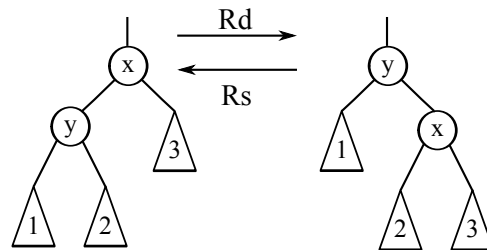


Figure 7.4: Rotații într-un arbore binar de căutare.

În figura 7.5 este ilustrat grafic modul în care se realizează rotația. Iar în figura 7.6 este prezentată o rotație într-un arbore AVL.

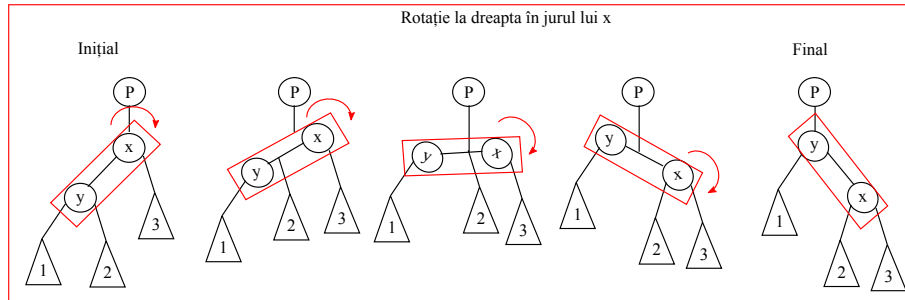
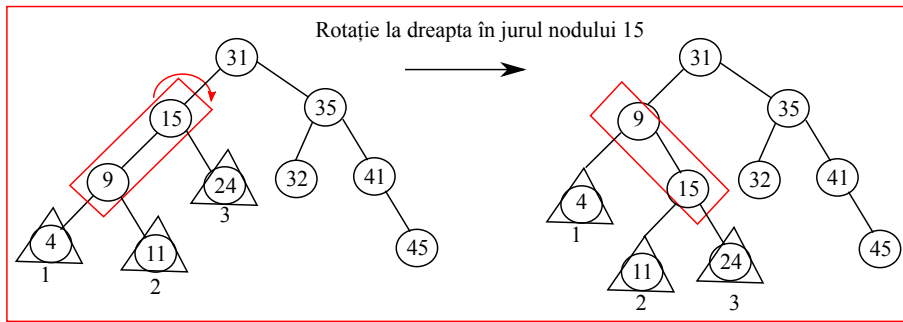


Figure 7.5: Rotație la dreapta în jurul nodului x într-un arbore binar de căutare.

Observații:

- pentru a putea efectua o rotație spre dreapta în jurul nodului x , este necesar ca nodul x să aibă un descendent stâng diferit de Nil.
- pentru a putea efectua o rotație spre stânga în jurul nodului y , este necesar ca nodul y să aibă un descendent drept diferit de Nil.

Figure 7.6: Rotație la dreapta în jurul nodului x într-un arbore AVL.

Algoritm pentru rotația spre stânga în jurul nodului y

Algoritm: ROT_ST(T, y)

```

 $x \leftarrow y.dr$ 
 $y.dr \leftarrow x.st$  //subarb 2 din imag se mută de la  $x$  la  $y$ 
daca  $x.st \neq nil$  atunci
    |  $x.st.p \leftarrow y$  //fac legatura subarb. 2 la noul părinte  $y$ 
sfarsit_daca
 $x.p \leftarrow y.p$ 
daca  $y.p = nil$  atunci
    |  $T.rad \leftarrow x$ 
sfarsit_daca
altfel
    | daca  $y = y.p.st$  atunci
    | | //fac legatura de la parintele lui  $y$  la  $x$ 
    | |  $y.p.st \leftarrow x$ 
    | sfarsit_daca
    | altfel
    | |  $y.p.dr \leftarrow x$ 
    | sfarsit_daca
sfarsit_daca
 $x.st \leftarrow y$  //fac legatura de la  $x$  la  $y$ 
 $y.p \leftarrow x$  //fac legatura de  $y$  la  $x$ 

```

Rotația înspre dreapta este simetrică.

Complexitate: $O(1)$

7.2.2 Inserția

Prin inserarea unui nod nou x se poate produce o debalansare în arbore. Acest lucru înseamnă că, cel puțin un nod din arbore va avea după recalcularea factorilor de balansare

un factor -2 sau 2.

Rebalansare: Nodul nou inserat x are factorul de balansare 0. Recalcularea factorilor

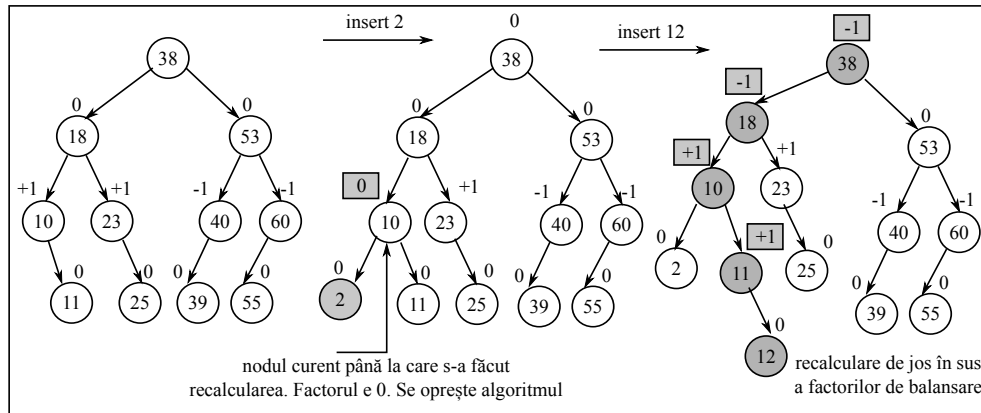


Figure 7.7: Exemplu de recalculare a factorilor de balansare de la nodul inserat în sus.

de balansare începe de la părintele nodului inserat înspre rădăcină.

Notăm cu x nodul curent. Se recalculează factorul de balansare al lui x .

- Dacă factorul de balansare nou 0, atunci nu a crescut înălțimea subarborelui de rădăcină x , față de cât era înainte de inserție, doar s-a echilibrat. Din acest motiv, nu mai are sens continuarea urcării în arbore. Deci algoritmul se oprește!
- Dacă factorul de balansare nou este -1 sau +1, atunci s-a produs o creștere a înălțimii subarborelui de rădăcină x pe una dintre ramuri (stângă respectiv dreaptă). Acest lucru produce o modificare a factorului de balansare al părintelui mai sus, deci se continuă cu procesul de la nodul curent $x = x.p$.
- Dacă factorul de balansare nou al lui x este -2 sau +2, atunci s-a produs o debalansare a subarborelui de rădăcină x și sunt necesare proceduri de rebalansare, care vor fi prezentate mai jos. În urma acestor proceduri, înălțimea subarborelui curent va reveni la cea de dinainte de inserție (se va vedea în continuare) și de aceea nu mai este necesară continuarea rebalansării mai sus, deci algoritmul se oprește.

Un exemplu de astfel de recalculare este prezentat în figura 7.7.

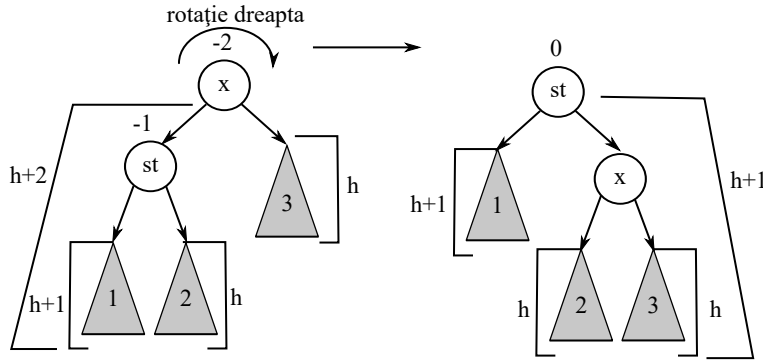


Figure 7.8: Rebalansarea arbore AVL în cazul 1 a).

Cazurile de rebalansare

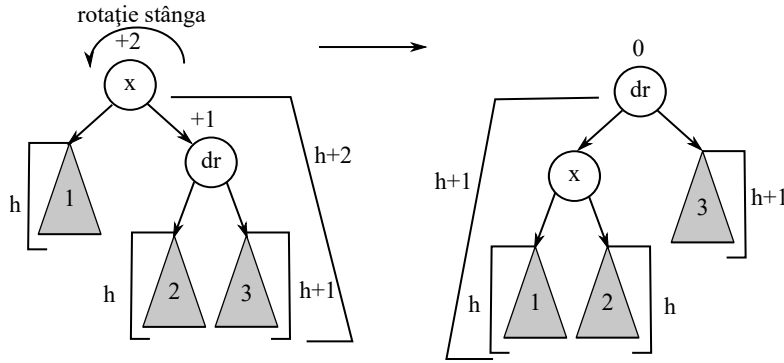


Figure 7.9: Rebalansarea arbore AVL în cazul 1 b).

Cazul 1

- a) $fb(x) = -2$, înseamnă că subarborul stâng este prea înalt. Notăm cu $st = x.st$. Dacă $fb(st) = -1$ atunci rebalansarea se realizează printr-o rotație spre dreapta în jurul lui x . Rezultatul acestei operații este ilustrat în figura 7.8. Se observă faptul că noii factori de balansare sunt 0 atât pentru x cât și pentru st . În plus, înainte de inserție, subarborul stâng al nodului x avea înălțimea $h + 1$, iar cel drept h . După rebalansare ambii subarbori au înălțimea $h + 1$. Rezultă că înălțimea subarborului care acum are rădăcina st nu a crescut, față de înălțimea înainte de inserție, când rădăcina era x . Astfel, nu este necesar să continuăm urcarea în arbore, deoarece mai sus nu vor exista modificări în factorul de balansare.
- b) $fb(x) = 2$, înseamnă că subarborul drept este prea înalt. Notăm $dr = x.dr$. Dacă $fb(dr) = 1$ atunci se efectuează o rotație la stânga în jurul lui x . Rezultatul acestei

operații este ilustrat în figura 7.9. Observații similare cazului 1.a pot fi făcute și în acest caz.

Cazul 2:

- a) $fb(x) = -2$, înseamnă că subarborele stâng este prea înalt. Notăm cu $st = x.st$. Presupunem că $fb(st) = 1$. Ce se întâmplă dacă se efectuează o rotație spre dreapta în jurul lui x ? Rezultatul unei astfel de rotații este ilustrat în figura 7.10.

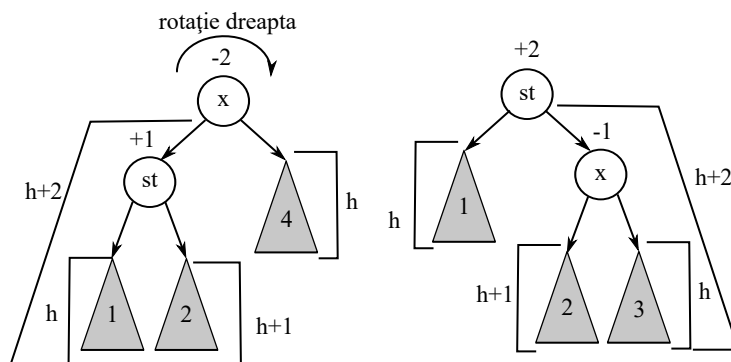


Figure 7.10: În cazul 2, o rotație simplă nu rezolvă problema debalansării arborelui

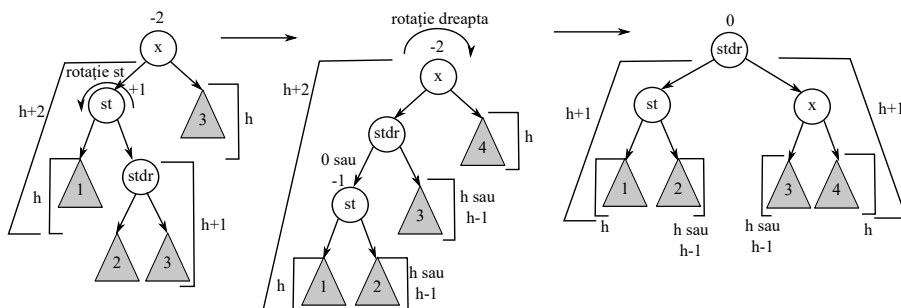


Figure 7.11: Rebalansarea arborelui în cazul 2.

Deci nu se rezolvă debalansarea, ci se mută pe cealaltă parte a arborelui. Soluția este următoarea:

- întâi rotație la stânga în jurul lui st
- apoi rotație la dreapta în jurul lui x .

Se obține rezultatul din fig. 7.11.

- b) $fb(x) = 2$, înseamnă că subarborele dreapta este prea înalt. Notăm cu $dr = x.dr$. Presupunem că $fb(dr) = -1$. Dacă efectuăm o rotație la stânga în jurul lui x se obține un efect similar ca la punctul a. Soluția este deci:

- întâi rotație la dreapta în jurul lui dr
- apoi rotație la stânga în jurul lui x

Un exemplu de inserție într-un arbore AVL este prezentat în figura 7.12.

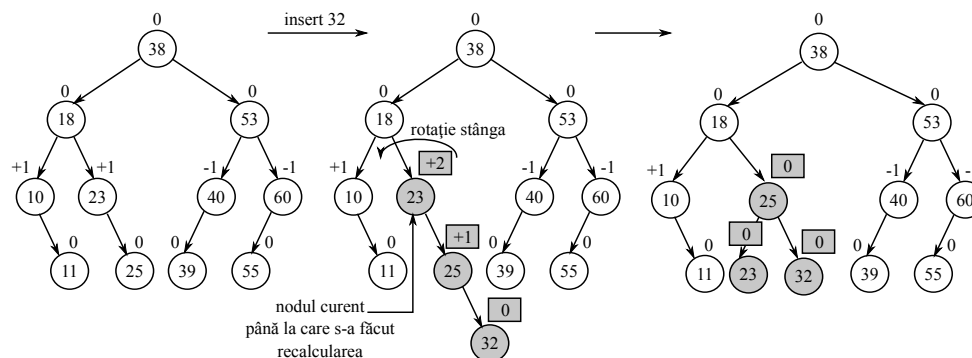


Figure 7.12: Inserția nodului cu cheia 32 în arborele AVL din figură.

Ștergerea unui nod

Ștergerea unui nod se realizează la fel ca pentru arborii binari de căutare obișnuiți. Dacă notăm cu z nodul care trebuie șters atunci:

- dacă z are cel mult un descendent, se pornește recalcularea factorilor de balansare de la părintele lui z
- dacă z are doi descendenți, atunci se determină y , succesorul nodului z , se înlocuiește nodul y cu descendentul său drept, iar nodul z cu nodul y . În acest caz recalcularea factorilor de balansare începe de la părintele succesorului nodului șters.

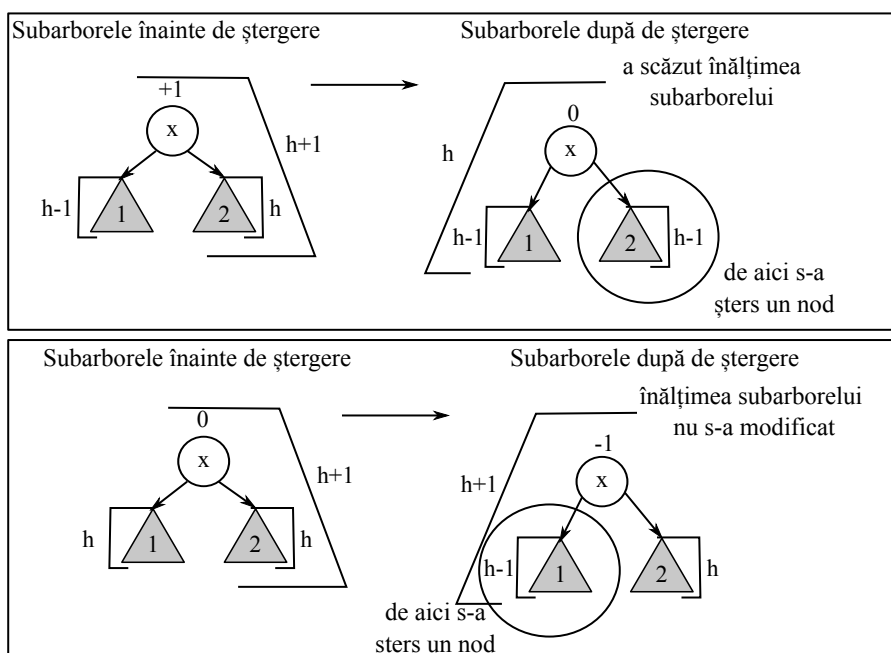


Figure 7.13: Modificarea înălțimii subarborelui curent după ștergerea unui nod.

Rebalansarea arborelui:

Notăm cu x nodul curent, pentru care se recalculează factorul de balansare. Spre deosebire de inserție, algoritmul nu se oprește atunci când factorul de balansare recalculat al nodului curent x este 0 , deoarece în acest caz a scăzut înălțimea subarborelui de rădăcină x . Acest lucru, ilustrat în figura 7.13, produce modificări ale factorului de balansare și la părintele său și deci continuă urcarea în arbore.

În schimb, atunci când factorul de balansare al nodului x devine -1 sau $+1$, înseamnă că doar s-a produs o scădere a înălțimii pe una dintre ramurile lui x , dar nu și a subarborelui de rădăcină x (fig. 7.13), deci algoritmul de rebalansare se poate opri.

De asemenea, algoritmul nu se oprește însă după prima rebalansare a unui nod cu factorul -2 sau $+2$, ci în cele mai multe cazuri trebuie să continue de la nodul curent la părinte până la rădăcină.

Considerând nodul curent x , rebalansarea are următoarele cazuri:

Cazul 1

- Dacă x are factorul de balansare nou -2 și factorul lui $st = x.st$ este -1 sau 0 , atunci rotație la dreapta în jurul lui x . În figura 7.14 este ilustrat modul de recalculare a factorilor de balansare pentru nodurile x și st . Se observă și următorul fapt important. Subarboarele de rădăcină x a avut înainte de ștergere înălțimea $h + 2$. În cazul în 1.a.1 din fig. 7.14, în care y a avut factorul de balansare 0 înainte de

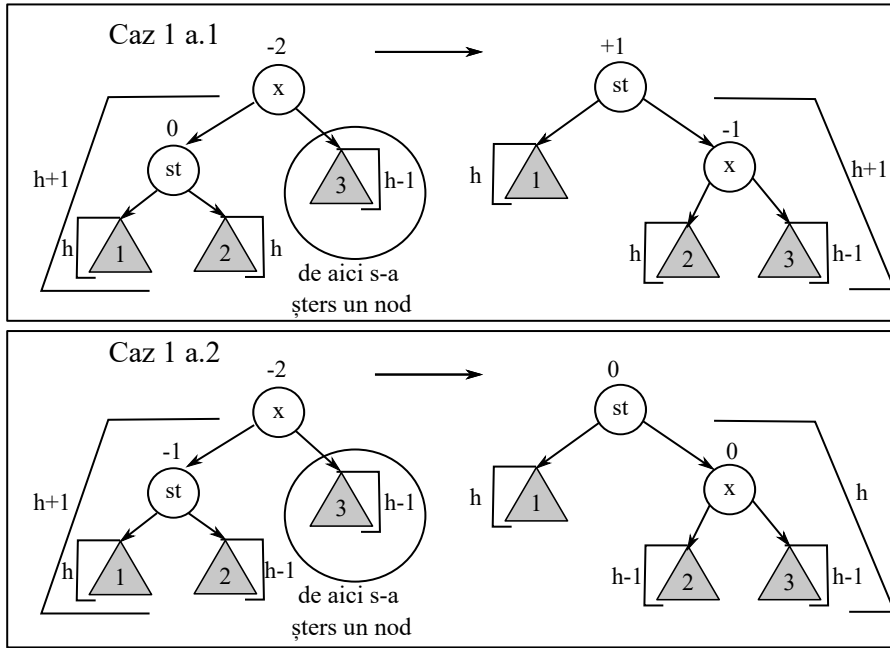


Figure 7.14: Rebalansarea unui arborele AVL după ștergerea unui nod, cazul 1 a.

rebalansare, după rebalansare se observă că nu s-a modificat înălțimea subarborelui, care acum are rădăcina st , și deci nu mai este necesară continuarea urcării în arbore.

În schimb, dacă st a avut factorul de balansare -1 , cazul 1.a.2, se observă din figura 7.14, că după rebalansare, înălțimea subarborelui care acum are rădăcina st , a scăzut cu 1, ceea ce produce eventuale debalansări mai sus în arbore, deci trebuie continuat la părintele lui x .

- b. Dacă x are factorul de balansare nou 2 și factorul de balansare al lui $dr = x.dr$ este 0 sau 1 atunci rotație la stânga în jurul lui x . În figura 7.15 este ilustrat modul de recalculare a factorilor de balansare pentru nodurile x și dr . La fel ca în cazul 1. a, se observă și următoarele: Subarborele de rădăcină x a avut înainte de ștergere înălțimea $h + 2$. În cazul în 1.b.1, în care dr a avut factorul de balansare 0 înainte de rebalansare, după rebalansare se observă că nu s-a modificat înălțimea subarborelui, care acum are rădăcina dr , și deci nu mai este necesară continuarea urcării în arbore.

În schimb, dacă dr a avut factorul de balansare $+1$, cazul 1.b.2, se observă din fig. 7.15, că după rebalansare, înălțimea subarborelui care acum are rădăcina dr , a scăzut cu 1, ceea ce produce eventuale debalansări mai sus în arbore, deci trebuie continuat la părintele lui x .

Cazul 2

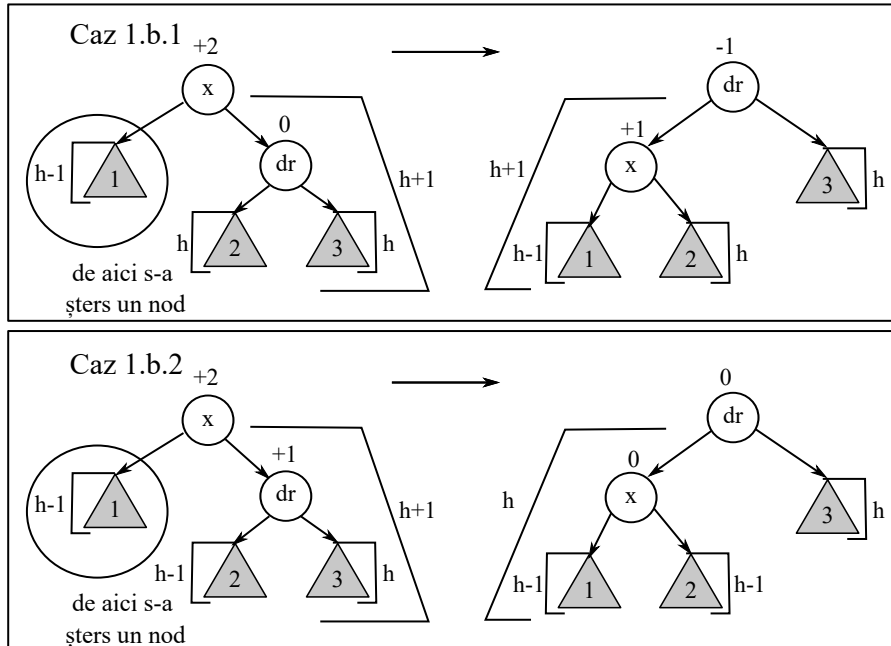


Figure 7.15: Rebalansarea unui arborele AVL după ștergerea unui nod, cazul 1 b.

- a. Dacă x are factorul de balansare nou -2 și nodul $y = x.st$ are factorul de balansare 1 , atunci pentru rebalansare se efectuează:
 - Întâi rotație la stânga în jurul lui y
 - Apoi rotație la dreapta în jurul lui x .
- b. Dacă x are factorul de balansare nou 2 și $y = x.dr$ are factorul de balansare -1 , atunci pentru rebalansare se efectuează:
 - Întâi rotație la dreapta în jurul lui y
 - Apoi rotație la stânga în jurul lui x .

În acest caz, după rebalansare scade înălțimea subarborelui curent, comparativ cu înălțimea avută înainte de ștergere. Este deci necesară continuarea algoritmului de la părintele nodului curent x .

Chapter 8

Arbori Roșu Negru

8.1 Noțiuni generale

Definiție: Un arbore roșu-negru (ARN) este un arbore binar de căutare în care fiecărui nod i se asociază o culoare - roșu sau negru - și care are următoarele proprietăți:

1. Fiecare nod este roșu sau negru - are deci un câmp suplimentar *color*
2. Rădăcina este neagră
3. Fiecare frunză este neagră și NIL
4. Dacă un nod este roșu, ambii fii sunt negri $\Rightarrow$ părintele unui nod roșu este negru.
5. Pentru fiecare nod x , oricare drum de la nod la o frunză NIL se întâlnește același număr de noduri negre (inclusiv frunza NIL și exclusiv nodul de la care se pornește). Acest număr se numește înălțimea neagră a subarborelui de rădăcină x . Notăm înălțimea neagră cu *bh* - *black height*.

Un exemplu de arbore roșu-negru este prezentat în figura 8.1.

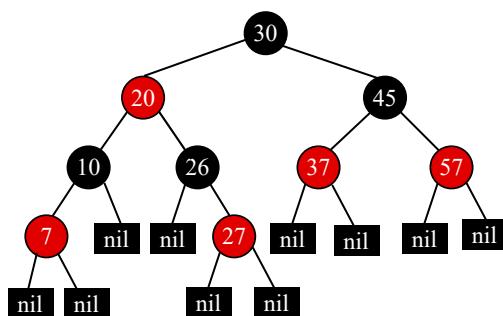


Figure 8.1: Exemplu de arbore roșu - negru.

Observații:

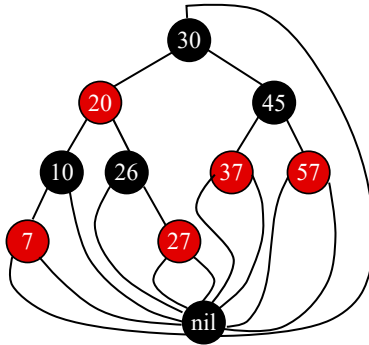


Figure 8.2: Exemplu de arbore roșu - negru cu santinelă.

- Într-un ARN niciun drum de la rădăcină la o frunză nu poate fi mai lung decât dublul unui alt drum la altă frunză. Acest lucru asigură o balansare relativă a arborelui .
- Înălțimea maximă a unui arbore ARN este $2\log_2(n + 1)$.
Se demonstrează prin inducție că orice subarbore de rădăcină x conține cel puțin $2^{bh(x)} - 1$ noduri interne. Notând cu h înălțimea arborelui și cu r rădăcina, din proprietatea 4 se obține $bh(r) \geq h/2$. Dar $n \geq 2^{bh(r)} - 1 \geq 2^{h/2} - 1$ de unde rezultă $h \leq 2\log_2(n + 1)$
- Definirea unui nod NIL pentru fiecare frunză presupune un consum inutil de memorie. Din acest motiv se poate considera în locul acestor frunze un singur nod santinelă T.nil către care să indice acele noduri interne care au ca descendenți frunze NIL. Un exemplu de ARN cu santinelă este prezentat în fig. 8.2.
- Pentru simplitate în continuare vom ignora în desene nodurile NIL.
- Datorită faptului că operațiile de căutare, maxim, minim, succesor, predecesor depind de înălțimea h a arborelui, înseamnă că aceste operații au complexitatea $O(\log_2 n)$.
- Operațiile de inserție și ștergere sunt ceva mai complicate decât în cazul arborilor binari de căutare simpli, deoarece după inserție/ștergere trebuie eventual refăcută structura de arbore roșu-negru.

Prin operații de inserție/ștergere se poate strica anumite proprietăți RN. Pentru refacerea acestor proprietăți de arbore ARN, sunt necesare operații de recolorare și de rotație.

8.1.1 Inserția într-un arbore roșu-negru

Inserția propriu-zisă a unui nod într-un arbore roșu-negru se realizează după același algoritm ca și inserția într-un arbore binar de căutare. Practic se pornește de la rădăcină

și se compară la fiecare pas cheia nodului curent x cu cheia nodului z care se inserează, coborându-se în subarborele stâng dacă $z.info < x.info$ și în cel drept altfel. Apoi se refac proprietățile ARN.

Observații:

- Întotdeauna nodul care se inserează are culoarea roșie
- În arborele T se consideră o santinelă $T.nil$

Proprietățile care pot fi neîndeplinite în cazul inserției:

- Proprietățile 1, 3 și 5 se păstrează, datorită faptului că se inserează un nod roșu, care are ca fiu două frunze NIL, deci se leagă de nodul santinelă $T.nil$
- Proprietatea 2 poate fi contrazisă dacă nodul inserat este chiar rădăcina sau ca urmare a cazului 1, care va fi discutat în cele ce urmează. Pentru rezolvarea acestei situații este suficientă colorarea rădăcinii cu negru.
- Proprietatea 4 poate fi contrazisă, dacă părintele de care s-a legat noul nod are culoarea roșie. În acest caz trebuie refăcută proprietatea. Există în această situație 3 cazuri care vor fi discutate în continuare.

Funcția de refacere a proprietăților RN este apelată doar dacă părintele nodului inserat este roșu.

Observații:

- Notăm cu z nodul inserat, cu P părintele lui z . Datorită faptului că P are culoarea roșie, deja înainte de inserție, iar inserția se produce într-un arbore roșu-negru valid, înseamnă că P nu este rădăcină și deci există un nod părinte al lui $P \neq T.nil$ pe care îl notăm cu B . Notăm cu U unchiu (fratele părintelui).
- P se poate afla la stânga sau la dreapta lui B . Cele două cazuri se tratează în mod similar, prin simetrie. Vom considera în continuare inserția pe stânga bunicului B .

Cazurile de refacere a proprietății 4 în urma inserției:

Cazu 1: unchiul lui z este roșu (fig. 8.3).

Deoarece U , P sunt roșii rezultă că B are culoarea neagră, altfel s-ar contrazice proprietatea 4, $\Rightarrow$ este suficientă recolorarea P , U , B , adică P și U devin negri iar B roșu.

În urma acestei modificări poate avea loc o contrazicere a proprietății 4 pentru B și părintele său, deci se reia procedura de refacere a proprietăților roșu-negru, considerând de data aceasta $z = B$. Același procedeu se aplică și în cazul în care z se află pe dreapta lui P .

Atunci când unchiul U este negru se disting cazurile 2 și 3, care diferă prin poziționarea nodului z relativ la P și la B .

Cazul 2: unchiul este negru și

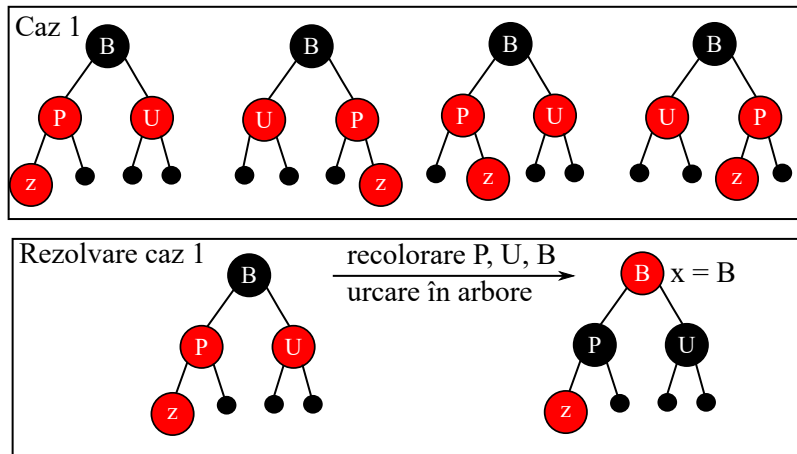


Figure 8.3: Cazul 1 pentru inserție.

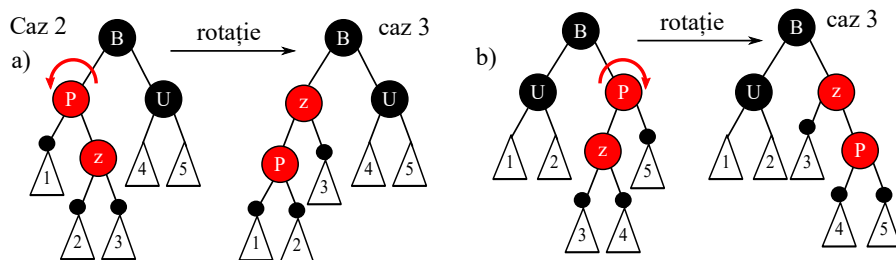


Figure 8.4: Cazul 2 pentru inserție.

a) z se află pe dreapta lui P , iar P se află la stânga lui B - fig.8.4

b) z se află pe stânga lui P , iar P se află la dreapta lui B

Soluționare: în cazul a) - rotație la stânga după P , în cazul b) - rotație la dreapta după P . Nu se soluționează complet, ci se trece practic în cazul 3, în care refacerea începe de la nodul care prin rotație a coborât, deci de la $z = P$.

Cazul 3: unchiul este negru și

a) z se află pe stânga lui P , iar P se află la stânga lui B - fig.8.5

b) z se află pe dreapta lui P , iar P se află la dreapta lui B

Soluționare::

1. Colorare B cu roșu, colorare P cu negru.
2. pentru situația a) Rotație la dreapta în jurul lui B , pentru situația b) Rotație la stânga în jurul lui B .

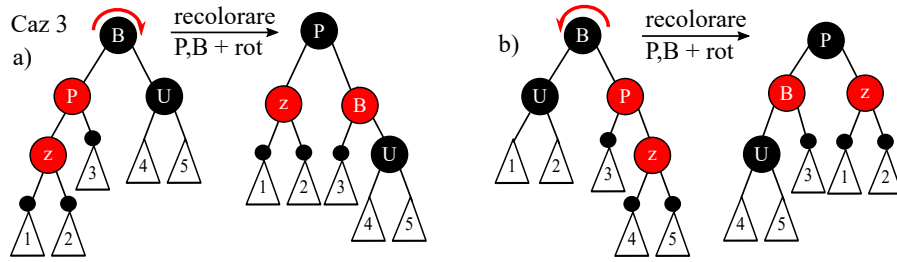


Figure 8.5: Cazul 3 pentru inserție.

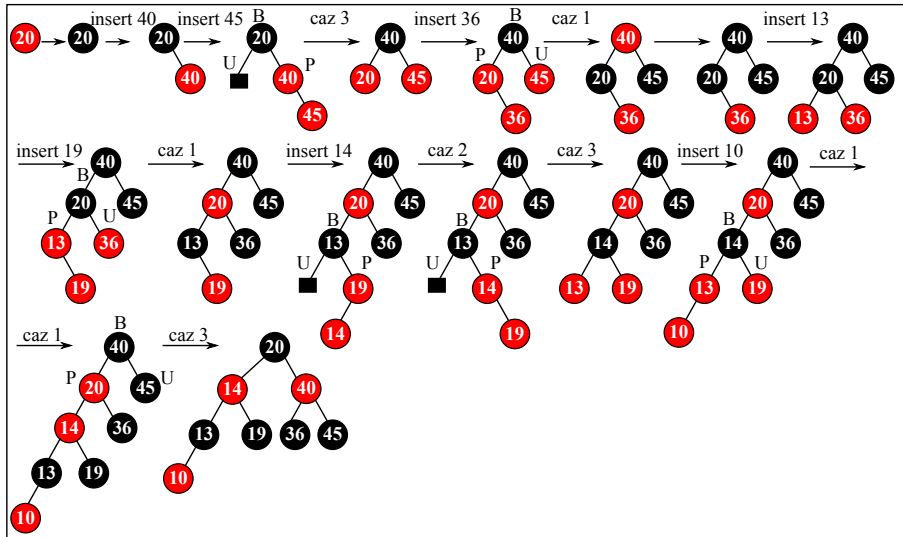


Figure 8.6: Inserarea cheilor 20, 40, 45, 36, 13, 19, 14, 10 într-un ARN inițial vid.

Un exemplu de construcție a unui ARN prin inserție succesivă de chei este prezentat în figura 8.6.

Algorithm: Insert(T, z)

Intrare: Arborele T , nodul z care se inserează $y \leftarrow T.nil$ $x \leftarrow T.rad$ **cat timp** $x \neq T.nil$ **executa** $\overline{y} \leftarrow x$ **daca** $z.info < x.info$ **atunci** $x \leftarrow x.st$ **sfarsit_daca** **altfel** $x \leftarrow x.dr$ **sfarsit_daca****sfarsit_cat_timp** $z.p \leftarrow y$ **daca** $y = T.nil$ **atunci** $T.rad \leftarrow z$ **sfarsit_daca****altfel** **daca** $z.info < y.info$ **atunci** $y.st \leftarrow z$ **sfarsit_daca** **altfel** $y.dr \leftarrow z$ **sfarsit_daca****sfarsit_daca** $z.st \leftarrow T.nil$ $z.dr \leftarrow T.nil$ $z.color \leftarrow rosu$ ARN-Insert-Repara(T, z)

Algoritm: ARN-Insert-Repara(T, z)

```

cat _timp  $z.p.color = rosu$  executa
  daca  $z.p = z.p.p.st$  atunci
     $U \leftarrow z.p.p.dr$ 
    _____ caz 1
    daca  $U.color = rosu$  atunci
       $z.p.color \leftarrow negru$ 
       $U.color \leftarrow negru$ 
       $z.p.p.color \leftarrow rosu$ 
       $z \leftarrow z.p.p$ 
    sfarsit_daca
  altfel
    _____ caz 2
    daca  $z = z.p.dr$  atunci
       $z \leftarrow z.p$ 
       $ROT\_ST(T, z)$ 
    sfarsit_daca
    _____ caz 3
     $z.p.color \leftarrow negru$ 
     $z.p.p.color \leftarrow rosu$ 
     $ROT\_DR(T, z.p.p)$ 
  sfarsit_daca
sfarsit_daca
altfel
  //similar dar simetric pentru nodul  $z$ 
  //aflat la stânga bunicului
sfarsit_daca
sfarsit_cat_timp
 $T.rad.color \leftarrow negru$ 
RETURN

```

Complexitate: Inserția are aceeași complexitate ca și în cazul arborilor binari de căutare simpli, deci $O(h)$. Cum înălțimea este cel mult $2\log_2(n+1)$ rezultă o complexitate $O(\log_2 n)$, iar algoritmul de refacere al proprietăților de arbore roșu-negru pornește de jos înspre rădăcină pe o ramură, deci complexitatea este tot $O(\log_2 n)$. Rezultă complexitatea pentru inserție este $O(\log_2 n)$.

8.1.2 Ștergerea dintr-un arbore roșu-negru

Operația de ștergere dintr-un arbore roșu-negru este mai complicată decât operația de inserție. În prima etapă, operația de ștergere are la bază ștergerea dintr-un arbore binar de căutare, cu câteva modificări, după care trebuie efectuată o operație de refacere a proprietăților de arbore roșu-negru.

Reamintim faptul că, în cazul ștergerii unui nod z dintr-un arbore binar de căutare

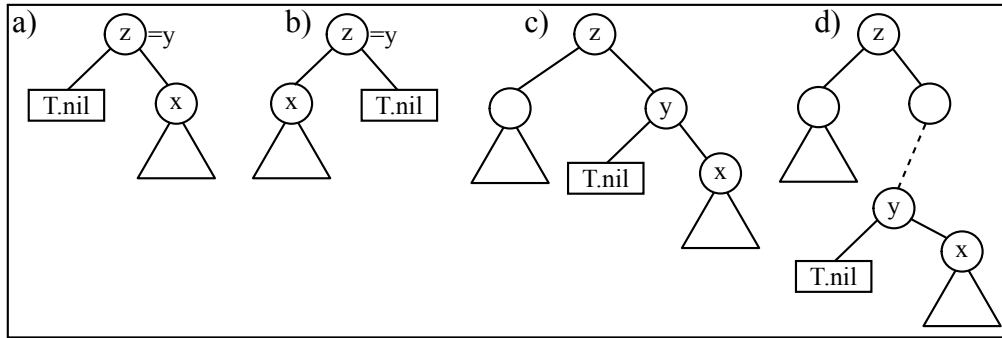


Figure 8.7: Cazurile pentru ștergerea nodului z . a) z nu are descendent stâng, b) z nu are descendent drept, c) z are 2 descendenți nenuli și succesorul lui z este fiu al lui z , d) z are doi descendenți și succesorul lui z nu este fiu al lui z .

T se iau în considerare cazurile:

1. $z.st = T.Nil$ - fig.8.7 a)
2. $z.dr = T.Nil$ - fig.8.7 b)
3. z are doi fii diferiți de $T.Nil$ și $y = \text{succesor}(T, z)$
 - a. y descendent direct al lui z - fig.8.7 c)
 - b. y nu este descendent direct al lui z - fig.8.7 d)

Observații:

- Algoritmul de refacere depinde în cazurile 1 și 2 de culoarea nodului șters z și de culoarea pe care o are x , iar în cazul 3 de culoarea originală a lui y (după ștergere y preia culoarea lui z) și de culoarea lui x , unde x este în primele 2 cazuri nodul cu care se înlocuiește z , iar în al treilea caz este fiul drept al lui y .
- În cazurile 1 și 2, prin ștergerea nodului z și înlocuirea cu unul dintre descendenții direcți, se poate produce o modificare a culorii în poziția deținută anterior de z , ceea ce poate duce la violarea proprietăților de arbore roșu-negru. La fel în cazul 3, la înlocuirea lui y cu x .
- Nodul x este acela care se deplasează în poziția deținută anterior de nodul y . Se observă faptul că, x poate fi santinela $T.Nil$

Refacerea proprietăților de arbore roșu-negru:

- Dacă nodul z în cazurile 1 și 2, respectiv y în cazul 3, a avut culoarea inițială roșu, atunci se păstrează proprietățile de arbore RN, deoarece
 1. Nu s-a modificat înălțimea neagră pe nici o ramură prin eliminarea unui nod roșu

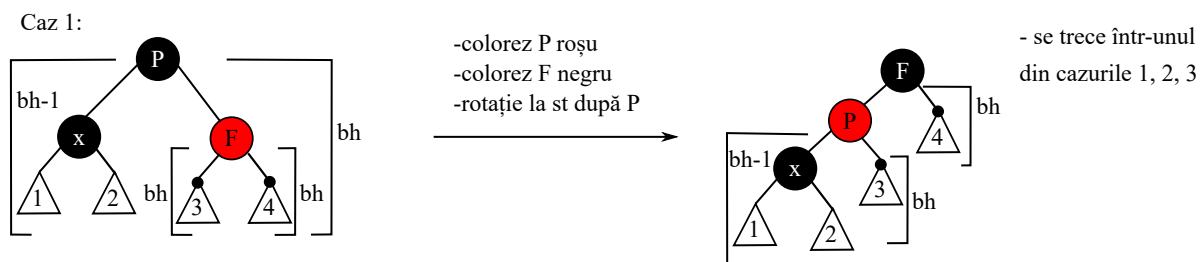


Figure 8.8: Cazul 1 în ARN

2. Nici un nod roșu nu a căpătat un fiu roșu.
 3. Deoarece nici nodul z , nici nodul y în cazul 3, nu au fost rădăcină (culoarea originală a fost roșie) nu s-a modificat nici culoarea rădăcinii, care a rămas neagră
- Rezultă deci că, proprietățile de arbore RN sunt violate doar dacă a fost eliminat un nod negru. Există mai multe cazuri care trebuie tratate.
 - Prin eliminarea unui nod negru din arbore
 1. În primul rând se micșorează înălțimea neagră pe ramura respectivă
 2. Poate apărea vecinătate între două noduri roșii

Cazul 0: culoarea nodului x cu care s-a făcut înlocuirea este roșie. În acest caz singurul lucru care trebuie făcut este recolorarea lui x în negru. Acest lucru rezolvă atât problema (1) cât și problema (2).

Notății: P =părintele lui x , F =fratele lui x , bh = înălțimea neagră originală a subarborelui cu rădăcina P (exclusiv nodul P).

În continuare vom considera că nodul x se află la stânga nodului P . Pentru x la dreapta lui P modul de rezolvare este simetric.

Cazul 1: F are culoarea roșie - fig. 8.8

Facem următoarele observații:

- Datorită faptului că înainte de ștergere arborele era RN valid, din F roșu și F descendent direct al lui P rezultă culoarea lui P este neagră.
- Copiii lui F sunt ambii negri (eventual T.nil)
- Prin ștergere s-a micșorat înălțimea neagră pe partea stângă a lui P , adică subarborele drept are înălțimea neagră bh , iar subarborele stâng $bh - 1$. Cum F este roșu rezultă că ambii săi subarbori au înălțimea bh și fiii lui F sunt ambii negri.

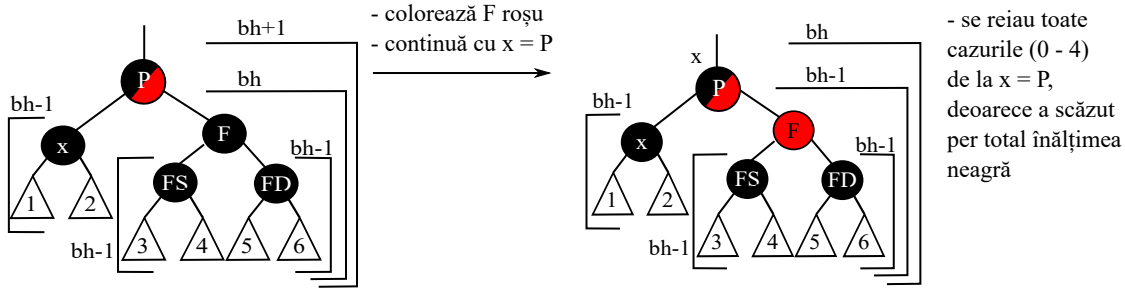


Figure 8.9: Cazul 2 în ARN

Refacerea proprietăților de arbore RN:

- Colorare P cu roșu și colorare F cu negru.
- Rotație la stânga în jurul lui P . (Dacă x se află pe dreapta lui P , atunci rotația este la dreapta).

În această situație F , acum negru, urcă în locul lui P . La dreapta lui F înălțimea neagră a rămas bh . Înălțimea neagră a subarborelui drept al lui P este bh , iar înălțimea neagră subarborelui stâng al lui P este $bh - 1$. Deci problema în nodul P a rămas, dar fiul drept al lui P nu mai este roșu ci negru, ceea ce conduce la unul dintre cazurile 2, 3 sau 4.

Deci din cazul 1 se trece la cazul 2, 3 sau 4!

În figura 8.8 este ilustrată procedura de refacere a proprietăților RN în cazul 1.

Cazul 2: F este negru și ambii fii ai săi sunt negri (eventual T.Nil) - fig. 8.9.

Observații:

- $F \neq T.Nil$, pentru că pe partea stângă a lui P s-a șters un nod negru, iar înainte de ștergere pe ambele părți ale lui P înălțimea neagră era aceeași și cel puțin 1.
- P poate fi roșu sau negru
- Prin ștergere s-a micșorat înălțimea neagră pe partea stângă a lui P , adică subarborele drept are înălțimea neagră bh , iar subarborele stâng $bh - 1$. Cum F este negru rezultă că ambii săi subarbori au înălțimea $bh - 1$.

Refacerea proprietăților de arbore RN:

- Se colorează F roșu $\Rightarrow$ înălțimea neagră a subarborelui drept al lui P este $bh - 1$, deci egală cu cea a subarborelui stâng, dar înălțimea neagră a arborelui P este mai mică decât înainte de ștergere, deci cu o unitate mai mică decât a fratelui său (dacă există). Această problemă se rezolvă astfel:

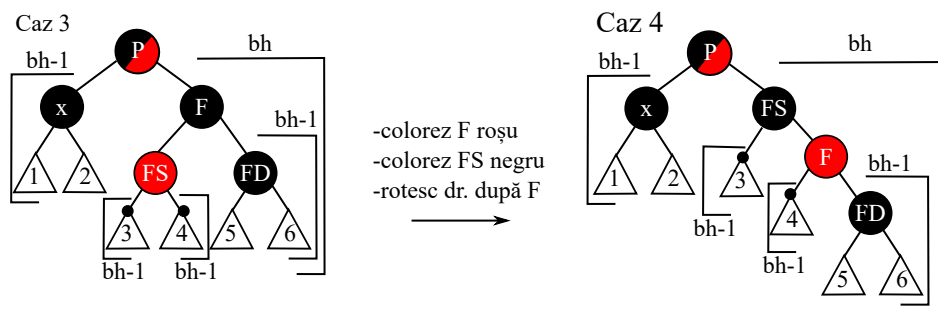


Figure 8.10: Cazul 3 în ARN. P poate avea culoarea roșie sau culoarea neagră.

- Dacă P este roșu atunci este suficient să colorăm P cu negru - fig. 8.9 a).
- Altfel problema de refacere a proprietăților RN se reia pentru subarborele care are ca rădăcină părintele lui P . Se observă din figura 8.9 b) că, în acest caz înălțimea neagră a întregului arbore care are rădăcina P a scăzut cu 1 și trebuie reechilibrat de la acest nod în sus.

Cazul 3: a) F are culoarea neagră, fiul stâng al lui F notat cu FS este roșu, iar cel drept notat cu FD este negru. (fig. 8.10). Respectiv cazul 3b) simetric, atunci când F este pe stânga lui P și x pe dreapta, F este negru, FD este roșu și FS este negru. Cazul 3b) se soluționează simetric cu 3a) prezentat mai jos.

Observații:

- Culoarea lui P poate fi roșie sau neagră.
- F sigur este diferit de $T.Nil$, pentru că pe partea stângă a lui P s-a șters un nod negru, iar înainte de ștergere pe ambele părți ale lui P înălțimea neagră era aceeași și cel puțin 1.
- Înălțimea neagră a subarborelui stâng al lui P este $bh - 1$, înălțimea neagră a subarborelui cu rădăcina F este bh . Înălțimile negre ale subarborilor FS și FD sunt $bh - 1$.

Refacerea proprietăților de arbore RN: - fig. 8.10

- Recolorare F și $FS \Rightarrow F$ devine roșu și FS devine negru $\Rightarrow$ înălțimea neagră a lui FS devine bh și înălțimea neagră a lui FD devine $bh - 1$.
- rotație la dreapta în jurul lui $F \Rightarrow FS$ urcă în locul lui F , înălțimea neagră a lui FS devine bh , înălțimea neagră a lui F devine $bh - 1 =$ înălțimea neagră a fiului stâng al lui FS .

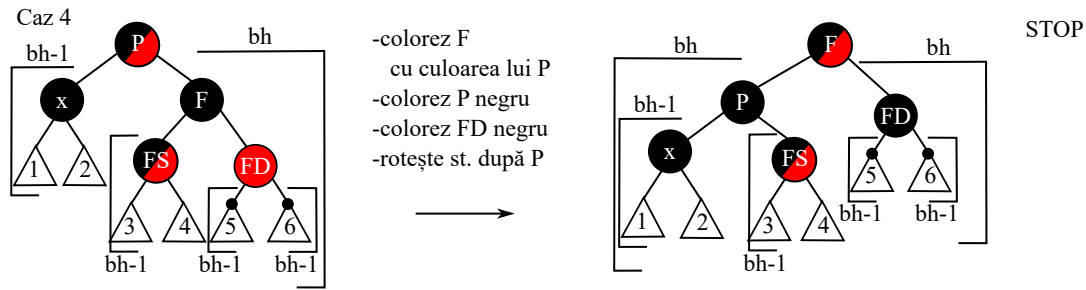


Figure 8.11: Cazul 4 în ARN

Problema la care am ajuns în continuare este refacerea proprietăților RN pentru cazul 4.

Cazul 4: F are culoarea neagră iar fiul drept notat cu FD este roșu. Fiul stâng, FS , poate avea oricare dintre cele două culori. (fig. 8.11). În cazul simetric, atunci când x este pe dreapta lui P , FS este roșu.

Observații:

- Culoarea lui P poate fi roșu sau negru.
- F sigur este diferit de $T.Nil$, pentru că pe partea stângă a lui P s-a șters un nod negru, iar înainte de ștergere pe ambele părți ale lui P înălțimea neagră era aceeași și cel puțin 1.
- Înălțimea neagră a subarborelui stâng al lui P este $bh - 1$, înălțimea neagră a subarborelui cu rădăcina F este bh . Înălțimile negre ale lui FS și FD sunt $bh - 1$.

Refacerea proprietăților de arbore RN: - fig. 8.11.

- Colorare F cu culoarea lui P
- Colorare P cu negru
- Colorare FD cu negru
- Rotație la stânga în jurul lui P (respectiv dreapta în cazul simetric).

Prin aceste operații rezultă: creșterea cu o unitate a înălțimii negre a subarborelui F , adică $bh(F) = bh + 1$. Dar $bh(x) = bh - 1$. Prin rotația în jurul lui P , P fiind acum negru, se rebalansează arborele.

Pseudocodul pentru ștergere poate fi găsit în bibliografia recomandată.

Chapter 9

Îmbogățirea structurilor de date

9.1 Noțiuni generale

Îmbogățirea structurilor de date se realizează în general cu scopul de-a putea efectua și alte operații -cereri - în afara celor uzuale, descrise în capitolele anterioare, sau pentru a crește eficiența celor uzuale. Îmbogățirea unei structuri de date se referă la atașarea de informații suplimentare elementelor structurii - adică adăugarea de câmpuri suplimentare fiecărui nod, astfel încât să permită efectuarea operațiilor dorite. Aceste câmpuri suplimentare trebuie actualizate odată cu modificarea mulțimii dinamice stocate cu ajutorul structurii de date.

Informația suplimentară adăugată nodurilor structurii de date poate fi o valoare, dar poate fi și o referință (pointer)!

Procesul de îmbogățire a unei structuri de date are 4 etape

1. Alegerea unei structuri de date de bază, pe care urmează să o îmbogățim: de exemplu un arbore roșu-negru.
2. Determinarea informației suplimentare care trebuie păstrată în structura aleasă.
3. Verificarea faptului că, informația suplimentară aleasă poate fi actualizată pe parcursul operațiilor de bază de modificare a structurii, respectiv inserție/ștergere, fără a crește complexitatea acestora. Va fi enunțată mai jos o teoremă, care permite în anumite condiții bine definite, demonstrarea păstrării complexității logaritmice în cazul îmbogățirii unui arbore binar care se autoechilibrează.
4. Dezvoltarea de noi operații utile asupra structurii de date îmbogățite.

Un exemplu clasic pentru îmbogățirea arborilor roșu-negru sunt arborii pentru statistici de ordine, prezentați în continuare.

9.2 Arbori pentru statistici de ordine

În cadrul cursului de algoritmică, odată cu descrierea algoritmului *Quicksort*, s-a discutat și despre statistici de ordine. De fapt *statistica de ordin i* a unei mulțimi de n elemente $a_1, a_2, \dots, a_n$, este acel element, care s-ar afla în șirul sortat al elementelor pe poziția a i -a. S-a discutat cum poate fi modificat algoritmul *Quicksort*, astfel încât această problemă să fie rezolvată în timp liniar.

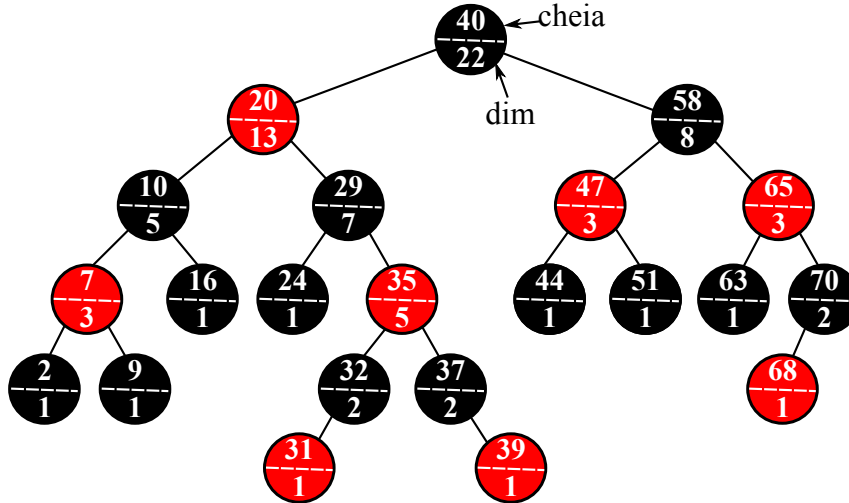


Figure 9.1: Exemplu de ARN pentru statistici de ordine. Câmpul dimensiune = dim al unui nod x = numărul de noduri din subarborele de rădăcină x

Prin îmbogățirea structurilor de date se poate obține însă o complexitate $O(\log_2 n)$ pentru determinarea statisticii de ordin i . În plus, pentru fiecare element din șir, poate fi calculat tot cu o complexitate $O(\log_2 n)$ rangul său, adică poziția pe care s-ar afla în șirul sortat.

Definiție: Un *arbore pentru statistici de ordine* este un ARN, în care fiecare nod x conține un câmp suplimentar, $x.dim$, care reprezintă numărul de noduri - fără frunzele nil - ale subarborelui cu rădăcina x . În cazul nodului santinelă $T.nil$, se consideră $T.nil.dim = 0$. Se observă ușor relația:

$$x.dim = x.st.dim + x.dr.dim + 1$$

În figura 9.1 este reprezentat un arbore roșu-negru pentru statistici de ordine.

Pentru un astfel de arbore îmbogățit cu informația dim pot fi definite două operații noi:

- (I) Căutarea elementului de rang i = elementul a cărui cheie s-ar afla pe poziția i în șirul sortat al cheilor din arbore.

- (II) Determinarea rangului unui element = poziția pe care s-ar afla cheia elementului respectiv în șirul sortat al cheilor din arbore.

Deoarece prin parcurgerea în inordine a unui arbore binar de căutare se obține șirul sortat al cheilor, rangul unui element este de fapt poziția sa în urma parcurgerii în inordine a arborelui.

(I) Căutarea elementului de rang i **Observații**

- Pornind de la rădăcina $T.rad$, în cazul parcurgerii în inordine, întâi se parcurg toate elementele din subarborele drept, care sunt în număr de $T.rad.st.dim$, după care se afișază cheia rădăcinii, iar apoi se parcurg elementele din subarborele drept. Deci rangul lui $T.rad$ este $T.rad.st.dim + 1$.
- Pornind de la un nod x , rangul lui x în cadrul mulțimii formate din elementele aflate în subarborele x este $x.st.dim + 1$.

Putem astfel determina elementul de rang i prin următorul algoritim:

Algorithm: Select(T, R)

```

 $x \leftarrow T.rad$ 
cat  $\_timp$   $x \neq T.Nil$  executa
     $\_rang \leftarrow x.st.dim + 1$ 
    daca  $R = \_rang$  atunci
        | RETURN  $x$ 
    sfarsit  $\_daca$ 
    daca  $R < \_rang$  atunci
        |  $x \leftarrow x.st$ 
    sfarsit  $\_daca$ 
    altfel
        |  $x \leftarrow x.dr$ 
        |  $R \leftarrow R - \_rang$ 
    sfarsit  $\_daca$ 
sfarsit  $\_cat\_timp$ 
RETURN  $x$ 

```

Exemplul: În arborele din figura 9.1 se caută nodul de rang $r = 16$. Exemplul este ilustrat în figura 9.2

Notăm cu x nodul curent.

- Inițial $x = T.rad$, deci $_rang = x.st.dim + 1 = 13 + 1 = 14$. Rădăcina se află pe poziția 14 în parcurgerea în inordine, deoarece trebuie întâi să se parcurgă toate cele 13 noduri din subarborele stâng.
- $_rang < r \Rightarrow$ cobor pe dreapta în arbore prin $x = x.dr$ iar variabila $r = r - _rang = 2$, adică se caută elementul de rang 2 în subarborele de rădăcină $x.dr$, care are cheia 58. Rădăcina se află pe locul 14, iar nodul pe care îl caut se află cu 2 poziții mai la dreapta de rădăcină, deci pe poziția 2 în subarborele drept.
- În continuare nodul curent este $x = T.rad.dr$, în această situație $_rang = x.st.dim + 1 = 4$, deci x se află pe poziția 4 în subarborele curent, dar nodul căutat se află pe poziția 2 în acest subarbore.

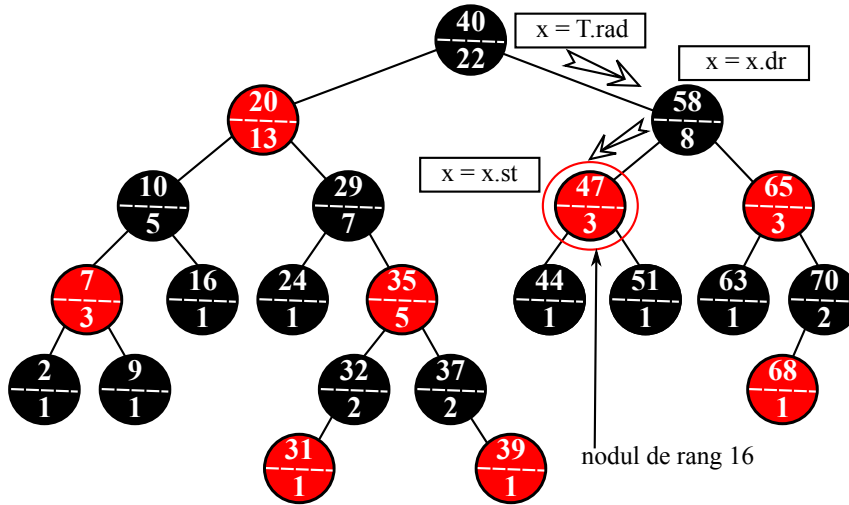


Figure 9.2: Determinarea elementului cu rangul 16.

- Se intră deci pe prima ramură a instrucțiunii condiționale și $x = x.st$ Astfel nodul curent este $T.rad.dr.st$ cu cheia 47 și variabila $rang = x.st.dim + 1 = 2$
- Nodul curent x cu cheia 47 are rangul $2 = r \Rightarrow$ am ajuns la nodul căutat.

Nodul cu rangul 16 este cel cu cheia 47. Se observă ușor faptul că șirul sortat al cheilor este $\{2, 7, 9, 10, 16, 20, 24, 29, 31, 32, 35, 37, 39, 40, 44, 47, 51, 58, 63, 65, 68, 70\}$, iar al 16-lea element este 47.

Complexitate: algoritmul coboară la fiecare apel iterație cu un nivel în arbore. Rezultă faptul că timpul de execuție este direct proporțional cu înălțimea arborelui, deci este $O(\log_2 n)$.

(II) Determinarea rangului unui element

Observăm următoarele lucruri:

- Dacă la nodul x se ajunge doar prin coborârea în arbore pe descendenți stângi, atunci rangul lui este chiar $x.st.dim + 1$, adică rangul său în cadrul subarborelui de rădăcină x .
- Dacă x se află la dreapta părintelui său, adică $x.p.dr = x$, atunci în parcurgerea în inordine subarborelui de rădăcină $x.p$, înaintea lui x se vor afla $x.p.st.dim + 1$ elemente.

Putem afla astfel rangul unui nod x în modul următor:

- Pornim de la nodul x către rădăcină, inițializând rangul cu $x.st.dim + 1$
- Atunci când nodul curent pe drumul spre rădăcină se află pe stânga părintelui său, se lasă rangul nemodificat
- Atunci când nodul curent pe drumul spre rădăcină se află pe dreapta părintelui său, la rang se adaugă $x.p.st.dim + 1$
- Algoritmul se încheie atunci când nodul curent l-a care s-a ajuns este chiar rădăcina

Algoritm: Rang(x)

```

rang  $\leftarrow x.st.dim + 1$ 
cat timp  $x \neq T.rad$  executa
     $\bar{y} \leftarrow x.p$ 
    daca  $x = y.dr$  atunci
        |  $rang \leftarrow rang + y.st.dim + 1$ 
    sfarsit _daca
     $x \leftarrow y$ 
sfarsit _cat _timp
RETURN rang

```

Exemplul: În arborele din figura 9.1 se determină rangul nodului x cu cheia 32.

- $x = x.p.st$, deci poziția lui x în parcurgerea în inordine a arborelui de rădăcină $x.p$ este aceeași cu poziția în parcurgerea în inordine a subarborelui cu rădăcina $x.p$, adică $x.st.dim + 1 = 2$, deci $rang = 2$
- nodul curent devine $x = x.p$ (adică nodul cu cheia 35), $rang = 2$. Noul nod curent cu cheia 35 se află pe dreapta părintelui, deci poziția lui în parcurgerea în inordine a subarborelui de rădăcină $x.p$ este poziția lui $x.p$ + rangul în subarborele de rădăcină x , adică $x.p.st.dim + 1 + rang = 4$.
- nodul curent devine din nou $x = x.p$ (adică nodul cu cheia 29). Acest nod se află pe dreapta părintelui, deci $rang = rang + x.p.st.dim + 1 = 4 + 5 + 1 = 10$
- se trece la nodul părinte, care este nodul cu cheia 20, care se află pe stânga rădăcinii
- Rezultatul final este: nodul cu cheia 32 are $rang = 10$, deci se află pe poziția 10 în parcurgerea în inordine a arborelui.

Complexitate: se parcurge arborele din nodul x către rădăcină, deci complexitatea este $O(\log_2 n)$

Refacerea informației dimensiunii subarborilor după operații de modificare

1. **La inserție:** la fiecare nod care este parcurs pe drumul spre locul de inserție se incrementează câmpul dim cu o unitate. Nodul nou adăugat va avea câmpul $dim = 1$.

2. **La ștergere:** se parcurge arborele de la părintele nodul z care a fost șters (dacă acesta a avut cel mult un descendent) respectiv de la părintele succesorului lui z , către rădăcină și pentru fiecare nod de pe acest drum, câmpul dim se decrementează cu o unitate.
3. **La refacerea proprietăților de ARN**, operația de rotație produce modificări în numărul de noduri ale unui subarbore în modul următor: considerăm rotația la dreapta în jurul nodului x din figura 9.3.

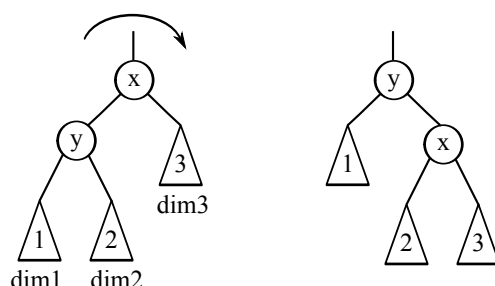


Figure 9.3:

Se observă din fig.9.3 că inițial $x.dim = y.dim + dim3 + 1$. După rotație y urcă în locul lui x , dar numărul de noduri din subarboarele considerat rămâne nemodificat, deci:

- $y.dim = x.dim$ (valoarea originală a câmpului).
- x coboară în dreapta lui y și are noua valoare a câmpului dimensiune:
 $x.dim = dim2 + dim3 + 1 = x.st.dim + x.dr.dim + 1$

Deci la funcția de rotație trebuie adăugate aceste două linii de cod:

```

y.dim = x.dim
x.dim = x.st.dim + x.dr.dim + 1

```

Etapele îmbogățirii în cazul arborilor pentru statistici de ordine

1. Alegerea unei structuri de date de bază: un ARN
2. Determinarea informației suplimentare: câmpul dimesiune = nr. de noduri di subarboarele curent
3. Verificarea actualizării în timp logaritmice: - s-a arătat mai sus
4. Dezvoltarea de noi operații utile asupra structurii de date îmbogățite: funcțiile: Stat_SELECT și Stat_RANG.

9.3 Îmbogățirea arborilor RN - teorema complexității

Următoarea teoremă demonstrează faptul că, dacă informația suplimentară păstrată în fiecare nod respectă anumite condiții, atunci complexitatea operațiilor de inserție/ștergere într-un arbore RN își păstrează complexitatea nemodificată, adică $O(\log_2 n)$.

Teoremă: Considerăm un atribut/câmp suplimentar f prin care se îmbogățeste un arbore roșu-negru T cu n noduri. Dacă valoarea câmpului f al oricărui nod x depinde doar de informațiile din nodurile x , $x.st$, $x.dr$ și eventual de valorile $x.st.f$ și $x.dr.f$, atunci valoarea câmpului suplimentar poate fi actualizată la orice operație de inserție/ștergere fără a afecta complexitatea $O(\log_2 n)$ a acestor operații.

Demonstrație: În cazul inserției: dacă s-a inserat nodul x ca descendent al nodului $x.p$, valoarea lui $x.f$ se calculează instantaneu, deoarece depinde doar de valorile din x și de santinela $T.nil$. Modificarea adusă de inserarea lui x se poate reflecta asupra părintelui $x.p$. Modificarea lui $x.p.f$ este de asemenea $O(1)$, iar modificările se refelectă doar asupra părintelui acestuia. Astfel se parcurge arborele de la x către rădăcină și se actualizează doar câmpul f pentru nodurile aflate pe acest drum. Ceea ce duce la complexitate $O(\log_2 n)$. În cazul operației de refacere a proprietăților RN, apar un număr limitat de rotații, la care doar câmpurile a două noduri sunt actualizate, iar modificările acestora se propagă doar către părinte și astfel, iar către rădăcină într-un timp $O(\log_2 n)$. Astfel per total se păstrează complexitatea $O(\log_2 n)$.

Aceeași argumentație se poate efectua în cazul operației de ștergere.

9.4 Arbori de intervale

Intervalele pot fi utilizate pentru reprezentarea și gestionarea evenimentelor/proceselor care se desfășoară în timp. Un arbore binar de căutare, care are ca informație un interval, poate fi utilizat de exemplu pentru căutarea într-o bază de date conținând evenimente ce se desfășoară în timp. Pentru a obține operații de inserție, ștergere și căutare eficientă se poate utiliza ca structură de bază un arbore roșu-negru.

Un interval închis $i = [t_1, t_2]$ este caracterizat prin cele două capete ale sale t_1 și t_2 .

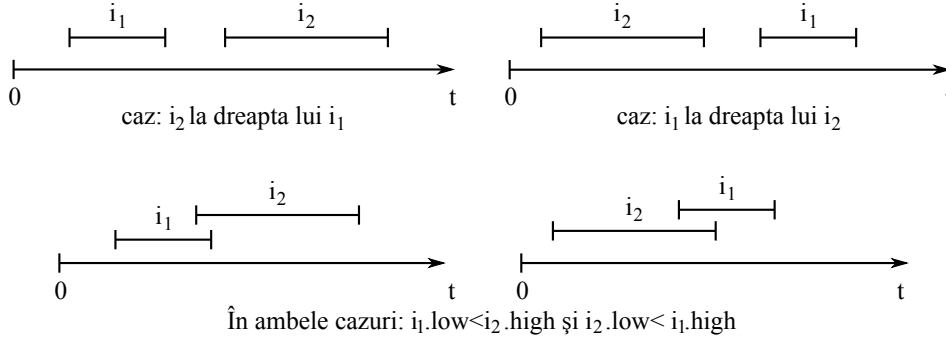
Notăm: $i.low = t_1$ și $i.high = t_2$.

Putem compara două intervale prin verificarea dacă se intersectează sau nu.

Observăm următoarele: considerând două intervale i_1 și i_2 putem avea următoarele situații:

- Cele două intervale nu se intersectează deloc, atunci sau $i_1.high < i_2.low$ (adică i_2 e la dreapta lui i_1) sau $i_2.high < i_1.low$ (adică i_1 e la dreapta lui i_2).
- Se intersectează, atunci $i_1.low \leq i_2.high$ și $i_2.low \leq i_1.high$

Un arbore de intervale se construiește respectând etapele generale definite mai sus:



1. Alegerea unei structuri de date de bază: un ARN, în care fiecare nod are ca informație un interval $x.int$, iar cheia după care se realizează inserția/ștergerea în arbore este $x.int.low$, adică ordonarea intervalelor se realizează în funcție de capătul din stânga. Inserția și ștergerea se efectuează ca în orice arbore RN. Căutarea se efectuează prin $INTERVAL_SEARCH(T, i)$, care caută în T un nod x pentru care $x.int$ se intersectează cu i sau $T.nil$ dacă nu există nici un astfel de interval.
2. Informația suplimentară prin care se îmbogățește structura de date: fiecare nod x conține un câmp $x.max = \max(y.int.high, y \text{ nod din subarborele de rădăcină } x)$.
3. Actualizarea informației suplimentare în urma modificării arborelui prin inserție/ștergere: s-a enunțat înainte o teoremă prin care se demonstra că, dacă informația suplimentară dintr-un nod x depinde doar de cheia lui x , de cheile descendenților săi și de informațiile suplimentare din descendenții săi, atunci actualizarea informației nu crește complexitatea operațiilor de inserție/ștergere. Dar în cazul unui arbore de intervale:

$$x.max = \max(x.int.high, x.st.max, x.dr.max)$$

deci, condițiile se respectă.

4. Operația nouă care se dezvoltă pe baza informației suplimentare este $INTERVAL_SEARCH(T, i)$, al cărei algoritm este prezentat în continuare.

Algoritm: Interval-Search(T, i)

```

 $x \leftarrow T.rad$ 
cat_timp  $x \neq T.rad$  si  $i$  nu se intersect cu  $x$  executa
    daca  $x.st \neq T.nil$  si  $i.low \leq x.st.max$  atunci
         $x \leftarrow x.st$ 
    sfarsit_daca
    altfel
         $x \leftarrow x.dr$ 
    sfarsit_daca
sfarsit_cat_timp
RETURN  $x$ 

```

Exemplu: Un exemplu de arbore de intervale este prezentat în fig. 9.4.

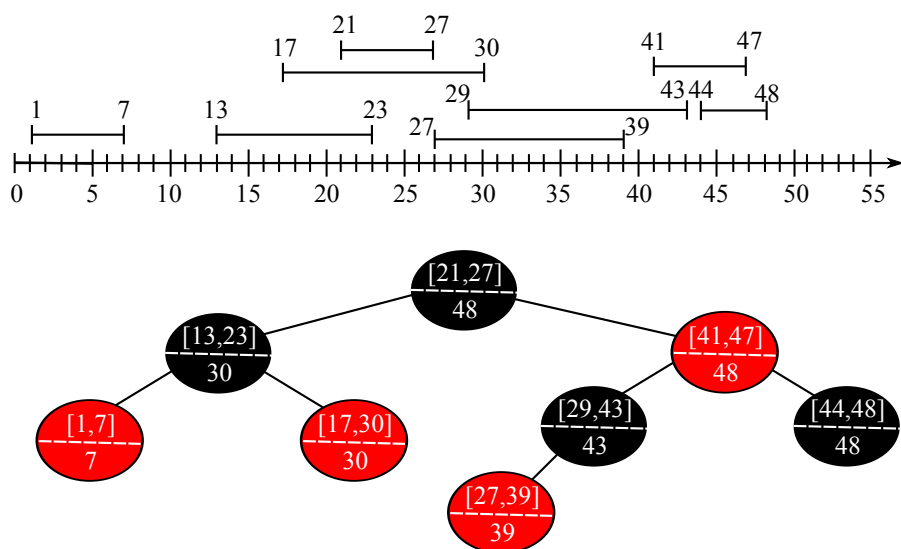


Figure 9.4: O mulțime de intervale împreună cu arborele RN corespunzător.

Chapter 10

B Arbori

B-arborii sunt arbori balansați în care toate frunzele au aceeași adâncime. Acest tip de arbori reprezintă o generalizare a arborilor binari de căutare. Fiecare nod poate stoca mai multe chei, de la câteva, la mai multe mii. Alături de chei, un nod al arborelui poate conține și alte informații de interes stocate în structura de date.

O variantă utilizată frecvent a B-arborilor sunt arborii B+, care sunt B-arbori cu proprietatea că informațiile suplimentare diferite de chei sunt stocate doar în frunze. Nodurile interne conțin doar chei.

10.1 Elemente de bază

Proprietăți

1. Fiecare nod x are următoarele atribute (câmpuri):
 - (a) $x.n$ numărul de chei stocate în nodul x .
 - (b) Vectorul de chei $x.key$, cu cheile sortate creșcător.
 - (c) $x.leaf$ = un câmp boolean care este TRUE, dacă x este frunză și FALSE altfel.
 - (d) Vectorul de legături către fiii lui x : $x.c$
2. Fiecare nod intern x are $x.n + 1$ fi.
3. Valorile cheilor $x.key(i), i = 1, \dots, x.n$, separă valorile cheilor fiilor lui x după cum urmează: notăm cu $y_i = x.c(i), i = 1, \dots, x.n + 1$ atunci:

$$\begin{aligned} y_1.key(1) &\leq y_1.key(2) \leq \dots \leq y_1.key(y_1.n) \leq x.key(1) \leq \\ &\leq y_2.key(1) \leq y_2.key(2) \leq \dots \leq y_2.key(y_2.n) \leq x.key(2) \leq \\ &\dots \end{aligned}$$

$$\leq y_{x.n+1}.key(1) \leq y_{x.n+1}.key(2) \leq \dots \leq y_{x.n+1}.key(y_{x.n+1}.n)$$

De fapt cheile $x.key(i-1)$ și $x.key(i)$ definesc intervalul în care se pot afla valorile cheilor din subarborele $x.c(i)$, $i = \overline{2, x.n}$. Cheile din subarborele de rădăcină $x.c(1)$ sunt mai mici decât cheia $x.key(1)$ și cheile din subarborele de rădăcină $x.c(x.n+1)$ sunt mai mari decât cheia $x.key(x.n)$.

De exemplu: în figura 10.1, în nodul x cheile sunt ordonate. x are 3 chei, deci are 4 copii. Copilul y_1 are cheile mai mici decât $x.key(1) = 10$, copilul y_2 are cheile în intervalul $[10, 25]$, determinat de cheile $x.key(1)$, $x.key(2)$, copilul y_3 are cheile în intervalul $[25, 30]$ determinat de cheile $x.key(2)$, $x.key(3)$, iar copilul y_4 are cheile mai mari decât $38 = x.key(3)$.

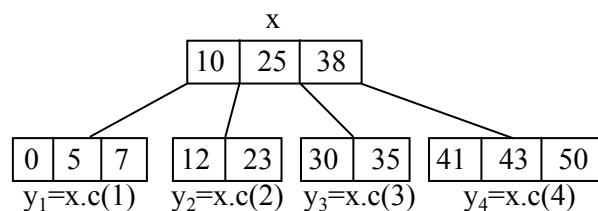


Figure 10.1: Exemplu de B-arbore

4. Toate frunzele au aceeași adâncime = adâncimea/înălțimea h a arborelui.
5. Numărul de chei ale unui nod este limitat inferior și superior pe baza unei constante t , $t \geq 2$, numită **gradul minim** al arborelui, după cum urmează:
 - (a) Fiecare nod x cu excepția rădăcinii, conține cel puțin $t - 1$ chei și are deci cel puțin t copii. Rădăcina conține cel puțin o cheie.
 - (b) Fiecare nod x conține cel mult $2t - 1$ chei, deci are cel mult $2t$ copii. Un nod care conține $2t - 1$ chei se numește **nod plin**.

Un exemplu de B-arbore cu $t = 2$ în care cheile sunt litere ale alfabetului este prezentat în figura 10.2. Ordinea considerată este cea lexicografică.

Înălțimea unui B-arbore

Considerăm un B-arbore de grad minim $t \geq 2$ cu n chei. Atunci înălțimea h a arborelui respectă inegalitatea:

$$h \leq \log_t \left(\frac{n+1}{2} \right)$$

Demonstrație: vom calcula înălțimea unui B-arbore T cu n chei pornind de la un

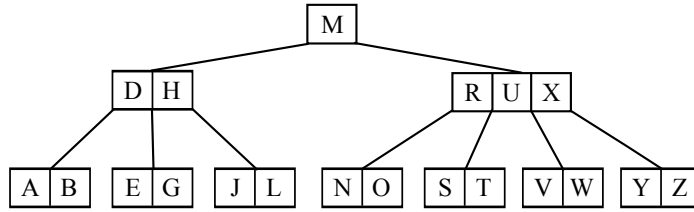
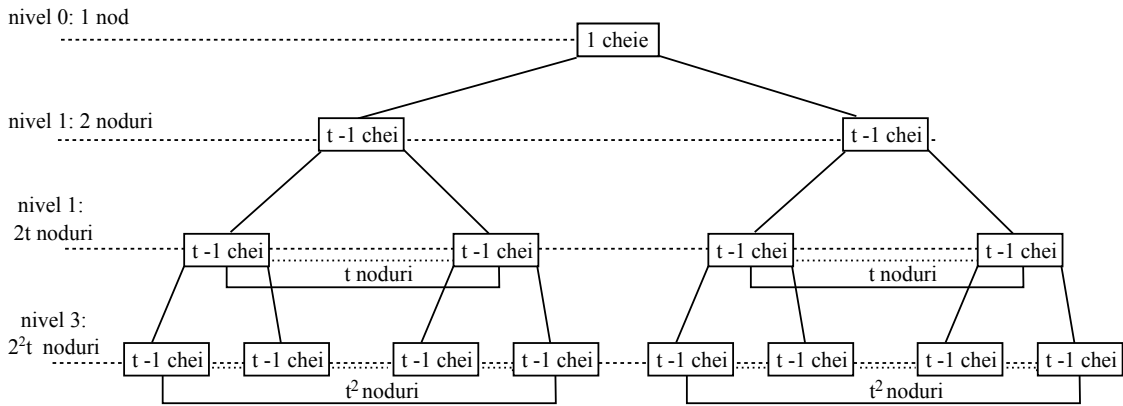


Figure 10.2: Exemplu de B-arbore

B-arbore T_1 cu aceeași înălțime h , dar cu număr minim de chei. Evident $n \geq$ numărul de chei T_1 .

Numărul de chei ale lui T_1 , reprezentat în figura 10.3, se calculează astfel:

Figure 10.3: Numărul minim de chei într-un B-arbore de grad minim t și înălțime h

Nivel 0 : rădăcina conține o singură cheie

Nivel 1 : conține 2 noduri a câte $t - 1$ chei

Nivel 2 : conține $2 * t$ noduri a câte $t - 1$ chei

Nivel 3 : conține $2 * t^2$ noduri a câte $t - 1$ chei

.....

Nivel h : conține $2 * t^{h-1}$ noduri a câte $t - 1$ chei

În total numărul de chei ale arborelui T_1 este:

$$\begin{aligned}
 &1 + 2(t - 1) + 2t(t - 1) + 2t^2(t - 1) + \dots + 2t^{h-1}(t - 1) = \\
 &= 1 + 2(t - 1)(1 + t + t^2 + \dots + t^{h-1}) =
 \end{aligned}$$

$$= 1 + 2(t-1)(t^h - 1)/(t-1) = 2t^h - 1.$$

Deci:

$$n \geq 2t^h - 1 \Rightarrow h \leq \log_t \left(\frac{n+1}{2} \right).$$

Înălțimea unui arbore este cu atât mai mică, cu cât gradul minim t este mai mare. Complexitatea operațiilor de căutare, inserție și ștergere depinde de înălțimea arborelui, deci de t .

Domenii de utilizare

B-arborii pot fi utilizați cu succes pentru accesul rapid al datelor de pe hard-disk, prin minimizarea numărului de operații de citire/scriere. În memoria principală (în memoria RAM), cu acces rapid, se păstrează rădăcina B-arborelui, iar nodurile din subarbore se păstrează în memoria secundară, mai lentă. Accesul la un nod din memoria secundară se realizează în complexitate $O(\log_t n)$, unde baza logaritmului depinde de gradul minim al arborelui, iar numărul de accesări ale discului în acest scop este dat de lungimea drumului de la rădăcină la nodul căutat. Cu cât gradul minim al arborelui este mai mare, cu atât înălțimea este mai mică și accesul la informație este mai rapid.

Multe baze de date utilizează B-arbori sau variante ale acestora pentru memorarea datelor.

10.2 Operații

Observație: precum s-a specificat anterior, B-arborii sunt utilizați în general pentru optimizarea accesului la memorie. Nodul rădăcină este memorat în memoria principală (în memoria RAM), astfel încât accesul la rădăcină nu necesită operații de citire pe disc. Accesul la noduri interne necesită citire pe disc.

Notăm prin: DR, DW operațiile de citire/scriere pe disc.

10.2.1 Căutarea unei chei

Căutarea într-un B-arbore este de fapt o generalizare a căutării binare, doar că spre deosebire de aceasta, pentru fiecare nod nu avem doar o decizie între două ramuri, ci avem o decizie între $n + 1$ ramuri, n fiind numărul de chei ale nodului curent.

Funcția $BT_CAUT(x, k)$ este definită recursiv și are ca parametri rădăcina x a sub-arborelui în care s-a ajuns cu căutarea și cheia căutată, k . Funcția returnează o pereche (x, i) , unde x = nodul care conține cheia k și i indicele cheii în nodul x , adică $x.key(i) = k$. Dacă în arbore nu se găsește cheia k , atunci funcția returnează NULL.

Algorithm: B_Search(*nod*, *key*)

```

i ← 1
cat timp i ≤ nod.n si nod.key(i) < key executa
|   i ← i + 1
sfarsit_cat_timp
daca i ≤ nod.n si nod.key(i) = key atunci
|   RETURN(nod, i)
sfarsit_daca
altfel
|   daca nod.leaf = true atunci
|   |   RETURN NULL
|   sfarsit_daca
|   altfel
|   |   DiskRead(nod.c(i))
|   |   RETURN B_Search(nod.c(i), key)
|   sfarsit_daca
sfarsit_daca

```

Complexitate: căutarea în fiecare nod este de ordinul $O(t)$, iar căutarea în arbore depinde de înălțimea h a arborelui, care este de ordinul $\log_t n$. Astfel complexitatea totală a algoritmului este $O(t \log_t n)$.

Datorită faptului că într-un nod cheile se păstrează sortate, se poate realiza și căutare binară. În acest caz, complexitatea devine: $O(\log_2 t \log_t n)$.

10.2.2 Inserarea unei chei

Inserarea unei chei într-un B-arbore diferă de inserarea într-un arbore binar prin faptul că nu putem pur și simplu crea o nouă frunză în B-arbore în care să inserăm cheia respectivă. Un astfel de procedeu ar duce la dezechilibrarea arborelui și ar contraveni proprietății unui B-arbore de a avea toate frunzele pe același nivel.

Insertia unei chei se realizează prin inserarea cheii într-un nod frunză deja existent. Problema apare atunci când frunza în care ar trebui inserat nodul este plină. În acest caz trebuie aplicată o procedură de DIVIZARE a frunzei în jurul cheii mediane (aflate pe poziția din mijloc în sirul cheilor).

Operația de divizare a unui nod

Se consideră un nod plin y , deci cu $2t - 1$ chei. Presupunem că y este al i -lea descendent al nodului x , care nu este plin. Atunci divizarea se realizează în modul următor:

- $x.n$ crește cu o unitate: $x.n = x.n + 1$.
- cheile lui x începând de la poziția i se deplasează cu o poziție la dreapta în nod.
- se deplasează pointerii către descendenții $x.c(i + 1), \dots, x.c(x.n)$ cu o poziție spre dreapta în nodul x .
- cheia $y.key(t)$ urcă în nodul x pe poziția i
- nodul y se divizează în două noduri noi care conțin câte $t - 1$ chei și anume primul nod conține cheile $y.key(1), \dots, y.key(t - 1)$ și al doilea nod conține cheile $y.key(t + 1), \dots, y.key(2t - 1)$
- noile noduri create devin descendenții $x.c(i)$ și $x.c(i + 1)$.

Această procedură este ilustrată în figura 10.4.

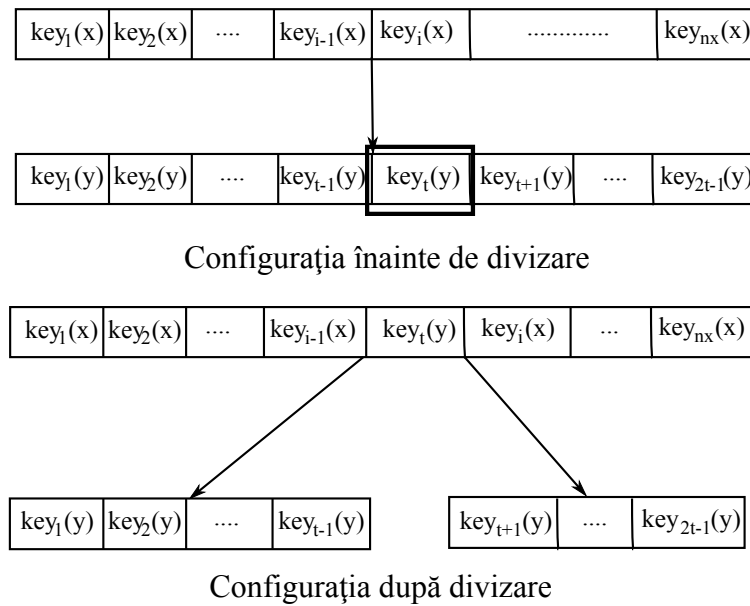


Figure 10.4: Divizarea unui nod plin

Observații:

1. Dacă x este la rândul său plin, nu putem efectua divizarea. De aceea la inserție trebuie să ne asigurăm deja pe parcursul căutării frunzei în care se realizează inserția că pe drum nu întâlnim noduri pline. În caz contrar se realizează o divizare.
2. Dacă nodul plin y este rădăcina, atunci nu există un părinte x , în care să se insereze cheia mediană din y . Astfel, dacă nodul care trebuie divizat este chiar rădăcina trebuie procedat în modul următor:

Divizarea rădăcinii

- Se creează un nou nod vid, care va fi noua rădăcină.
 - Se leagă vechea rădăcină ca descendent al noii rădăcini.
 - Se realizează operația de divizare.
3. Înălțimea unui B-arbore crește doar prin divizarea rădăcinii.

Exemplu: Inserarea cheii P în arborele din figura 10.5 cu $t = 2$. Căutarea începe

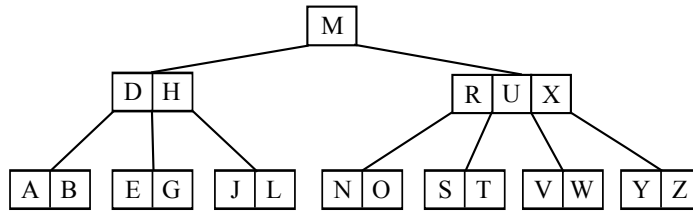


Figure 10.5: B-arbore în care se va realiza inserția de chei

de la nodul rădăcină. $T.rad.n = 1$, iar $T.rad.key(1) < P$, deci trebuie coborât la descendentul $T.rad.c(2)$. Dar Se observă că $T.rad.c(2).n = 2t - 1 = 3$, deci nodul este plin. Înainte de a continua, nodul $T.rad.c(2)$ trebuie divizat. Se obține arborele din fig 10.6:

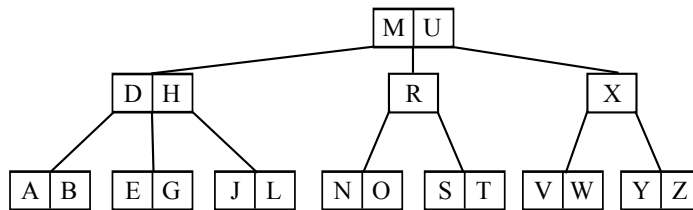


Figure 10.6: B-arborele după divizarea nodului cu cheile R, U, X

Acum $T.rad.key(1) = M < P$ și $T.rad.key(2) = U > P$, deci se coboară la descendentul $x_1 = T.rad.c(2)$.

Avem: $x_1.n = 1, x_1.key(1) = R > P$, deci se coboară la nodul $x_2 = x_1.c(1)$, care este frunză. $x_2.n = 2 < 2t - 1$, deci nodul nu este plin și se poate insera P pe poziția potrivită. Rezultă arborele din fig. 10.7.

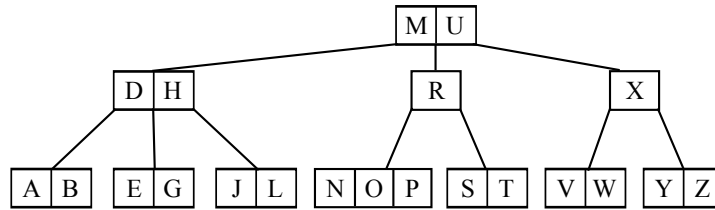


Figure 10.7: B-arborele rezultat după inserția cheii P

10.2.3 Eliminarea unei chei

Eliminarea unei chei dintr-un B-arbore este ceva mai delicată decât inserția, deoarece uneori trebuie eliminată și o cheie dintr-un nod intern. Acest lucru ar avea însă ca urmare reducerea numărului de chei din nod și ca urmare a numărului de descendenți ai nodului respectiv. Este deci necesară o rearanjare a cheilor și a descendenților în arbore. În plus, la eliminarea unei chei trebuie avut grijă ca numărul de chei dintr-un nod să nu devină mai mic decât $t - 1$. Astfel nu se poate pur și simplu extrage o cheie dintr-un nod cu $t - 1$ chei.

Aceste probleme se rezolvă prin operații de rotație și prin fuziuni.

Rotația într-un B-arbore

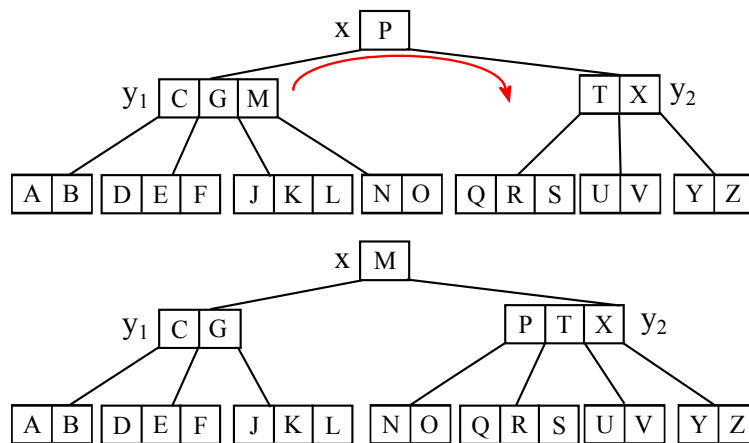


Figure 10.8: Rotație la dreapta în jurul cheii P din rădăcină.

Se consideră nodul x .

- O rotație la dreapta în jurul cheii $k = x.key(i)$:
 - mută ultima cheie a fiului $y_i = x.c(i)$ în nodul x în locul cheii k
 - mută cheia k pe prima poziție în nodul $y_{i+1} = x.c(i+1)$
 - mută ultimul fiu al lui y_i ca prim fiu al lui y_{i+1} .
- O rotație la stânga în jurul cheii $k = x.key(i)$:
 - mută prima cheie a fiului $y_{i+1} = x.c(i+1)$ în nodul x în locul cheii k
 - mută cheia k pe ultima poziție din nodul $y_i = x.c(i)$
 - mută primul fiu al lui y_{i+1} ca ultim fiu al lui y_i .

Un exemplu de rotație la dreapta într-un B-arbore este prezentat în figura 10.8.

Fuziunea a două noduri vecine într-un B-arbore

O fuziune nu poate fi realizată decât între doi frați vecini, fii ai aceluiași nod x , și care au ambii exact $t - 1$ chei. De asemenea părintele x trebuie să aibă cel puțin t chei. Se consideră nodul x , iar descendenții care fuzionează sunt $y = x.c(i)$ și $z = x.c(i+1)$. Fuziunea lui y cu z presupune reunirea celor două noduri într-unul singur, iar cheia $x.key(i)$ coboară din nodul x în noul nod rezultat pe poziția aflată între cheile lui y și cele ale lui z .

Această operație este reprezentată grafic în fig. 10.9.

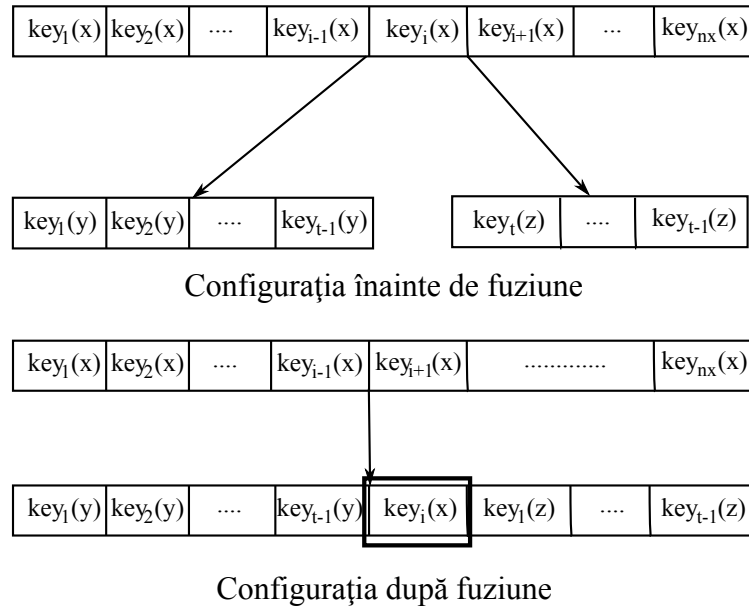


Figure 10.9: Fuziunea a două noduri vecine.

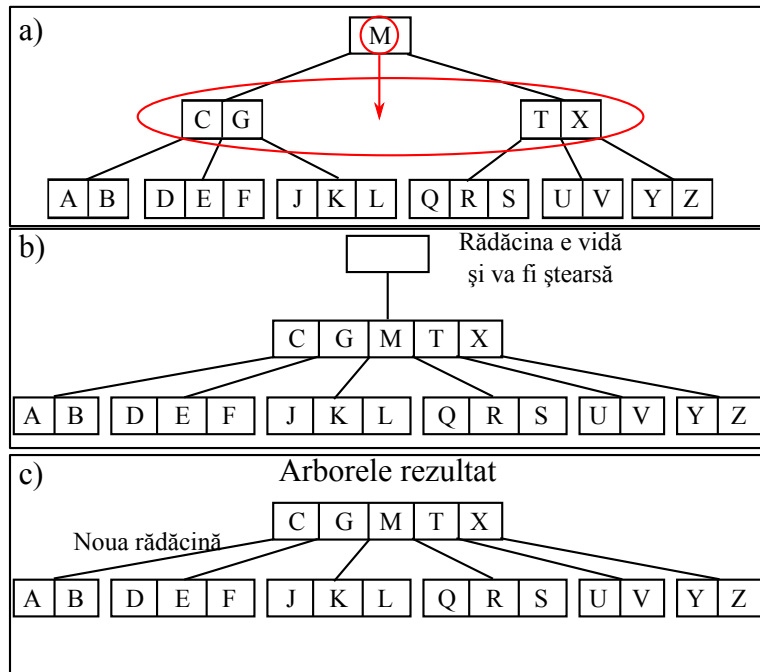


Figure 10.10: a) Nodurile care urmează să fuzioneze împreună cu cheia M a rădăcinii; b) Arborele după fuzionare; c) Rezultatul după eliminarea rădăcinii vide.

Un exemplu de fuziune a descendenților rădăcinii din fig. 10.10 este prezentat în ??.

Observații:

- Înălțimea unui arbore se micșorează atunci când rădăcina are doi fii care fuzionează, ca în exemplul din fig. 10.10.
- O fuziune se poate realiza doar dacă părintele nodurilor care fuzionează are cel puțin t chei. Din acest motiv, așa cum la inserție pe parcursul căutării nodului în care se inserează, toate nodurile pline întâlnite pe parcurs trebuie divizate, în cazul eliminării unei chei, pe parcursul căutării cheii respective, înainte de a coborî la un descendent, trebuie asigurat acestui descendent un minim de t chei.

Algoritm de eliminare a unei chei dintr-un B-arbore

Eliminarea chei k din B-arborele T cu gradul minim t va fi tratată în mod recursiv. Se pornește de la rădăcină și se coboară în arbore până la întâlnirea nodului ce conține cheia k .

Notăm cu x nodul curent. Pe parcurs ne asigurăm că orice nod în care se coboară are cel puțin t chei.

Cazul I: x este frunză și k este cheie a lui x . Din faptul că s-a avut grijă ca nodurile

în care se coboară să aibă cel puțin t chei, x are cel puțin t chei, deci pur și simplu se extrage cheia k din nodul x și se încheie algoritmul.

Cazul II: x nu este frunză și $k = x.key(i)$. Atunci:

- Dacă** $y = x.c(i)$ (copilul care îl precede pe k) are $y.n \geq t$ atunci se caută k' predecesorul lui k în subarborele de rădăcină y , se șterge recursiv k' din subarborele de rădăcină y și se înlocuiește k cu k' în x .
- Altfel dacă** $y = x.c(i + 1)$ (copilul care îi urmează lui k) are $y.n \geq t$ atunci se caută k'' succesorul lui k în subarborele de rădăcină y , se șterge recursiv k'' din acest subarbor și se înlocuiește k cu k'' în x .
- Altfel** - adică ambii copii aflați de o parte și de alta a lui k au exact $t - 1$ chei - atunci: se realizează o fuziune între cei doi copii $x.c(i)$ și $x.c(i + 1)$. Astfel se obține un nou nod, care va conține cheia k . Aceasta se șterge recursiv din noul nod obținut prin fuziune.

Cazul III: x nu este frunză și k nu este cheie a lui x .

Atunci se determină i astfel încât $x.key(i - 1) < k < x.key(i)$. Rezultă faptul că trebuie căutată cheia k în subarborele de rădăcină $x.c(i)$. Înainte de a coborî la fiul $x.c(i)$ al lui x trebuie să ne asigurăm că acest fiu are cel puțin t chei. Dacă $x.c(i)$ are doar $t - 1$ chei atunci:

- Dacă $x.c(i)$ are un frate vecin cu cel puțin t chei se realizează o rotație astfel încât o cheie din acel frate să urce la părintele x și o cheie din x să coboare la nodul $x.c(i)$. Apoi se șterge recursiv k din $x.c(i)$.
- Dacă ambii frați vecini au $t - 1$ chei trebuie realizată fuziunea lui $x.c(i)$ cu unul dintre acești frați. Apoi se șterge recursiv k din nodul rezultat prin fuziune.

Exemplu: Considerăm arborele din fig. 10.11 cu $t = 3$.

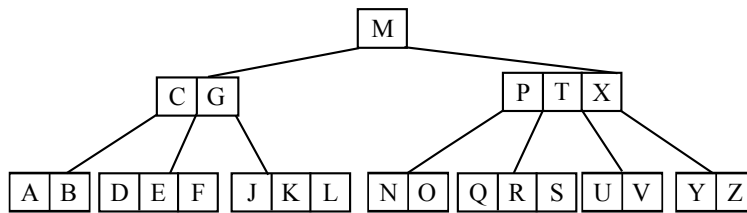


Figure 10.11: B-arbore cu $t = 3$ în care vor fi efectuate ștergeri de chei

Se șterge cheia G : se pornește de la rădăcină $G < M$ și suntem în cazul III. Trebuie coborât la primul copil. Acesta însă are exact $t - 1 = 2$ chei. Fratele drept al acestui nod are t chei, deci suntem în cazul III (a) $\Rightarrow$ rotație la stânga în jurul cheii M din rădăcină. Se obține arborele din fig. 10.12 a).

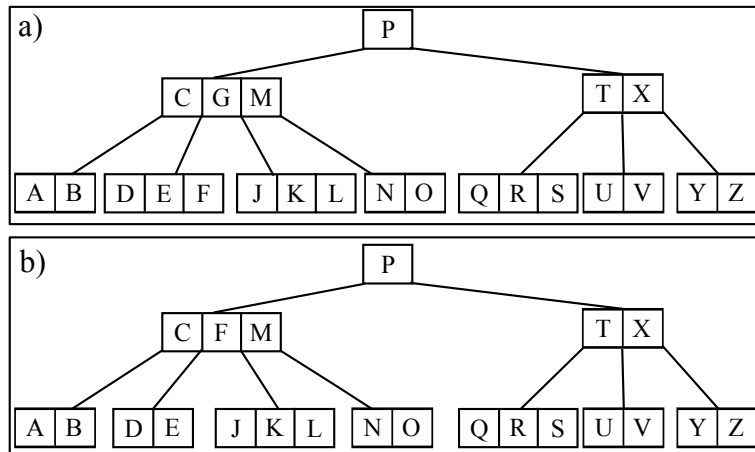


Figure 10.12: a) B-arborele din fig. 10.11 după rotație; b) Arborele după ștergerea cheii G .

Acum se coboară la nodul x care conține cheile C, G, M . Se observă că acesta nu e frunză, dar conține cheia $G = x.key(2)$ și ambii fii $x.c(2)$ și $x.c(3)$ au t chei, deci pot aplica cazul II (a) $\Rightarrow$ caut predecesorul lui G , care este F . Înlocuiesc G cu F și șterg F din fiul $x.c(2)$. Se obține arborele din fig. 10.12 b).

Se șterge cheia B : se pornește cu $x = T.rad$, se compară $B < x.key(1) \Rightarrow$ se studiază fiul $x.c(1)$. Acesta are t chei $\Rightarrow x = x.c(1)$.

Se observă că în acest moment $B < x.key(1) = C \Rightarrow$ se studiază fiul $x.c(1)$. Acest fiu nu are decât $t - 1$ chei, iar singurul său frate are tot $t - 1$ chei $\Rightarrow$ suntem în cazul III (b) $\Rightarrow$ trebuie realizată o fuziune între $x.c(1)$ și $x.c(2)$. Se obține arborele din fig. 10.13.

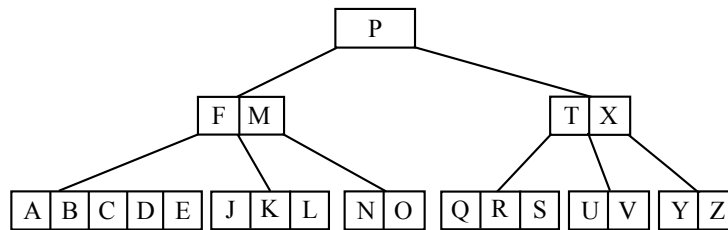
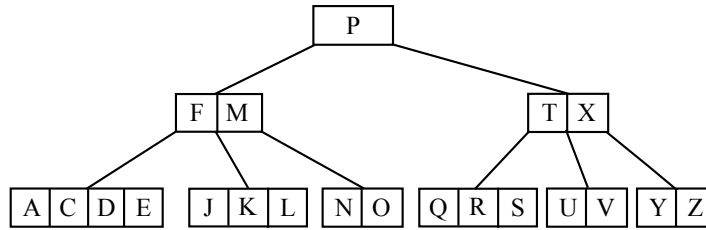


Figure 10.13: B-arborele după fuziunea nodurilor cu cheile A, B și respectiv D, E împreună cu cheia C .

Se coboară acum la $x = x.c(1)$ și se șterge cheia B din nod. Rezultă arborele din fig. 10.14.

Se șterge cheia X : se pornește de la rădăcina $x = T.rad$. $X > x.key(1) \Rightarrow$ se studiază fiul $x.c(2)$. Acesta are doar $t - 1$ chei. Unicul frate al său are tot $t - 1$ chei $\Rightarrow$

Figure 10.14: B-arborele după ștergerea cheii B .

fuziune. Rădăcina inițială rămâne fără nici o cheie, deci se șterge și noua rădăcină va fi nodul rezultat în urma fuziunii. Rezultă arborele din figura 10.15.

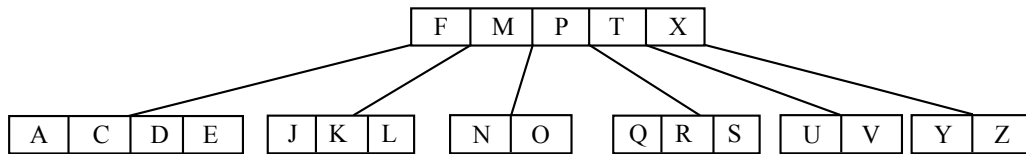
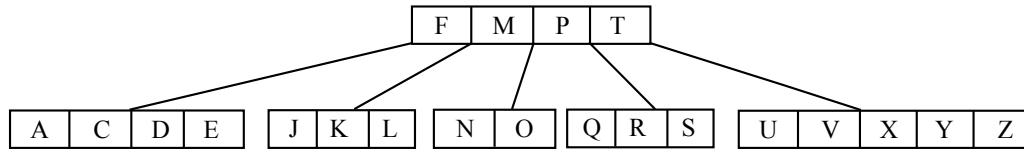


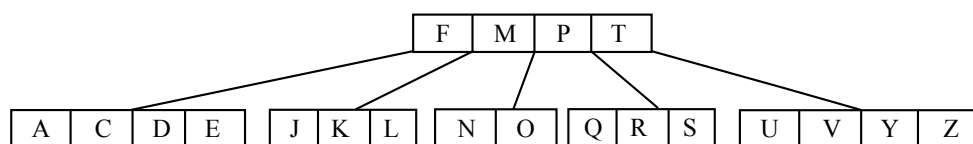
Figure 10.15: B-arborele după fuziunea fiilor rădăcinii și ștergerea acesteia.

Cheia X se găsește în rădăcină și ambii fi, de o parte și de alta a lui X au exact $t - 1$ chei $\Rightarrow$ fuziune. Rezultă arborele din fig. 10.16.

Figure 10.16: B-arborele după fuziunea nodurilor U, V și Y, Z .

Acum se poate șterge X din frunza care îl conține și se obține arborele din fig. 10.17.

Complexitate: Operațiile într-un B-arbore presupun căutare în interiorul unui nod, care este $O(t)$ precum și coborâre în arbore, care depinde de înălțimea h , deci este $O(\log_t n)$. Astfel complexitatea este $O(t \log_t n)$. Pentru t mare, poate fi utilizată căutarea binară într-un nod, astfel scade complexitatea acestei operații la $O(\log_2 t)$.

Figure 10.17: B-arborele după ștergerea cheii X .

Chapter 11

Arbori quad

Arborii quad sunt structuri de date ierarhice care au la bază un principiu de descompunere recursivă a spațiului de tip *divide et impera*. Denumirea provine de la faptul că fiecare nod intern al unui arbore quad are 4 fii.

Arborii quad se diferențiază după:

- Tipul de date pe care îl stochează: point data, regiuni, curbe, suprafețe, volume.
- Principiul de descompunere: descompunere în părți egale la fiecare nivel (eg. poligoane regulate) sau descompunere pe baza anumitor condiții de intrare.
- Rezoluția descompunerii: poate fi fixată dinainte sau poate depinde de anumite proprietăți ale datelor de intrare.

11.1 Quadrees pentru regiuni

Un arbore quad pentru regiuni se utilizează în general pentru partiționarea spațiului bi- sau tridimensional în regiuni, conform unui anumit criteriu de descompunere.

Exemplu: partiționarea unei imagini digitale în regiuni de culoare uniformă, conform algoritmului *Split and Merge*. Ideea principală a etapei de descompunere - *split* - este aceea de a descompune recursiv suprafața pătrată curentă în patru suprafețe pătrate de dimensiuni egale, până când se ajunge la suprafețe uniforme. Se consideră pentru aceasta imagini de dimensiune $2^n \times 2^n$.

În figura 11.1 este prezentat un exemplu de descompunere a unei imagini binare $2^3 \times 2^3$, iar în fig. 11.2 arborele quad corespunzător. Codurile asociate frunzelor vor fi explicate la sfârșitul secțiunii.

Observații:

- Structura pătrată pentru partiționarea unei regiuni este cea mai simplă, dar există și alte tipuri de descompuneri: triunghiuri echilaterale, triunghiuri dreptunghice isoscele, hexagoane echilaterale.

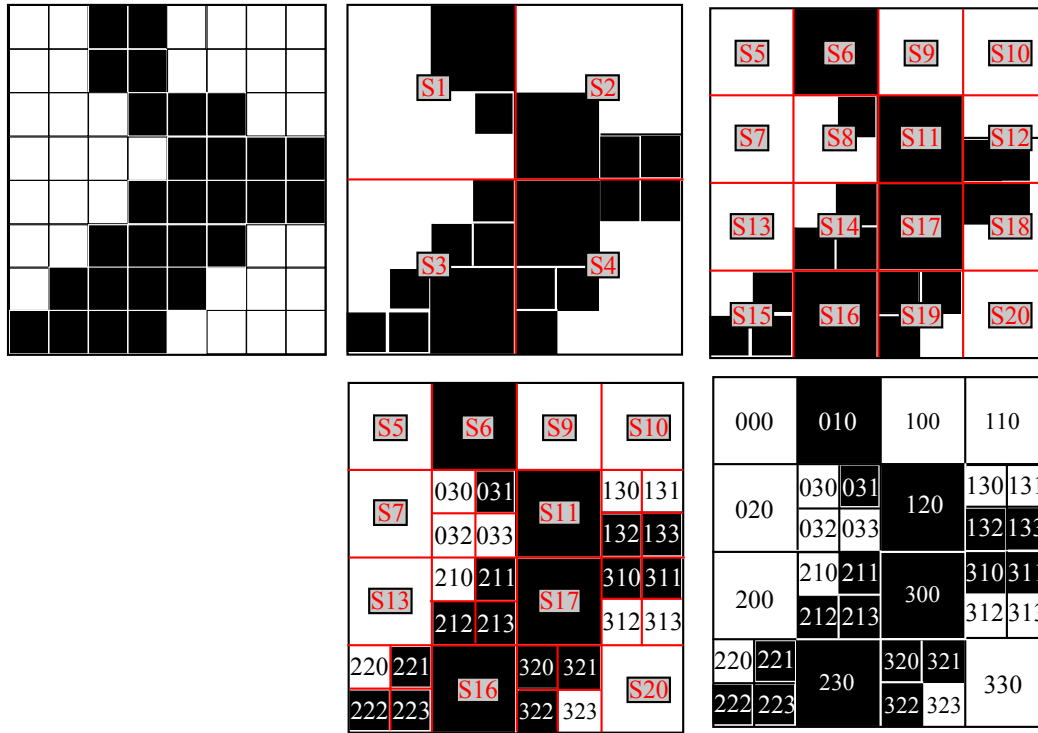


Figure 11.1: a) Imaginea binară originală; b) Rezultatul descompunerii

- Arborii quad cresc viteza de execuție a anumitor algoritmi pe imagini, suprafețe, structuri geometrice, dat fiind că majoritatea acestor algoritmi se reduc la o parcurgere, de obicei în preordine, a quadtree-ului asociat.
- În diferite situații - de exemplu etapa de merge a regiunilor vecine uniforme - este necesară identificarea vecinilor unui anumit nod.

11.1.1 Adiacență și vecinătate

Două regiuni se numesc vecine, dacă sunt adiacente.

Adiacența în spațiu nu înseamnă neapărat o relație simplă în cadrul nodurilor corespunzătoare din arbore. De exemplu, în figura 11.1 - regiunile 211 și 033 sunt adiacente, dar în cadrul quadtree-ului corespunzător din figura 11.2 nu sunt nici în relația de părinte-fiu, nici în relația de frate-frate. Se pune deci problema găsirii în arbore a două noduri, care în spațiu reprezintă regiuni vecine.

Fiecare nod al unui quadtree corespunde unei regiuni/unui bloc din imagine. Fiecare bloc are 4 laturi și 4 colțuri.

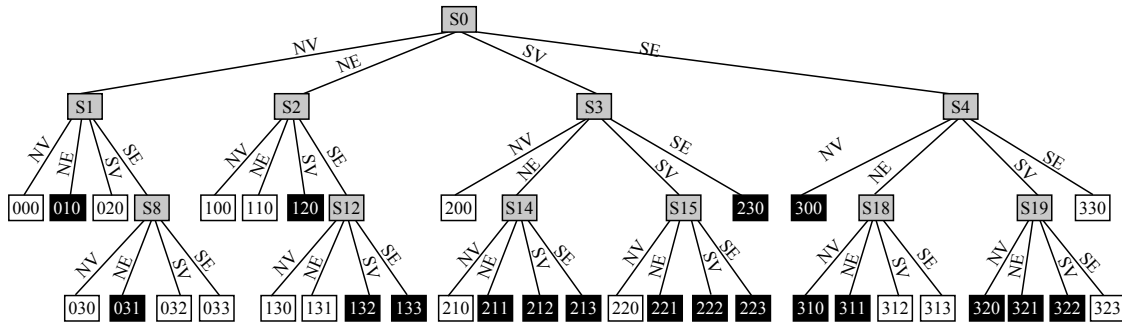


Figure 11.2: Arborele quad corespunzător imaginii binare din fig. 11.1

Notății:

- Cele patru laturi ale unui bloc: N (nord), S (sud), E (est), V (vest).
- Cele patru colțuri: NV, NE, SV, SE

Adiacență: spunem că două blocuri P și Q disjuncte sunt adiacente după direcția D ($D = N, S, E, V, NV, NE, SV, SE$), dacă

- P are o parte din latura de pe direcția D comună cu Q - de exemplu în figura 11.3 blocul F este vecin la E cu blocul G .
- Colțul din direcția D al blocului P este adiacent cu colțul opus al blocului Q - în exemplu, blocul 38 este NE adiacent cu blocul H

Probleme:

- (1) Determinarea vecinilor care sunt adiacenți cu întreaga latură a unui anumit bloc - de exemplu în figură, blocul I la V cu blocul H , sau blocul 37 la V cu blocul J . În acest caz problema se reduce pentru un anumit bloc P la determinarea celui mai mic bloc cu latura mai mare sau egală cu cea a lui P și care este adiacent cu P după direcția D .
- (2) Determinarea acelor vecini care sunt adiacenți numai cu un segment din latura unui anumit bloc - de exemplu în figură blocul 57 la V cu blocul M . În acest caz vecinul căutat trebuie specificat prin informații suplimentare

În continuare prezentăm tipurile de cereri care pot fi efectuate asupra unui quad-tree pentru determinarea de vecini.

Notății:

- G = "greater then or equal"
- C = "corner"

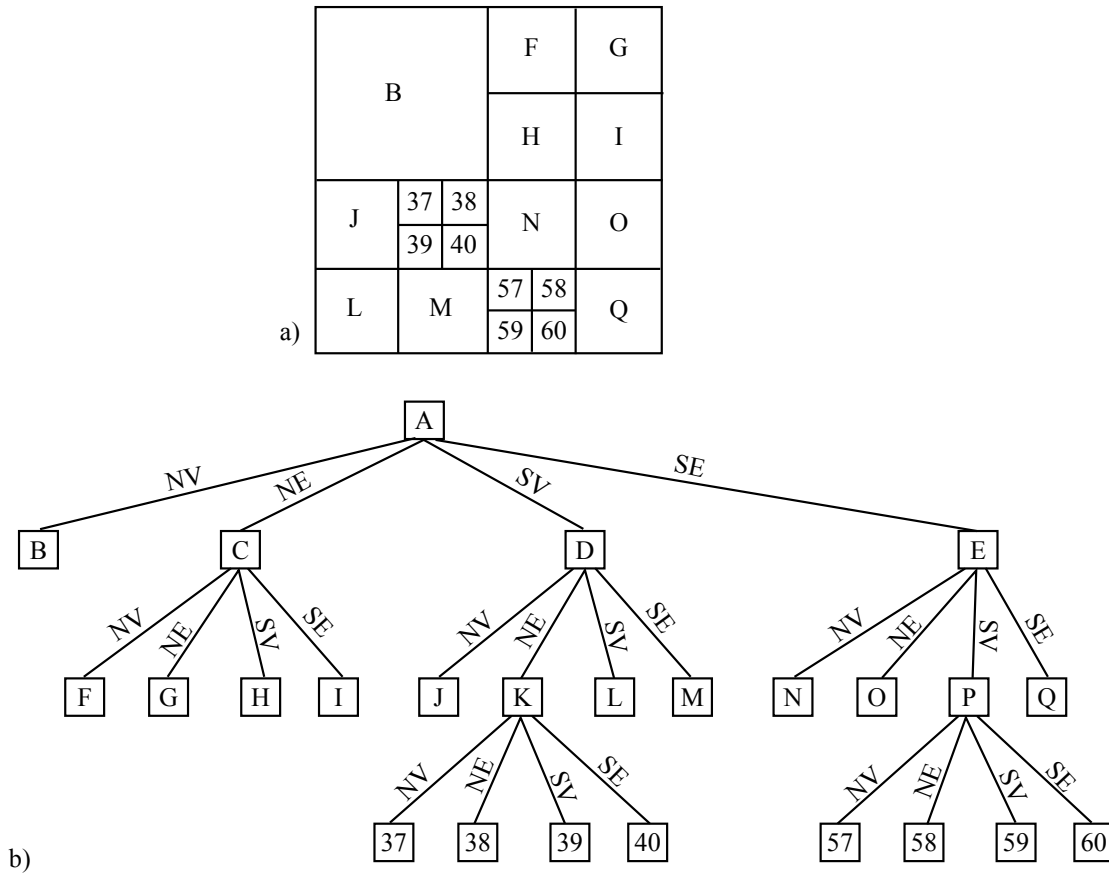


Figure 11.3: a) Suprafață descompusă în suprafețe pătrate uniforme; b) Arborele quad corespunzător

- S = "side"
- N = "neighbor"

Cereri

- (1) **GSN(P,D)** = cel mai mic bloc adiacent cu latura din direcția D a blocului P și care are latura mai mare sau egală cu latura lui P .
Exemplu: $GSN(J,N) = B$; $GSN(N,V) = K$ (care se descompune în blocurile 37,38,39,40).
- (2) **CSN(P,D,C)** = cel mai mic bloc care este adiacent cu latura D și colțul C al nodului P .
Exemplu: $CSN(J,E,SE) = 39$; $CSN(Q,V,NV) = 58$.
- (3) **GCN(P,C)** = cel mai mic bloc adiacent cu P și aflat de partea opusă a colțului C al blocului P și care are latura mai mare sau egală cu latura lui P . **Atenție:** acest

bloc trebuie să intersecteze suprafața determinată de colțul opus al colțului C .

Exemplu: $GCN(J, NE) = B$ NU 37; $GCN(B, SE) = E$; $GCN(58, NV) = N$.

- (4) $CCN(P, C) =$ cel mai mic bloc aflat de partea opusă a colțului C al blocului P .
Și în acest caz este obligatoriu ca blocul astfel determinat să se intersecteze cu suprafața determinată de colțul opus colțului C al lui P .

Exemplu: $CCN(58, NV) = N$; $CCN(59, NE) = 58$.

Observații:

- GSN , CSN , GCN , CCN nu definesc relații 1 - la - 1. Pot exista mai multe blocuri care au același vecin: $GSN(H, E) = B$ și $GSN(F, E) = B$.
- Relațiile de vecinătate astfel definite nu sunt neapărat simetrice: $GSN(H, E) = B$, dar $GSN(B, V) = C$.
- Un bloc care nu se află pe marginea imaginii are minim 5 vecini. Acest lucru poate fi dedus din următoarele observații:
 - Un bloc nu poate fi adiacent cu două blocuri de dimensiuni mai mari aflate pe direcții opuse (vezi blocul 37, fig. 11.3).
 - Un bloc poate avea doar doi vecini de latură mai mare, aflați pe direcții care nu sunt opuse (ex: vest și nord). Pe celelalte două laturi și pe un colț mai sunt vecini mai mici sau egali ca dimensiune. (vezi fig. 11.3).
- Un nod are maxim 8 vecini (de dimensiuni mai mari sau egale).

În continuare vor fi considerate doar cererile GSN și GCN .

11.1.2 Determinarea vecinilor adiacenți după o direcție verticală sau orizontală

Problematică: determinare unui bloc Q vecin al lui P , $Q = GSN(P, D)$.

Idee generală este aceea de a urca de la nodul P către primul strămoș comun cu Q , iar apoi de a coborî de la acest strămoș comun către Q .

Observații:

- Dacă direcția $D = E$, căutăm vecinul de la E , deci P se află pe ramuri SV sau NV față de strămoșul comun.
- Dacă direcția este $D = V$, căutăm vecinul la V deci P se află pe ramuri SE sau NE față de strămoșul comun.

- Dacă direcția $D = N$, căutăm vecinul de la N , deci P se află pe ramuri SV sau SE față de strămoșul comun.
- Dacă direcția $D = S$, căutăm vecinul de la S , deci P se află pe ramuri NV sau NE față de strămoșul comun

Strămoșul comun se obține când se urcă de la nodul curent către părinte pe o ramură ce nu conține direcția de căutare!

Exemplu: $GSN(N, V) = K$. Se observă că se urcă de la N către E , N fiind descendentul NV al lui E . Apoi de la E spre A , E fiind descendentul SE al lui A . În acest moment subarborele care îl conține pe K se află la **EST** față de nodul curent, care este chiar rădăcina. Deci vecinul de la **VEST** va fi pe una dintre ramurile NV sau SV ale lui A . Coborârea în arbore către vecinul căutat se face pe ramuri simetrice față de direcția D relativ la ramurile pe care s-a făcut urcarea către A .

Atenție: urcarea s-a făcut pe ramurile NV , SE , iar simetria este $D = V$, deci ramurile simetrice vor fi SV (simetricul lui SE față de verticală) și NE (simetricul lui NV față de verticală) și se observă că se ajunge la vecinul căutat K .

Algoritm general pentru GSN

```

GSN(P,D)
    nod=P
    parinte=nod.p
    Stiva={}
    cat timp (parinte≠NULL si ramura pe care urc de la nod la parinte
        contine D)
        pune pe Stiva simetricul ramurii de urcare de la nod la parinte
        fata de directia D
        nod=parinte
        parinte=nod.p
    sfarsit cat timp
    daca parinte=NULL atunci
        RETURN NULL
    sfarsit daca

    plaseaza pe stiva ramura simetrica fata de D
    a ramurii corespunzatoare a lui nod fata de parinte

    nod=parinte
    cat timp Stiva≠ {}
        directie ← Stiva

```

```

    daca nod ≠ frunza atunci
        nod = nod.directie
    altfel BREAK
    sfarsit cat timp
RETURN nod

```

Exemple: considerăm din nou figura

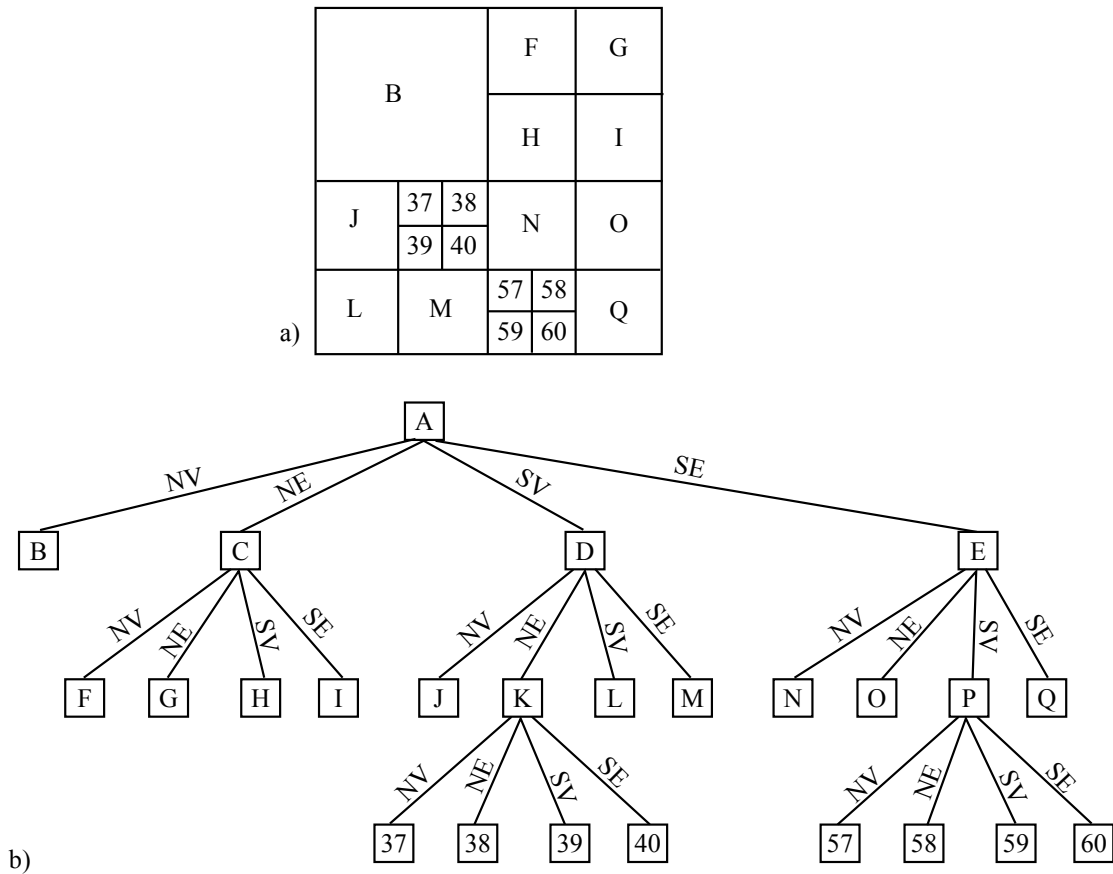


Figure 11.4: a) Suprafață descompusă în suprafețe pătrate uniforme; b) Arborele quad corespunzător

1. $GSN(39, N)$: $Stiva = \emptyset$, $nod = (39)$, $parinte = (K)$, $39 = K.SV$, $D = N$.

Deci ramura de urcare nu conține $N \Rightarrow$ strămoșul comun este chiar $K \Rightarrow$ se oprește bucla cât timp.

Se plasează pe stivă simetricul lui SV față de orizontală $\Rightarrow Stiva = (NV) \Rightarrow$ cobor din K pe ramura NV și ajung la nodul 37.

Din imagine se vede clar că s-a determinat vecinul dorit.

2. $GSN(59, V)$: Stiva = $\emptyset$, nod = (59), parinte = (P), (59) = (P).SV, $D = V$

Deci ramura de urcare conține direcția $V \Rightarrow$ plasez pe stivă simetricul lui SV față de verticală $\Rightarrow$ Stiva = (SE)

În plus nod = (P), parinte = (E) (P) = (E).SV, deci ramura de urcare conține $V \Rightarrow$ Stiva = (SE, SE).

În plus nod = (E), parinte = (A) și (E) = (A).SE. Nu mai conține direcția $\Rightarrow$ ne oprim din ciclare.

În plus se pune pe stivă SV, deci Stiva = (SV, SE, SE).

De la rădăcină se coboară pe ramurile din stivă: $A.SV = D$, $D.SE = M$ dar M este frunză deci ne oprim și vecinul căutat este M (se verifică corectitudinea în fig. 11.6)

$GSN(59, V) = M$

B		F		G	
		H		I	
J	37	38	N		O
	39	40			
L	M		57	58	Q
			59	60	

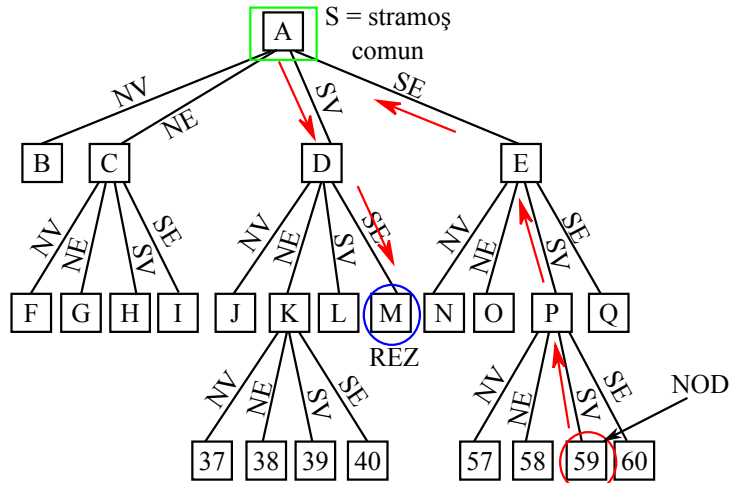


Figure 11.5: a) Suprafață descompusă în suprafețe pătrate uniforme; b) Drumul parcurs în arbore pentru găsirea vecinului cu $GSN(59, V)$

11.1.3 Determinarea vecinilor diagonali

Problematică: determinare unui bloc Q vecin al lui P , $Q = GCN(P, D_1 D_2)$, $D_1 \in \{N, S\}$, $D_2 \in \{E, V\}$.

Idee generală: caut un nod strămoș al lui P vecin orizontal / vertical cu un nod strămoș nodului căutat. Acest lucru se realizează după cum urmează:

- se urcă de la nodul curent spre părinte, până când nodul se află pe partea $D_1 D_2 \neq D_3 D_4$ a părintelui. În acest moment nodul curent devine acest părinte, pe care îl notăm cu Q .

- **dacă** $D_1 \neq D_3$ și $D_2 \neq D_4$ atunci Q este strămoș comun și tot ceea ce trebuie să fac este să cobor din Q pe arce complementare cu cele pe care s-a urcat, adică dacă s-a urcat pe arc SV se coboară pe arc NE , dacă s-a urcat pe arc SE se coboară pe arc NV , etc.
- **altfel**
 - **dacă** $D_1 \neq D_3$ atunci caut vecinul R lui Q după direcția D_2 cu algoritmul $R = GSN(Q, D_2)$, R va deveni nodul curent.
 - **altfel** dacă $D_2 \neq D_4$ atunci caut vecinul R lui Q după direcția D_1 cu algoritmul $R = GSN(Q, D_1)$, R va deveni nodul curent.
 - Din nodul R astfel obținut se coboară pe arce complementare cu cele pe care s-a ajuns la Q .

Perechile de arce complementare: $(NV, SE)(NE, SV)$.

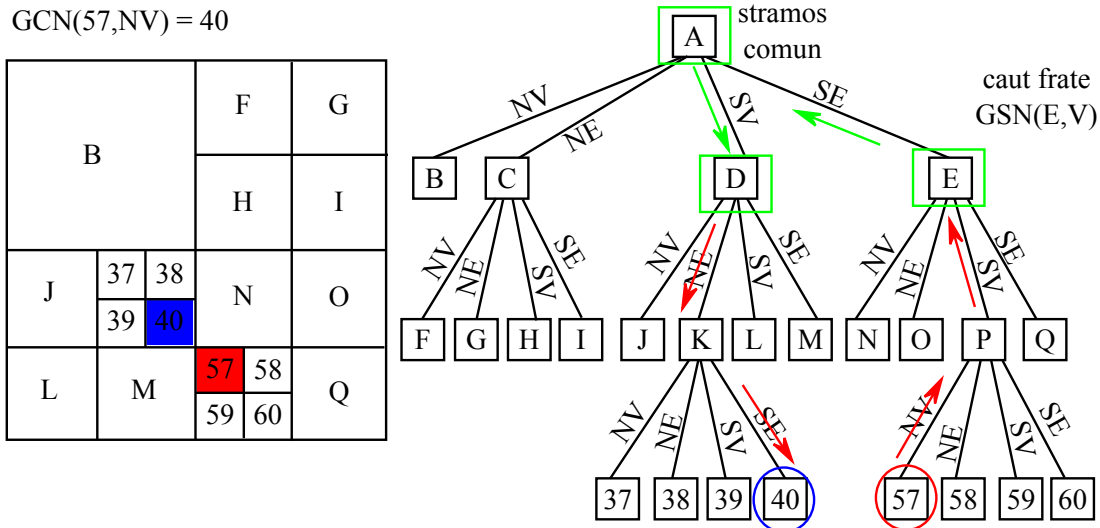


Figure 11.6: a) Suprafață descompusă în suprafețe pătrate uniforme; b) Drumul parcurs în arbore pentru găsirea vecinului cu $GCN(57, NV)$

Exemple:

1. $GCN(57, NV)$:

- urcăm la părintele P pe ramura NV egală cu direcția din apel, deci continuăm la părintele lui P , E , pe ramură $SV \neq NV$
- Dar $D_1 = N$, $D_2 = V$ și $D_3 = S$, $D_4 = V$, deci $D_1 \neq D_3$ și $D_2 = D_4$ $Q = E$, rezultă căutarea vecinului R al lui Q , după algoritmul $GSN(E, V)$, adică urc la părinte, cât timp sunt pe o ramură cu V . Ramura de urcare de la E la A este SE care nu conține pe V , deci ne oprim.

- Se mai urcă o dată. Părintele este A = rădăcina. Acum se coboară conform algoritmului GSN pe ramura SV , adică la D , care este vecinul la vest al lui E .
- Acum, conform algoritmului GCN , cobor pe ramură opusă a lui SV , adică NE și ajung la K și de aici cobor pe ramura SE și ajung la 40.
- Din figura 11.6 se observă că am ajuns exact la vecinul căutat.

2. $GCN(38, NE)$:

- Urcăm la părintele K pe o ramură NE , egală cu direcția de apel, deci se continuă urcarea la părintele lui K , D , pe o ramură NE , tot egală cu ramura de apel. Se urcă la părintele lui D , care este rădăcina A , pe o ramură $SV \neq NE$, cu $D_1 = N$, $D_2 = E$, $D_3 = S$, $D_4 = V$.
- Deci $D_1 \neq D_3$ și $D_2 \neq D_4$. Am ajuns la un părinte comun al lui 38 cu vecinul căutat. Se coboară acum pe ramurile NE la C și SV la H . Nu mai există descendenți pentru H , deci algoritmul se oprește.
- S-a obținut vecinul H , care este cel căutat (vezi fig. 11.5).

3. $GCN(N, NE)$:

- Urc de la N la părintele E pe o ramură NV . Observăm $NV \neq NE$ cu $D_1 = N = D_3$, $D_2 = E$, $D_4 = V$.
- Deci se aplică $GSN(E, N)$. Urcăm de la E la părintele A (rădăcina) pe o ramură SE . De aici se coboară pe o ramură simetrică față de orizontală, deci pe ramura NE la nodul C .
- Acum s-a găsit vecinul la N al lui E , care este C .
- De aici se continuă coborârea conform algoritmului GCN pe o ramură complementară cu NV , adică SE și se ajunge la I .
- Vecinul căutat este deci I , ceea ce rezultă și din figura 11.5.

Aplicații: algoritmul *Split and Merge* de segmentare pentru imagini digitale.

11.1.4 Linear Quadrees

Principala problemă a implementării clasice a unui arbore quad, adică folosind o structură de pointeri, în cazul unei imagini, este numărul mare de noduri interne necesare. De fapt în aplicații reale, în cazul imaginilor se utilizează o structură de date numită arbori quad liniari - *Linear Quadrees* - care reprezintă de fapt lista frunzelor în parcurgerea de la stânga la dreapta. Fiecare frunză este o structură de tip nod, care conține un câmp pentru culoare, un câmp pentru nivelul la care se află în arbore și un cod unic în baza 4.

Codul fiecărei frunze este alcătuit prin interclasarea codurilor binare ale coordonatelor y și x ale pixelului din colțul stânga-sus al blocului din imagine reprezentat prin frunză. Codul binar obținut prin interclasarea celor două coduri este transformat într-un cod în baza 4 prin gruparea cifrelor binare două câte două într-o cifră din baza 4.

Algoritmul presupune imagini de dimensiuni $2^n \times 2^n$.

Exemplu: în imaginea $2^3 \times 2^3$ din fig. 11.6 coordonatele y, x ale colțului stânga sus:

- ale blocului B : sunt 0, 0, în binar pe trei poziții $y = 000, x = 000$. după interclasare codul este 000000. Grupând două câte două cifrele binare și transformând în baza 4 obținem codul 000
- ale blocului F : sunt 0, 4, în binar pe trei poziții $y = 000, x = 100$. după interclasare codul este 010000. Grupând două câte două cifrele binare și transformând în baza 4 obținem codul 100
- ale blocului G : sunt 0, 6, în binar pe trei poziții $y = 000, x = 110$. după interclasare codul este 010100. Grupând două câte două cifrele binare și transformând în baza 4 obținem codul 110
- ale blocului 40: sunt 5, 3, în binar pe trei poziții $y = 101, x = 011$. după interclasare codul este 100111. Grupând două câte două cifrele binare și transformând în baza 4 obținem codul 213

În figura 11.7 este prezentat modul de codificare al frunzelor obținute prin descompunerea unei imagini binare $2^3 \times 2^3$ în blocuri de culoare uniformă.

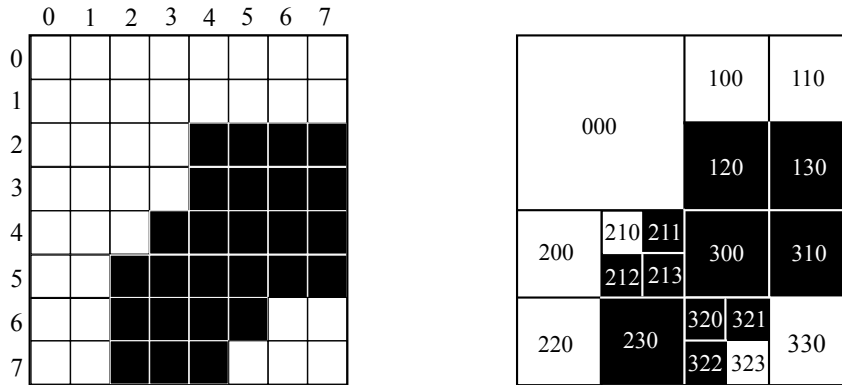


Figure 11.7: Suprafață descompusă în suprafețe pătrate uniforme și etichetele corespunzătoare în baza 4

Arborele corespunzător este reprezentat în fig. 11.8.

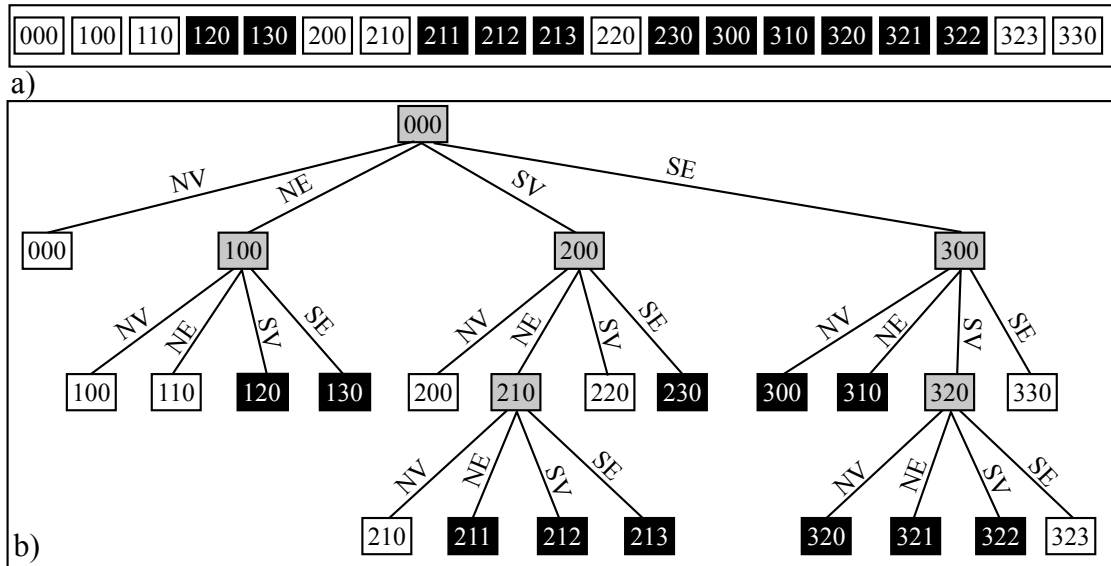


Figure 11.8: Arborele liniar păstrat sub forma unui vector ce conține doar etichetele frunzelor considerate de la stânga la dreapta și arborele quad corespunzător,

Determinarea vecinilor într-un QuadTree liniar

- **calculul codului unui vecin:** este realtiv simplă determinarea codului unui vecin de pe același nivel de descompunere, de exemplu pentru un anumit nod, vecinii de pe același nivel care au același părinte au codul în care diferă doar cifra corespunzătoare nivelului respectiv.
- **căutarea în lista de frunze:** nu este obligatoriu ca să existe un vecin pe același nivel, de exemplu blocul cu codul 210 nu are la nord vecin pe același nivel de descompunere, ci doar pe un nivel superior. Rezultă că, după calculul codului corespunzător vecinului, acesta trebuie căutat în lista de frunze.

11.2 Point Region Quad Trees - PR-Quadtrees

Arborii quad de tip point-region împart o suprafață pătrată în mod similar ca și arborii quad pentru suprafețe/imagini, cu deosebirea că împărțirea nu se realizează pe baza culorii suprafeței, ci pe baza unui set de puncte plasate în suprafața dată.

Idee generală: se consideră o suprafață bidimensională cu coordonatele (x, y) în domeniul $[0, dim] \times [0, dim]$ și n puncte $P_i = (x_i, y_i)$, $i = \overline{1, n}$ plasate pe această suprafață.

Suprafața se împarte în patru suprafețe egale în mod recursiv, până când fiecare frunză conține cel mult unul dintre punctele P_i .

Un exemplu este prezentat în figura 11.9.

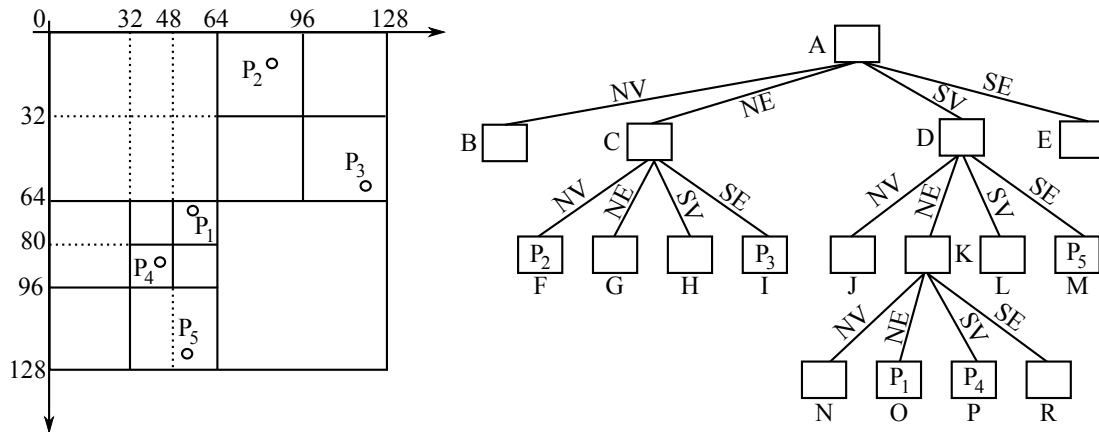


Figure 11.9: a) Suprafața pentru care se realizează împărțirea în regiuni împreună cu 5 puncte. b) PR-tree-ul corespunzător.

11.2.1 Căutarea unui nod

Se dorește căutarea nodului în care s-ar afla punctul P de coordonate $P.x$ și $P.y$, dacă acest punct ar exista pe suprafața dată. Dacă se consideră nodul curent Z la care s-a ajuns în arbore, notăm cu $Z.top_left$ colțul stânga sus al suprafeței reprezentate de Z și cu $Z.bottom_right$ colțul dreapta jos al acestei suprafețe. Pe baza coordonatelor acestor colțuri poate fi stabilit în care cadran ar trebui să se afle punctul P și deci pe ce ramură trebuie coborât în arbore.

Algoritm

```

PR_SEARCH(T,P)
    daca  $P.x < 0$  sau  $P.x > T.bottom\_right.x$ 
        sau  $P.y < 0$  sau  $P.y > T.bottom\_right.y$  atunci
            RETURN NULL
    sfarsit daca
    cat timp  $Z$  nu e frunza
        daca  $P.x \leq (Z.bottom\_right.x)/2$ 
            si  $P.y \leq (Z.bottom\_right.y)/2$  atunci
                 $Z = Z.NV$ 
        altfel
            daca  $P.x > (Z.bottom\_right.x)/2$  si
                 $P.y > (Z.bottom\_right.y)/2$  atunci
                     $Z = Z.SE$ 
            altfel
                daca  $P.x \leq (Z.bottom\_right.x)/2$  atunci
                     $Z = Z.SV$ 
                altfel  $Z = Z.NE$ 

```

```

                sfarsit daca
            sfarsit daca
        sfarsit daca
    sfarsit cat timp
RETURN Z

```

Exemplu: considerăm arborele din fig. 11.9, construit pe baza setului de puncte $\{(55, 67), (80, 15), (119, 58), (41, 85), (53, 117)\}$.

SEARCH(T, (50,105)):

$Z = A$ nu e frunză $\Rightarrow$ verific în care dintre cele 4 blocuri descendente poate fi găsit $P = (50, 105)$.

$$\left. \begin{array}{l} 0 < P.x = 50 < 64 \\ 64 < P.y = 105 < 128 \end{array} \right\} \Rightarrow Z = Z.SV = D$$

D nu este frunză $\Rightarrow$ verific în care dintre cele 4 blocuri descendente se plasează P .

$$\left. \begin{array}{l} 32 < P.x = 50 < 64 \\ 96 < P.y = 105 < 128 \end{array} \right\} \Rightarrow Z = Z.SE = M$$

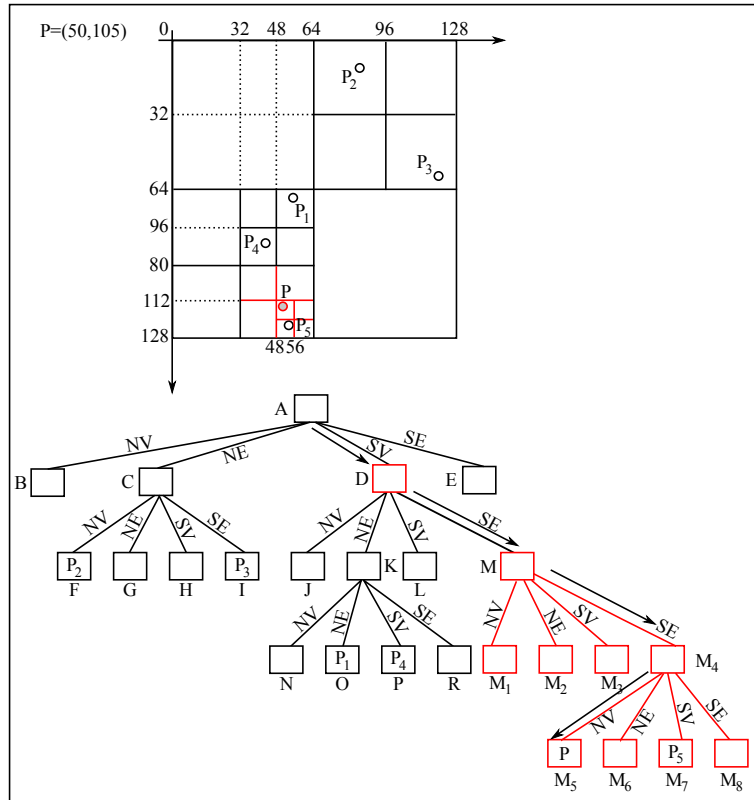
M este frunza corespunzătoare punctului $(50, 105)$.

11.2.2 Inserarea într-un PR-arbore

Dacă se dorește inserarea punctului P într-un PR-arbore quad, atunci, după identificarea frunzei M care corespunde punctului P , se verifică dacă frunza respectivă conține deja un punct, în exemplul anterior P_5 :

- Dacă nu, atunci se inserează P în câmpul informație.
- Altfel (cazul exemplului) se sparge blocul corespunzător frunzei M în 4 blocuri noi, frunza se înlocuiește cu un nod interior, care are 4 descendenți, corespunzători noilor blocuri obținute, se plasează punctul aflat în frunza M în nodul nou construit corespunzător și se continuă procesul de căutare a unei frunze corespunzătoare lui P , cu eventuale noi spargeri, dacă este necesar.

Exemplul inserției nodului $P = (50, 105)$ în arborele din fig. 11.9 este reprezentat în fig. 11.10.

Figure 11.10: Inserția punctului $P = (50, 105)$ în arborele din figură.

Algoritm

```

PR_INSERT(T, point, dim)
    daca point.x < 0 sau point.x > dim
        sau point.y < 0 sau point.y > dim atunci
            Scrie("nu poate fi inserat")
            RETURN

    sfarsit daca
    daca T = NULL atunci
        Aloca T
        T.info = point
        T.top_left = (0,0)
        T.bottom_right = (dim,dim)
        T.SV = T.SE = T.NV = T.NE = T.p = NULL
        RETURN
    sfarsit daca
    X = T
    repeta

```

```

//cauta frunza in care se plaseaza punctul point
Z = PR_SEARCH(X,point)
daca Z.info = NULL atunci
    Z.info = point
altfel
    1.  sparge suprafata din Z in patru suprafete
        plasate in noduri noi  $Z_1, Z_2, Z_3$  si  $Z_4$ 
    2.  plaseaza Z.info in nodul corespunzator  $Z_1, Z_2, Z_3$  sau  $Z_4$ 
    3.   $Z.NV = Z_1, Z.NE = Z_2, Z.SV = Z_3, Z.SE = Z_4$ 
    4.  Z.info = NULL
    5.   $Z_1.p = Z, Z_2.p = Z, Z_3.p = Z, Z_4.p = Z$ 
sfarsit daca
X = Z
//acum Z nu mai este frunza si se reia cautarea
pana cand X = frunza
RETURN

```

11.2.3 Construcția unui PR-arbore quad

Construcția unui PR-arbore quad se realizează prin inserții succesive într-un arbore inițial vid.

Algoritm

```

PR_CONSTRUCT(P,n,dim)
    Initializare T
    pentru i=1, n
        PR_INSERT(T, P[i])
    sfarsit pentru
RETURN T

```

11.2.4 Eliminarea unui punct P

Se caută frunza M corespunzătoare nodului P în arbore. Dacă M nu conține P , atunci P nu există în arbore și deci nu poate fi șters.

Altfel: se șterge P din frunza M .

Dacă frații lui M sunt frunze și în blocul reprezentat prin aceste 4 frunze se află cel mult un punct Q din mulțimea de puncte ale PR-arborelui, atunci cele patru frunze se șterg, iar punctul Q se mută în părintele lui M , care acum devine frunză.

Această etapă se repetă, până când se ajunge la un nod ai cărui descendenți nu sunt cu toții frunze sau sunt 4 frunze care conțin împreună cel puțin 2 puncte.

Utilizăm funcția IS_MERGEABLE(Y) care verifică dacă nodul Y conține 4 descendenți care pot fi eliminați și în caz afirmativ returnează informația (dacă există) a unicului

descendent frunză care nu este vid.

Algorithm

```

IS_MERGEABLE(Y)
    nr = 0
    info = NULL
    daca Y.NV ≠ frunza sau Y.NE ≠ frunza sau Y.SV ≠
        frunza sau Y.SE ≠ frunza atunci
        RETURN NULL

    sfarsit daca
    daca Y.NV.info ≠ NULL atunci
        info=Y.NV.info
        nr=nr+1
    sfarsit daca
    daca Y.NE.info ≠ NULL atunci
        info=Y.NE.info
        nr=nr+1
    sfarsit daca
    daca Y.SE.info ≠ NULL atunci
        info=Y.SE.info
        nr=nr+1
    sfarsit daca
    daca Y.SV.info ≠ NULL atunci
        info=Y.SV.info
        nr=nr+1
    sfarsit daca
    daca nr>1 atunci info=NULL
RETURN info

PR_DELETE(T,P)
    Z=PR_SEARCH(T,P)
    daca Z.info = P atunci
        Z.info = NULL
        Y = Z.p
        ok = true
        cat timp Y≠NULL si ok
            Info = IS_MERGEABLE(Y)
            daca Info = NULL atunci
                ok = false
            altfel
                Y.info = Info
                Y.NV = Y.NE = Y.SV = Y.SE = NULL

```

```

        Z = Y
        Y = Y.p
        sfarsit daca
    sfarsit cat timp
altfel
    scrie("Nu exista P")
sfarsit daca
RETURN

```

Exemplu ștergem punctul $(50, 105)$ din arborele care conține punctele: $\{P_1 = (55, 67), P_2 = (80, 15), P_3 = (119, 58), P_4 = (41, 85), P_5 = (53, 117), P_6 = (50, 105)\}$ (fig. 11.11 și 11.12)

Observații

1. Forma unui PR-QuadTree nu depinde de ordinea în care sunt inserate punctele în arbore.
2. Dacă se inserează două puncte foarte apropiate este posibil să trebuiască efectuate numeroase spargeri ale blocurilor, până se ajunge la separarea celor două puncte, ceea ce conduce la creșterea în adâncime a arborelui și la introducerea unui număr semnificativ de noduri ce nu conțin nici un punct.

11.2.5 Înălțimea unui PR-QuadTree

Considerând d = distanța minimă între două puncte din blocul original și n dimensiunea laturii acestui bloc, atunci adâncimea h a PR-Quadtree-ului corespunzător este:

$$\log_4 n \leq h \leq \log_2 \left(\sqrt{2}n/d \right) + 1$$

Demonstrație: evident, dacă toate frunzele sunt la aceeași adâncime, $h = \log_4 n$.

Altfel: cele mai mici blocuri, adică cele mai adânci frunze, se obțin pentru separarea celor mai apropiate două puncte dintr-un cadran (fig. 11.13). Presupunem că aceste două puncte P_1 și P_2 se află la adâncimea h , părintele din care au provenit se afla la adâncimea $h - 1$ și avea dimensiunea laturii $L = n/2^{h-1}$. Distanța între P_1 și P_2 este d și

$$d \leq \sqrt{2}n/2^{h-1}$$

De aici rezultă

$$h - 1 \leq \log_2 \left(\sqrt{2}n/d \right) \Rightarrow h \leq \log_2 \left(\sqrt{(2)}n/d \right) + 1$$

Determinarea vecinilor: același algoritm ca și pentru Quadrees.

Exemplu de aplicație

Se consideră un bloc pătrat de dimensiune $2^n \times 2^n$ ce reprezintă o hartă, pe care se află amplasate m orașe. Acestea pot fi reprezentate cu ajutorul unui PR-arbore quad. Să se determine toate orașele aflate la distanța maximă d de un oraș dat P .

Idee de rezolvare: Se parcurge în preordine PR-arborele. Pentru fiecare nod curent X , se verifică dacă poate conține orașe la distanța cel mult d de P . Acest lucru poate fi determinat pe baza coordonatelor colțurilor blocului $X.top_left$ și $X.bottom_right$:

$$P.x - d \geq X.top_left.x \text{ și } P.y - d \geq X.top_left.y$$

$$\text{și } P.x + d \leq X.bottom_right.x \text{ și } P.y + d \leq X.bottom_right.y.$$

În caz afirmativ se continuă coborârea pe subarborele de rădăcină X , altfel se oprește căutarea în acel subarbore.

Un exemplu de astfel de căutarea este prezentat în figura 11.14.

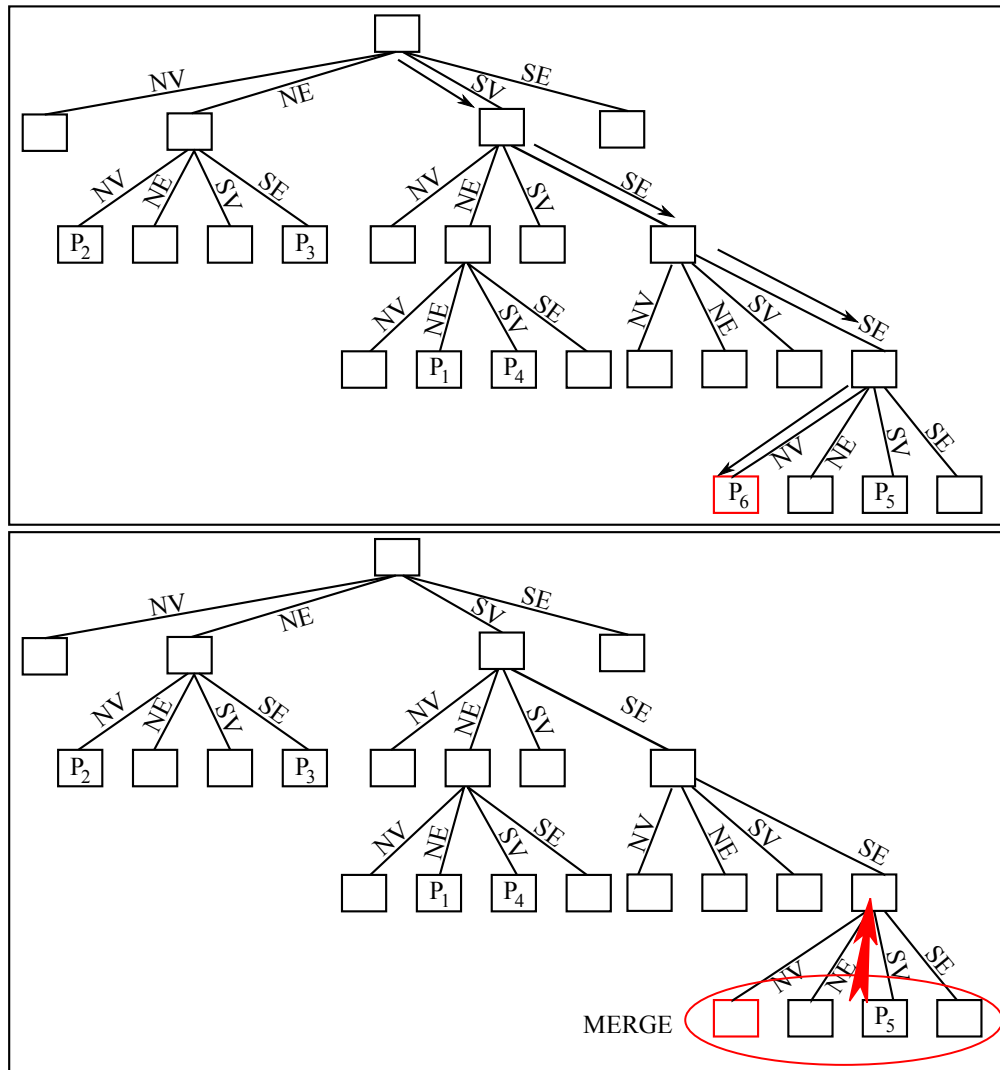


Figure 11.11: Ștergerea punctului $P = (50, 105)$ din arbore.

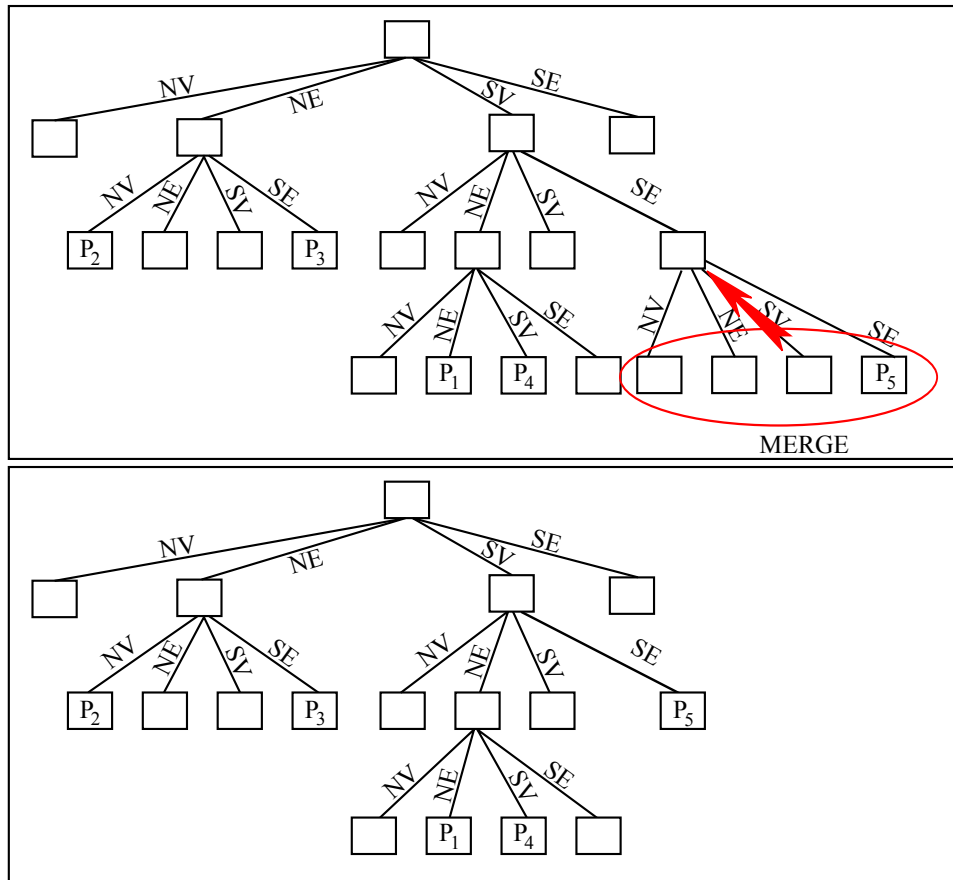
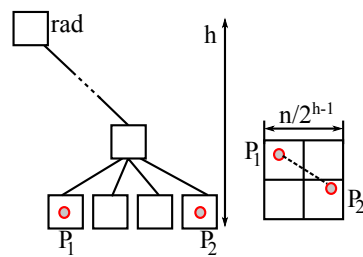
Figure 11.12: Ștergerea punctului $P = (50, 105)$ (continuare).

Figure 11.13: Cele mai apropiate două puncte de la cel mai de jos nivel.

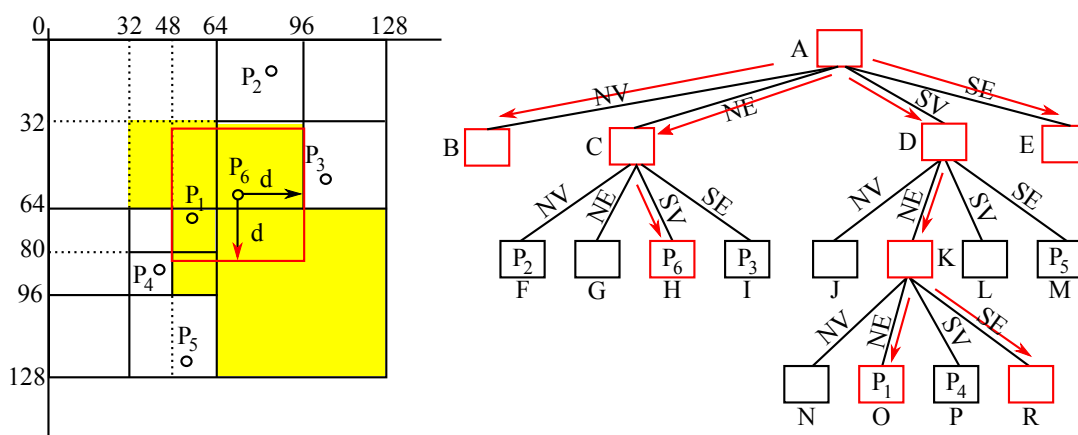


Figure 11.14: Căutarea punctelor aflate la distanța cel mult d față de un punct dat.

Bibliography

- [1] Cormen T.H, Leiserson C.E, Rivest R., Clifford S. *Intorduction to Algorithms*, third Edition, MIT Press 2009
- [2] Knuth D.E, *The Art of Computer Programming : fundamental algorithms*, third edition 1983
- [3] Weiss M.A., *Data Structures and Problem Solving Using C++*, second edition Pearson, Addison Wesley 2002
- [4] Weiss M.A., *Data Structures and Algorithm Analysis in C++*, fourth edition Pearson 2014
- [5] Barnett G., Del Tongo L., *Data structures and Algorithms*, 2008
- [6] Jakub Pachocki, Jakub Radoszewski, *Where to Use and How not to Use Polynomial String Hashing*, Olympiads in Informatics, Vilnius University, 2013
- [7] D. Eastlake, G. Fowler, K.-P. Vo, and L. Noll, “The fnv noncryptographic hash algorithm,” 2012. [Online]. Available: <http://tools.ietf.org/html/draft-eastlake-fnv-03>
- [8] Samet H., *An Overview of Quadtrees, Octrees and Related Hierarchical Data structures*, SpringerLink 1988
- [9] <https://dzone.com/articles/performance-of-array-vs-linked-list-on-modern-comp>
- [10] <https://baptiste-wicht.com/posts/2012/11/cpp-benchmark-vector-vs-list.html>
- [11] <https://www.codeproject.com/Articles/5425/An-In-Depth-Study-of-the-STL-Deque-Container>
- [12] <https://adamgold.github.io/posts/python-hash-tables-under-the-hood/#dictionary-resize>
- [13] <https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/hashtable.h>